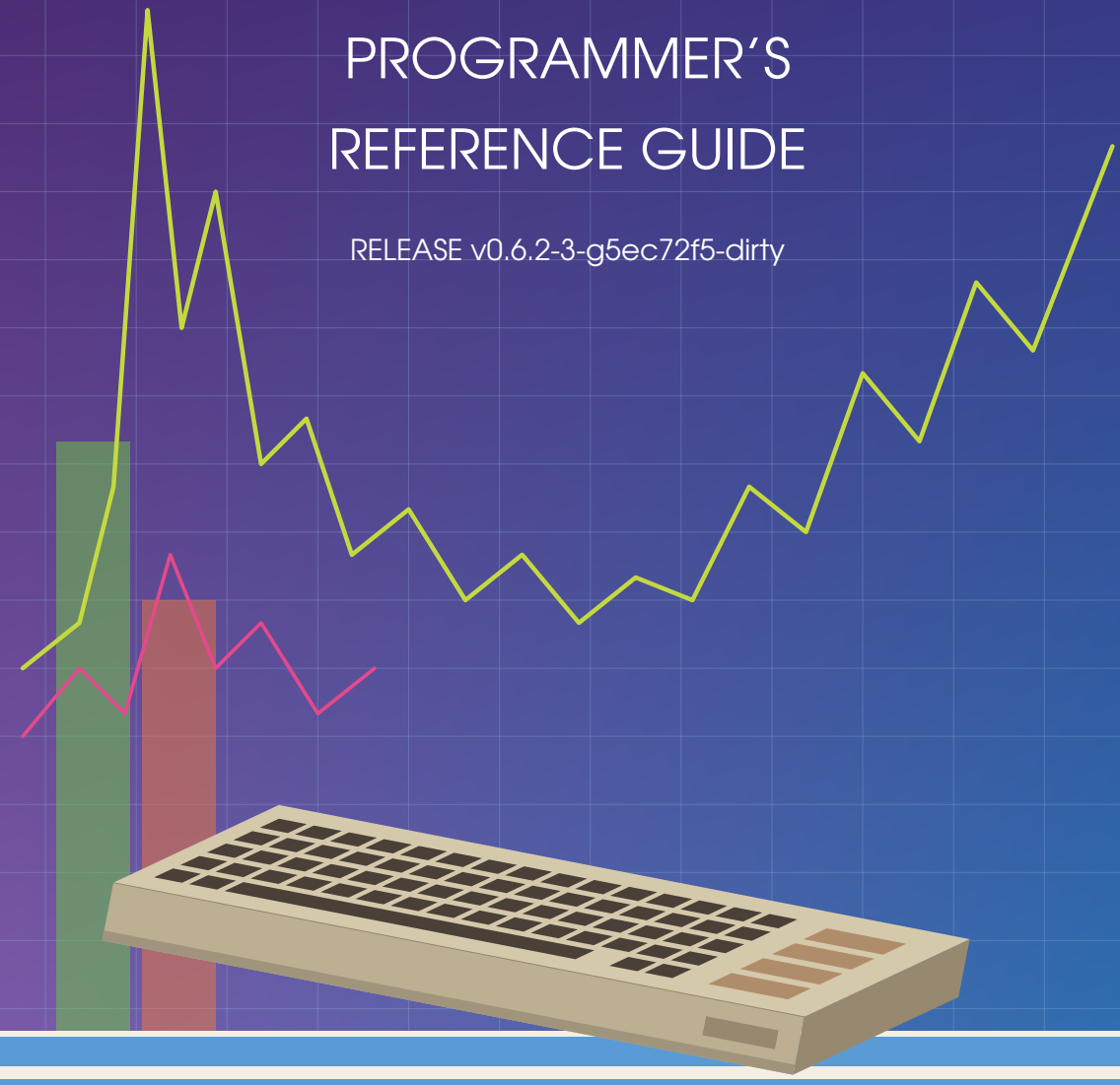
ULTIMATE SDK

PROGRAMMER'S REFERENCE GUIDE

RELEASE v0.6.2-3-g5ec72f5-dirty



ULTIMATE SDK

PROGRAMMER'S REFERENCE GUIDE

For Commodore 64 programs using the Ultimate Command Interface

First edition.

This guide documents the Ultimate SDK, a software development kit for the Commodore 64 and Ultimate hardware with the command interface.

Released under the MIT licence. Commodore, the Commodore logo and Commodore 64 are trademarks of their respective owners; this guide is not affiliated with or endorsed by them.

Contents

Introduction **vii**

1 GETTING STARTED **2**
What The SDK Requires . . . 2
Enable The Command Interface . . . 2
Choose A Binding . . . 2
Link Only The Services You Use . . . 6
Recover From Common Failures . . . 8

2 PROGRAMMING IN BASIC **10**
Two Ways To Install The Same Wedge . . . 10
Every BASIC Keyword By Example . . . 10
BASIC-specific Limits . . . 14

3 PROGRAMMING IN C **17**
How To Read The Examples . . . 17
Start And Discover The Machine . . . 18
Read SID Addresses Through Deprecated HWINFO . . . 20
Read And Change The Live Palette . . . 21
Use Ultimate 64 Turbo Control . . . 21
Composite A Software Vsprite . . . 23
Change And Read The Current Directory . . . 25
Open, Read And Close A File . . . 26
Ask What A File Is . . . 28
Rename And Copy A File . . . 29
Load And Save C64 Memory . . . 29
Mount A Disk Image . . . 30
Control The Machine And Its Drives . . . 31
Read And Set The Clock . . . 32
Move Bytes Through The REU . . . 33
Load And Play PCM Through Ultimate Audio . . . 35
Inspect Network Interfaces . . . 37

Use TCP And UDP Sockets . . . 38
Fetch A URL In One Call . . . 40
Build A Multi-step HTTP Request . . . 40
Build A Request Body . . . 42
Probe And Configure The Raw Transport . . . 45
Execute A Raw Command With And Without Arguments . . . 46
Read A Raw Multi-block Reply . . . 47
Decode And Inspect Raw Status . . . 48

4 PROGRAMMING IN ASSEMBLY 50
The Calling Convention . . . 50
A Native PRG Without C Startup . . . 51
Place The SDK's Working Memory . . . 53
Initialize, Detect And Describe The Machine . . . 54
Read And Change The Palette . . . 59
Use Turbo And Badline Control . . . 60
Composite A Software Vsprite . . . 61
Change And Read The Current Directory . . . 63
Open, Seek, Read, Write And Close Files . . . 65
Ask What A File Is . . . 67
Rename And Copy A File . . . 68
Load And Save C64 Memory . . . 69
Mount A Disk Image . . . 69
Control The Machine And Its Drives . . . 71
Read And Set The Clock . . . 73
Move Bytes Through The REU . . . 74
Load And Play PCM Through Ultimate Audio . . . 77
Inspect Network Interfaces . . . 79
Use TCP And UDP Sockets . . . 80
Use The HTTP Client . . . 82
Build A Request Body . . . 85
Probe And Configure The Raw Transport . . . 88
Raw Commands: Optional Arguments . . . 89
Read A Raw Multi-block Reply . . . 91

5 OTHER TOOLCHAINS 93
Build And Identify The Blob . . . 93
Call The Stable Jump Table . . . 94
Use The Parameter Block . . . 95
Call Audio And Vsprite Extensions . . . 96
Call The Blob From BASIC . . . 96
Choose A Different Base Address . . . 97

6 BASIC KEYWORD REFERENCE 100

Common Rules . . . 100
Raw Command Interface . . . 100
Target Constants . . . 105
Turbo Control . . . 107
Files And Directories . . . 108
RAM Expansion . . . 111

7 C API REFERENCE 115
Common Rules . . . 115
SDK Data Types . . . 115
Initialization And Discovery . . . 121
Capability Macros . . . 125
Palette, Turbo And Vsprite Graphics . . . 128
Directories And Files . . . 133
Disk Images And Clock . . . 141
Loading And Saving Memory . . . 144
RAM Expansion . . . 146
Ultimate Audio . . . 149
Machine And Drive Control . . . 154
Network Sockets . . . 157
HTTP Requests . . . 161
HTTP Request Bodies . . . 165
Errors And Diagnostics . . . 171
Low-Level UCI Transport . . . 172

8 ASSEMBLY API REFERENCE 181
Common Rules . . . 181
Initialization And Discovery . . . 181
Palette, Turbo And Vsprite Graphics . . . 185
Directories And Files . . . 191
Disk Images And Clock . . . 200
Loading And Saving Memory . . . 203
RAM Expansion . . . 206
Ultimate Audio . . . 210
Machine And Drive Control . . . 215
Networking . . . 219
HTTP Client . . . 225
HTTP Request Bodies . . . 229
Low-Level UCI Transport . . . 235

9 APPENDIX 243
Result Codes . . . 243

INTRODUCTION

The **ULTIMATE SDK** gives a C64 program one API for the Ultimate Command Interface, the UCI, and for supported features that use direct hardware registers: transfers through the RAM Expansion Unit, the REU; Ultimate 64 turbo control; and Ultimate Audio. It also includes a local software-vsprite compositor for C64 bitmaps. The language chapters cover every public call.

The implementation is 6502 assembly. cc65 links it as a library; ca65 callers use its assembly ABI, the Application Binary Interface described in Chapter 4; BASIC reaches a banked build through a wedge; other toolchains call the same object code through a standalone jump-table binary.

How to read this

Chapter 1 covers setup and chooses a binding. The language chapters demonstrate the services in complete programs; the three reference chapters specify every public BASIC, C and assembly entry independently. The appendix holds the result-code table.

Boxes titled OUTPUT reproduce text actually printed by the example. Boxes titled RESULT describe returned data or a hardware effect that the example does not print.

A word about the name

The library is called the Ultimate SDK rather than the Ultimate 64 SDK because the command interface predates the Ultimate 64. Programs probe targets with `ultimate_detect()` and test the REU and turbo registers separately.

CHAPTER 1

GETTING STARTED

- What the SDK requires
- Enabling the command interface
- Choosing and checking a binding
- Recovering from common failures

WHAT THE SDK REQUIRES

The SDK runs on a C64 with Ultimate hardware whose command interface is mapped at `$DF1B–$DF1F`. It is supported on the 1541 Ultimate-II and later products.

There is no blanket minimum firmware version. Targets were added over time, so programs call `ultimate_detect()` and test the capability they need. The palette is newer than firmware 3.15; HTTP appears in 3.15.

The control target predates the palette commands, so its presence is not a palette capability test. Call a palette function and handle `ULTIMATE_ERR_NOT_SUPPORTED`; the SDK returns that result when older firmware answers that it does not know the command.

ENABLE THE COMMAND INTERFACE

Open the Ultimate configuration menu and set `COMMAND INTERFACE` to `ENABLED`. The exact key used to open that menu depends on the product and its configuration, so the SDK does not prescribe one.

With the interface disabled, the five registers are not mapped and `ultimate_init()` returns `ULTIMATE_ERR_NO_DEVICE`.

REU transfers, Ultimate 64 turbo control and Ultimate Audio use their own registers, not the command interface. Each still depends on its corresponding machine setting. Software-vsprite drawing is local 6510 code and needs none of those settings.

CHOOSE A BINDING

BASIC V2	`make wedge`; load `src/basic/uci.prg`, or mount `src/basic/uci.crt`
cc65 C	`make lib`; include `ultimate.h` and link `bindings/cc65/build/ultimate.lib`
ca65 assembly	link the same library and include `bindings/asm/ultimate.inc`
other toolchains	`make blob`; load the standalone binary described in Chapter 5

BASIC check

Load and run the wedge. It installs itself, prints a banner and performs a `NEW`, so the BASIC program area is empty again afterwards:

```
LOAD"UCI",8
SEARCHING FOR UCI
LOADING
READY.
RUN
ULTIMATE UCI BASIC WEDGE
SDK v0.6.2-3-g5ec72f5-dirty
VIVA LA COMMODORE!
READY.
```

Now send a command. `UDIR` lists the Ultimate's current directory, and `UERR` reports how that command ended:

```
UDIR
DATA
BIG.BIN
README.TXT
READY.
PRINT UERR
0
READY.
```

The entry names depend on the machine and the medium; the printed result does not. A printed `0` is `ULTIMATE_OK`; `1` is `ULTIMATE_ERR_NO_DEVICE`. Together these two screens exercise wedge installation, a directory command and the shared result model.

C check

Build the library once before either of the two checks below:

```
make lib
```

RESULT

bindings/cc65/build/ultimate.lib is created.

This complete program initializes the SDK and reports any failure. Save it as `hello.c`:

```
#include <stdio.h>
#include <ultimate.h>

int main(void)
{
  uint8_t err = ultimate_init();

  if (err != ULTIMATE_OK)
    printf("ultimate: %s\n", ultimate_strerror(err));
  return err != ULTIMATE_OK;
}
```

It prints nothing on success. With UCI disabled, it prints:

OUTPUT

```
ULTIMATE: NO ULTIMATE FOUND
```

Build it from the repository root:

```
cl65 -t c64 -I include -o hello.prg hello.c \
    bindings/cc65/build/ultimate.lib
```

RESULT

hello.prg is created with no compiler errors.

Assembly check

This is the same check in ca65 assembly, complete enough to assemble, link and run. Save it as `hello.s`:

```
    .include "ultimate.inc"

    .forceimport __STARTUP__
    .export _main

CHROUT = $FFD2

    .code

_main:  jsr ultimate_init
    cmp #ULTIMATE_OK
    bne failed

    ldx #0
found:  lda msg_found,x
    beq done
    jsr CHROUT
    inx
    bne found

done:    rts

failed:  ldx #0
absent:  lda msg_absent,x
    beq done
    jsr CHROUT
    inx
    bne absent

    .rodata

msg_found:  .byte "ultimate found", 13, 0
msg_absent: .byte "no ultimate found", 13, 0
```

Build it from the repository root:

```
cl65 -t c64 --asm-include-dir bindings/asm \
    -o hello.prg hello.s bindings/cc65/build/ultimate.lib
```

OUTPUT

```
ULTIMATE FOUND
```

Three details in that listing matter. A program that leaves any of them out will not link, or will link and print nothing.

`.forceimport __STARTUP__` and the name `_main` pull in cc65's start-up code, which is the simplest way to get a running `.prg`. The SDK itself needs no C runtime; Chapter 4 covers linking without one.

`--asm-include-dir` is how `ultimate.inc` is found.

The message text is lowercase in the source and prints as capitals. ca65's C64 character map turns source `a`–`z` into the byte values the machine displays as `A`–`Z`.

The result convention is the one every entry point uses: `jsr`, then `cmp` against `ULTIMATE_OK`, then branch. Compare `A` straight away, before another instruction overwrites it.

LINK ONLY THE SERVICES YOU USE

Link `ultimate.lib` normally: both cc65 C and ca65 assembly select referenced archive objects and their dependencies automatically. No feature flag or separate service library is needed. Including a header does not include every service. Selection is per object/service, not per function: calling one turbo function includes the other functions in `turbo.o` too.

This complete program is `guide/examples/selective.c`. The listing is read directly from the source file compiled by the guide build:

```c
#include <ultimate.h>

int main(void)
{
    unsigned char color = 2;
    if (ultimate_turbo_available())
        color = 5;
    *(volatile unsigned char *)0xd020 = color;
    return 0;
}
```

From the repository root:

```
make lib
cl65 -t c64 -I include \
  -m guide/examples/selective-c.map \
  -o guide/examples/selective-c.prg \
  guide/examples/selective.c \
  bindings/cc65/build/ultimate.lib
```

Run the PRG from the Ultimate file browser or your usual PRG loader.

RESULT

The border becomes green when the turbo register is available, red otherwise. The program returns to BASIC without printing text or changing CPU speed.

Turbo availability reads a register and needs no `ultimate_init()`. Command-interface services such as files and palette access still need initialization. Do not remove required startup calls just to reduce size.

Open `guide/examples/selective-c.map`. Under **Modules list**, the only SDK member is `ultimate.lib(turbo.o)`. The other archive entries, `c64.lib(...)`, are the normal C runtime and compiler helpers, not extra SDK services. Chapter 4's native PRG walkthrough (page 51) builds the same effect without that runtime.

Read the map in three parts:

- **Modules list**: selected objects and their segment contributions. `Offs` is a segment offset, not an absolute C64 address.
- **Segment list**: final addresses and total sizes of code, data and BSS (uninitialized RAM, not stored in the PRG).
- **Imports list**: which object references each symbol. Follow these references to explain why an unexpected service was included.

Services bring their dependencies: file loading needs DOS and transport helpers; C calls may need argument adapters. Shared objects are included only once. Adding `ultimate_init()` adds initialization/identity code. The size of `ultimate.lib` on disk is not the SDK footprint in your PRG; fixed-address layouts can also leave padding in a PRG, as Boing does.

Program tip: Keep a map as your program grows. Link the archive, not every SDK object individually, and do not force-import unused services. A reference in a retained dispatcher or application object can keep a service linked even if your program never takes that branch at runtime. This is not whole-program dead-code analysis.

The standalone blob is still a complete SDK. BASIC retains the wedge's compiled command set regardless of which commands a BASIC program uses. Existing demo and example Makefiles recheck the SDK and emit maps. `make -C guide check-examples` builds these examples; host link tests check the selected objects, not execution on Ultimate

hardware.

RECOVER FROM COMMON FAILURES

No device

`ULTIMATE_ERR_NO_DEVICE` means the signature is absent. Check the Command Interface setting before assuming the hardware is missing.

An abandoned exchange

A program can stop while the firmware is holding command or reply state. `uci_abort()` releases it; `ultimate_init()` performs that recovery during startup. From BASIC, a bare `UCI` calls the same abort path:

```
UCI:PRINT UERR
0
READY.
```

This example demonstrates recovery only; it does not send a command.

A missing feature

Use `ultimate_detect()` for command-interface targets, `ultimate_reu_available()` for the REU, and `ultimate_turbo_available()` for turbo. Call `ultimate_audio_init()` and then `ultimate_audio_available()` for mapped Ultimate Audio. Do not infer capabilities from a model name.

A short buffer

`ULTIMATE_ERR_TRUNCATED` means an exchange completed but not all reply bytes fitted. Output lengths report the bytes retained, not the bytes discarded.

CHAPTER 2

PROGRAMMING IN BASIC

- How the two wedge builds differ
- The twenty-three keywords
- Clear examples of each service group
- BASIC-specific limits

TWO WAYS TO INSTALL THE SAME WEDGE

Chapter 1 shows the `uci.prg` installation. Its loader copies the SDK under BASIC ROM at `$A000` and the wedge to RAM at `$C000`, installs four BASIC vectors, then performs `NEW`. The normal BASIC program area remains available.

The `uci.crt` build carries the same runtime bytes in an 8K cartridge. It autostarts and its warm-start vector reinstalls the wedge after reset. While the cartridge remains mapped at `$8000–$9FFF`, BASIC's program area is 8K smaller.

Both builds contain the same wedge; only their loaders differ.

> **Careful:** The wedge owns BASIC's tokenise, list, statement-dispatch and expression vectors and does not chain previous owners. Do not combine it with another extension that needs any of those vectors unless that combination provides an explicit compatibility layer.

EVERY BASIC KEYWORD BY EXAMPLE

The wedge exposes twenty-three keywords. This section uses every one. Services without a dedicated keyword remain available through the raw `UCI` statement. Output below is from a representative successful run; model names, times, directory entries and REU size depend on the machine. When a result is fixed, it appears directly after `RUN` in the same screen.

Run a command and inspect every result

```
10 UCI UCTRL,1
20 PRINT "RESULT";UERR;" DEVICE";UDEV;" BYTES";ULEN
30 PRINT "STATUS: ";UST$
40 PRINT "DATA: ";UDAT$
50 FOR I=0 TO ULEN-1:PRINT UBYTE(I);" ";:NEXT:PRINT
RUN
RESULT 0 DEVICE 0 BYTES 14
STATUS: 00,OK
DATA: CONTROL TARGET
67 79 78 84 82 79 76 32 84 65 82 71 69 84
READY.
```

`UCI UCTRL,1` asks the control target to identify itself. `UERR` is the SDK result, `UDEV` is the target's raw status number, and `ULEN` is the retained reply length. `UST$` returns the status text, `UDAT$` returns up to 255 reply bytes as a string, and `UBYTE(N)` returns one retained

byte or zero when `N` is past the end.

Use every target constant

```
10 PRINT UDOS1,UDOS2,UNET,UCTRL,UIEC,UHTTP
RUN
1  2  3  4  5  6
READY.
```

The printed values are 1 through 6: the two DOS instances, network, control, SoftwareIEC and HTTP targets. DOS here is the Ultimate's own disk operating system, which is what serves files from its storage. Use these names as the first argument to `UCI`; do not copy their numeric values into programs.

Send a raw command with and without an optional argument

```
10 UCI UDOS1,38: PRINT UDAT$
20 UCI UDOS1,38,0: PRINT UDAT$
30 UCI UCTRL,40: PRINT UDAT$
40 UCI UCTRL,40,0: PRINT UDAT$
50 IF UERR THEN PRINT "COMMAND FAILED";UDEV
```

Command 38 is `DOS_CMD_GET_TIME`. Line 10 omits its optional format byte; line 20 supplies it explicitly. Command 40 is `CTRL_CMD_GET_HWINFO`; the firmware documentation marks it deprecated. Lines 30 and 40 show its omitted and explicit model selector only to make the wire format clear. Both commands default an omitted argument to zero. A number normally contributes one byte; known wide arguments are widened automatically.

Read SID addresses through deprecated HWINFO

The SID is the C64's Sound Interface Device, its sound chip. An Ultimate 64 can present more than one, and this reports where each one is mapped.

```
5 REM LEGACY: HWINFO IS DEPRECATED
10 UCI UCTRL,40,1
20 IF UERR THEN PRINT "SID INFO FAILED";UERR:END
30 PRINT "SID COUNT";UBYTE(0)
40 FOR I=0 TO UBYTE(0)-1
50 A=1+5*I
60 PRINT "SID";I+1;UBYTE(A)+256*UBYTE(A+1)
70 NEXT
RUN
SID COUNT 4
SID 1 54272
SID 2 54272
SID 3 54272
SID 4 54272
READY.
```

The explicit selector 1 requests SID information through a command the firmware documentation marks deprecated. Firmware still implements it but may remove it, and no replacement C64-side UCI command is currently published. Byte zero is the record count. Each five-byte record starts with the primary address, least-significant byte first; line 60 reconstructs that address. Decimal 54272 is `$D400`. The following two bytes are the optional secondary address and the fifth is a raw type value, not a 6581 or 8580 model identifier.

Force raw argument widths

```
10 UCI UDOS1,240,UW(4660),UL(65536)
20 FOR I=0 TO ULEN-1:PRINT UBYTE(I);" ";:NEXT:PRINT
RUN
1 240 52 18 0 0 1 0
READY.
```

Command 240 echoes the target byte, command byte and arguments. The leading 1 and 240 confirm the framing. `UW(4660)` sends 52,18 and `UL(65536)` sends 0,0,1,0: both are little-endian. These overrides are for commands newer than the wedge's shape table or deliberately raw byte layouts.

Abort an unfinished exchange

```
UCI:PRINT UERR
0
READY.
```

A bare `UCI` sends no command. It aborts any exchange left unfinished by a stopped program and returns the interface to idle; `UERR` reports whether the recovery succeeded.

Set and read turbo

```
10 UTURBO 3: IF UERR THEN PRINT "TURBO UNAVAILABLE":END
20 PRINT "SPEED INDEX";UTURBO
RUN
SPEED INDEX 3
READY.
```

The statement form sets an index; the function form reads it. Portable indices 0 through 3 select 1, 2, 3 and 4MHz. A read of 255 means the turbo registers are unavailable.

Load a PRG with and without the optional address

A PRG is a Commodore program file: two bytes giving the address it loads at, then the bytes themselves.

```
10 ULOAD "GAME.PRG": IF UERR THEN END
20 ULOAD "FONT.PRG",49152: IF UERR THEN END
```

Line 10 consumes the PRG's two-byte header and loads at the address stored there. Line 20 still consumes the header but loads the remaining bytes at 49152. Dedicated filename keywords copy at most 63 bytes and add a terminator.

Load and save raw memory

```
10 UBLOAD "FONT.BIN",49152,2048: IF UERR THEN END
20 USAVE "SCREEN.BIN",1024,1000: IF UERR THEN END
```

`UBLOAD` reads at most 2048 raw bytes into address 49152; it consumes no header. `USAVE` replaces `SCREEN.BIN` with 1000 bytes beginning at C64 address 1024.

List the current directory

```
UDIR
```

`UDIR` takes no parameters. It opens the current Ultimate directory, prints every entry and leaves `UERR` at zero when the normal end marker is reached.

Copy memory to and from the REU

```
10 R=UREU:PRINT "REU BANKS";R:IF R=0 THEN END
20 USTASH 1024,0,1000:IF UERR THEN END
30 PRINT CHR$(147)
40 UFETCH 1024,0,1000:IF UERR THEN END
```

`UREU` returns the size of the RAM expansion in 64K banks. `USTASH` copies the 1000-byte C64 text screen to REU address zero; `UFETCH` copies it back. The expansion moves the bytes itself with the processor halted, which is called direct memory access, or DMA. The REU address is 24 bits. Keep the address plus length within `UREU` times 65536; a DMA length of zero means 65536 bytes.

BASIC-SPECIFIC LIMITS

A statement extension cannot follow THEN directly

BASIC V2's `IF` path enters the ROM statement dispatcher instead of the wedge vector, so this is rejected:

```
10 IF X = 1 THEN ULOAD "GAME.PRG"
RUN
?SYNTAX ERROR IN 10
READY.
```

Branch to a line containing the extension statement:

```
10 IF X = 1 THEN 100
20 END
100 ULOAD "GAME.PRG"
```

This demonstrates the workaround. Extension functions are unaffected, so `IF UERR THEN 100` remains valid.

Reply storage is finite

The wedge retains 256 reply bytes. `UDAT$` can expose only 255 because that is BASIC's maximum string length; `UBYTE(0)` through `UBYTE(255)` can inspect the retained bytes individually. A longer raw reply sets `UERR` to `ULTIMATE_ERR_TRUNCATED`.

CHAPTER 3

PROGRAMMING IN C

- Startup, discovery and diagnostics
- Files, disk images, the clock and the REU
- Machine, drive, graphics and PCM audio control
- Network and HTTP services
- The complete low-level UCI API

HOW TO READ THE EXAMPLES

When linking with cc65, `ultimate.lib` includes only referenced service objects, their dependencies and any required C adapters. Including a header does not include every service. Selection is per object, not per function: other routines within a selected service and shared state remain. The same selective linking works for ca65 assembly. Other toolchains using the standalone blob receive the complete SDK; BASIC retains the wedge's compiled command set.

The selective-linking walkthrough (page 6) gives a complete small C program, the build command with a map file, and how to check exactly what was linked.

Every public function from `ultimate.h` and `uci.h` appears below. Each listing shows one complete operation within a program; calls share state only when the text says so. The guide build compiles every displayed call form. Unless a listing shows otherwise, it runs after including `stdio.h`, `string.h`, `ultimate.h` and `uci.h`. Result-returning calls use this pattern:

```
uint8_t err = ultimate_init();
if (err != ULTIMATE_OK)
  printf("init: %s\n", ultimate_strerror(err));
```

The call prints nothing on success. With UCI disabled, it prints:

OUTPUT

```
INIT: NO ULTIMATE FOUND
```

An OUTPUT box contains only text the listing actually prints on the C64. A RESULT box describes a return value, memory change or visible hardware effect. Captured output below came from an Ultimate 64 Elite; model names, paths, directory entries, addresses and network replies vary with the machine and the files or servers used.

The C64 holds text in PETSCII, the PET Standard Code of Information Interchange. The Ultimate speaks ASCII. The two codes agree on the digits and on the uppercase letters and disagree on case, and cc65 maps lowercase source letters to the uppercase byte values the two share. Path, filename and header literals therefore appear in lowercase source even though the firmware receives uppercase text.

Zero is success. Appendix A lists the remaining results. Include `ultimate.h` for the service API; include `uci.h` only for the raw transport examples at the end of this chapter.

START AND DISCOVER THE MACHINE

```
int main(void)
{
  ultimate_capabilities caps;
  char text[64];
  uint16_t length;
  uint8_t err = ultimate_init();

  if (err != ULTIMATE_OK || !ultimate_available()) return 1;
  err = ultimate_detect(&caps);
  if (err == ULTIMATE_OK && ultimate_has_http(&caps))
    puts("http target present");

  err = ultimate_identify(UCI_TARGET_DOS1,
      text, sizeof text, &length);
  if (err == ULTIMATE_OK) printf("dos: %s\n", text);
  err = ultimate_identify(UCI_TARGET_DOS1,
      text, sizeof text, NULL);

  err = ultimate_get_model(text, sizeof text, &length);
  if (err == ULTIMATE_OK)
    printf("model received: %u bytes\n", length);
  err = ultimate_get_model(text, sizeof text, NULL);
  return err != ULTIMATE_OK;
}
```

OUTPUT

```
HTTP TARGET PRESENT
DOS: ULTIMATE-II DOS V1.2
MODEL RECEIVED: 17 BYTES
```

`ultimate_init()` performs the required startup check; `ultimate_available()` reads the resulting SDK state without touching hardware. `ultimate_detect()` fills `caps`; use the `ultimate_has_*()` macros to test targets. The paired identify and model calls show the optional final parameter: pass `&length` to receive the stored string length, or `NULL` when only the terminated string matters. The firmware documentation marks the command behind `ultimate_get_model()` deprecated; the SDK retains this existing wrapper for compatibility.

Report an error and its raw device code

```
if (err != ULTIMATE_OK) {
  printf("%s (device %u)\n",
    ultimate_strerror(err), ultimate_device_code());
}
```

RESULT

The message names the SDK error and includes the last raw device status. When the target supplied no numeric status, that value is `UCI_DEVICE_CODE_NONE` (65535).

`ultimate_strerror()` returns stable SDK text. `ultimate_device_code()` returns the last target-specific numeric status for diagnostics; do not use that firmware-specific number as application control flow.

READ SID ADDRESSES THROUGH DEPRECATED HWINFO

The SID is the C64's sound chip, the Sound Interface Device. An Ultimate 64 can present more than one of them, and this call reports where each one is mapped.

```
ultimate_sid_info info;
uint8_t i;

err = ultimate_legacy_get_sid_info(&info);
if (err == ULTIMATE_ERR_NOT_SUPPORTED)
  puts("legacy sid query unavailable");
else if (err == ULTIMATE_OK)
  for (i = 0; i < info.count; ++i)
    printf("sid %u: $%04x type $%02x\n",
      (uint8_t)(i + 1), info.sid[i].primary_address,
      info.sid[i].type);
```

OUTPUT

```
SID 1: $D400 TYPE $83
SID 2: $D400 TYPE $83
SID 3: $D400 TYPE $85
SID 4: $D400 TYPE $85
```

Firmware without that command instead prints `LEGACY SID QUERY UNAVAILABLE`.

Firmware still implements this query, but its documentation marks `CTRL_CMD_GET_HWINFO` deprecated and publishes no replacement C64-side UCI command. The function is therefore named `ultimate_legacy_get_sid_info()`; always handle `ULTIMATE_ERR_NOT_SUPPORTED` in case the command is removed.

The call fetches the count and every configured address in one operation. Each record also has a `secondary_address`; zero means there is no second mapping. The type byte is retained exactly as the firmware returns it. It distinguishes current SID implementations and mapping modes, not 6581 from 8580, so do not use it as a chip model. Check the result before reading the records.

READ AND CHANGE THE LIVE PALETTE

```
uint8_t palette[ULTIMATE_PALETTE_BYTES];
uint8_t err;

err = ultimate_palette_get(palette);
if (err == ULTIMATE_OK) {
  palette[0] = 0;
  palette[1] = 0;
  palette[2] = 0;
  err = ultimate_palette_set(palette);
}

err = ultimate_palette_set_color(6, 255, 0, 0);
err = ultimate_palette_reset();
```

RESULT

SET COLOUR 6 TO RED: ULTIMATE_OK
RESTORE BUILT-IN PALETTE: ULTIMATE_OK

`ultimate_palette_get()` reads all sixteen RGB triples and `ultimate_palette_set()` writes all of them. The single-colour call sets index 6 to red. `ultimate_palette_reset()` restores the built-in palette; none of these calls writes flash or a palette file. On firmware without the palette commands, each call returns `ULTIMATE_ERR_NOT_SUPPORTED`; test the result before using data from `ultimate_palette_get()`.

USE ULTIMATE 64 TURBO CONTROL

```
uint8_t speed;
uint8_t err;

if (ultimate_turbo_available()) {
  speed = ultimate_turbo_get();
  err = ultimate_turbo_set(U64_SPEED_4MHZ);
  err = ultimate_turbo_badlines(0);
  err = ultimate_turbo_badlines(1);
  err = ultimate_turbo_set(speed);
}
```

RESULT

```
AVAILABLE = 1
PREVIOUS SPEED = U64_SPEED_1MHZ
SET 4 MHZ, THEN RESTORE SPEED AND BADLINES: ULTIMATE_OK
```

Every portable speed and the machine-specific maximum use the same call:

```
err = ultimate_turbo_set(U64_SPEED_1MHZ);
err = ultimate_turbo_set(U64_SPEED_2MHZ);
err = ultimate_turbo_set(U64_SPEED_3MHZ);
err = ultimate_turbo_set(U64_SPEED_4MHZ);
err = ultimate_turbo_set(U64_SPEED_MAX);
```

RESULT

The first four constants mean the same speed on every Ultimate 64. MAX selects the fastest index implemented by this machine.

`ultimate_turbo_available()` is the capability check and `ultimate_turbo_get()` reads the index. The set call changes only speed. The two badline calls explicitly show both forms: zero disables badlines and non-zero restores them. The final set restores the speed read at entry.

COMPOSITE A SOFTWARE VSPRITE

`ultimate_vsprite_draw()` draws a rectangular image directly into a standard 40-column C64 bitmap. It is local 6510 code: it works without UCI or Ultimate hardware.

This example draws a two-byte-wide, eight-line masked image. Image and mask bytes are column-major: every line of byte column zero comes first, followed by every line of byte column one.

```
static const uint8_t image[16] = {
  0x00,0x18,0x3c,0x7e,0x7e,0x3c,0x18,0x00,
  0x00,0x18,0x3c,0x7e,0x7e,0x3c,0x18,0x00
};
static const uint8_t mask[16] = {
  0xff,0xe7,0xc3,0x81,0x81,0xc3,0xe7,0xff,
  0xff,0xe7,0xc3,0x81,0x81,0xc3,0xe7,0xff
};
ultimate_vsprite sprite = {
  (uint8_t *)0x6000, image, mask, NULL,
  12, 72, 2, 8, 0, VSPRITE_F_MASKED
};

err = ultimate_vsprite_draw(&sprite);
```

RESULT

The 16 source bytes are composited at cell column 12, raster line 72.

The bitmap pointer is its 8K base. `x` is a bitmap byte and screen-cell column from 0 to 39, not a pixel coordinate; `y` is a raster line from 0 to 199. Width is measured in bitmap byte columns and height in raster lines. The SDK does no clipping: the complete rectangle must fit.

Flags	Source layout	Operation
`0`	column-major image	destination OR source
`VSPRITE_F_MASKED`	image and mask	(destination AND mask) OR source
`VSPRITE_F_COPY`	another C64 bitmap	copy the same rectangle; useful for restoring a clean background
`VSPRITE_F_COLOR`	column-major image	OR the image and write `color` to screen cells containing a non-zero source byte

Starting from the descriptor above, these calls exercise the other three operations:

```
sprite.flags = 0; /* OR */
sprite.mask = NULL;
err = ultimate_vsprite_draw(&sprite);

sprite.flags = VSPRITE_F_COPY; /* restore */
sprite.source = (uint8_t *)0x4000;
err = ultimate_vsprite_draw(&sprite);

sprite.flags = VSPRITE_F_COLOR; /* OR and recolor */
sprite.source = image;
sprite.screen = (uint8_t *)0x4400;
sprite.color = 6;
err = ultimate_vsprite_draw(&sprite);
```

RESULT

OR adds source bits; COPY restores the same rectangle from another bitmap; COLOR adds source bits and writes screen value 6 to touched cells. The first listing demonstrated MASKED.

For color mode, set `screen` to the 1K screen base. Color mode cannot be combined with masked or copy mode. Mask and screen may be `NULL` when their modes do not use them. A null required pointer, zero size, invalid flag combination or out-of-bounds rectangle returns `ULTIMATE_ERR_INVALID_ARGUMENT`.

The routine patches its own instructions for speed, so it must execute from RAM and is not re-entrant. Movement, sub-byte image shifting, background policy, buffering and frame timing belong to the caller; the `demos/vsprites` and `demos/boing` programs show those two different policies using this same primitive.

Program tip: For moving objects, save or copy the clean background under last frame's rectangles, restore those rectangles, update every position, then draw the new frame. Keeping restoration separate prevents old pixels from trailing behind the objects.

CHANGE AND READ THE CURRENT DIRECTORY

```
char path[128];
uint16_t length;
uint8_t err;

err = ultimate_chdir("/USB1/GAMES");
err = ultimate_getpath(path, sizeof path, &length);
err = ultimate_getpath(path, sizeof path, NULL);
```

RESULT

path = "/USB1/GAMES"
length = 11

The change-directory call selects an absolute or relative firmware path. The paired getpath calls show its optional output: receive the stored length through `&length`, or pass `NULL` when the terminated path is enough.

Read every directory entry

```
char name[64];
uint8_t attrib;
uint8_t err = ultimate_opendir();

while (err == ULTIMATE_OK) {
  err = ultimate_readdir(name, sizeof name, &attrib);
  if (err == ULTIMATE_OK) puts(name);
}
if (err == ULTIMATE_END) err = ULTIMATE_OK;
```

OUTPUT

```
DATA
BIG.BIN
README.TXT
```

`ultimate_opendir()` starts one live exchange and each `ultimate_readdir()` releases one entry. `ULTIMATE_END` is normal. Issue no unrelated call inside the loop.

When attributes are not needed, pass the documented optional value:

```
err = ultimate_readdir(name, sizeof name, NULL);
```

RESULT

name receives the entry; no attribute byte is stored.

If a program stops the walk before `ULTIMATE_END`, call `uci_abort()` before issuing another command.

Make a directory, and return to the configured one

```
err = ultimate_mkdir("SAVES");
err = ultimate_home();
```

RESULT

SAVES is created inside the current directory.

`ultimate_mkdir()` creates one directory in the current one. `ultimate_home()` changes to the directory the machine's owner configured as its home. Firmware that has no configured home returns `ULTIMATE_ERR_NOT_SUPPORTED`.

OPEN, READ AND CLOSE A FILE

```
uint8_t buf[128];
uint16_t got;
uint8_t err;
uint8_t close_err;

err = ultimate_open("README.TXT", DOS_FA_READ);
if (err == ULTIMATE_OK) {
  err = ultimate_seek(0);
  if (err == ULTIMATE_OK)
    err = ultimate_read(buf, sizeof buf, &got);
  close_err = ultimate_close();
  if (err == ULTIMATE_OK) err = close_err;
}
```

RESULT

got = 128
err = ULTIMATE_OK

`ultimate_open()` establishes the firmware's current file. `ultimate_seek()` selects an absolute position, `ultimate_read()` reads from it, and `ultimate_close()` releases the same file. A short read is end-of-file, not an error; `got` is the number stored.

The four open bits combine into the common policies below:

Call	Policy
`ultimate_open(name, DOS_FA_READ)`	open an existing file for reading
`ultimate_open(name, DOS_FA_WRITE)`	open an existing file for writing without truncating it
`ultimate_open(name, DOS_FA_WRITE | DOS_FA_CREATE_NEW)`	create only when the name does not exist
`ultimate_open(name, DOS_FA_WRITE | DOS_FA_CREATE_ALWAYS)`	create or replace the file
`ultimate_open(name, DOS_FA_READ | DOS_FA_WRITE)`	update an existing file and read it back

The byte-count output is optional:

```
err = ultimate_read(buf, sizeof buf, NULL);
```

RESULT

buf receives bytes and the file advances; the count is discarded.

Create, write and delete a file

```
static const uint8_t message[] = {0x4f,0x4b,13};

err = ultimate_open("RESULT.TXT",
    DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE);
if (err == ULTIMATE_OK) {
  err = ultimate_write(message, sizeof message);
  close_err = ultimate_close();
  if (err == ULTIMATE_OK) err = close_err;
}
err = ultimate_delete("RESULT.TXT");
```

RESULT

RESULT.TXT: WROTE 3 BYTES, CLOSED, DELETED

The open flags create or replace the firmware-side file. `ultimate_write()` appends the specified bytes at its current position; `ultimate_delete()` removes the named file after it is closed.

ASK WHAT A FILE IS

```
ultimate_fileinfo info;
uint16_t year;

err = ultimate_stat("README.TXT", &info);
if (err == ULTIMATE_OK) {
  year = 1980 + (info.date » 9);
  printf("%s: %lu bytes, %u\n",
    info.name, info.size, year);
}
```

OUTPUT

```
README.TXT: 1024 BYTES, 2026
```

`ultimate_stat()` takes a name in the current directory. `ultimate_fstat()` answers the same question about the file `ultimate_open()` left open, and takes no name:

```
err = ultimate_open("README.TXT", DOS_FA_READ);
if (err == ULTIMATE_OK) {
  err = ultimate_fstat(&info);
  ultimate_close();
}
```

RESULT

info describes the open file.

The reply has this layout on the wire and is received straight into the structure:

`size`	length in bytes
`date`	year less 1980 in bits 15–9, month in 8–5, day in 4–0
`time`	hour in 15–11, minute in 10–5, two-second units in 4–0
`extension`	three bytes, space padded, with no terminator
`attrib`	the `DOS_ATTR_*` bits, as `ultimate_readdir()` reports
`name`	the name, NUL-terminated by the SDK

`date` and `time` are the packed forms used by FAT, the File Allocation Table filesystem the Ultimate's storage uses, and they are passed through without conversion. The example above unpacks the year; the other fields come out the same way, with a shift and a mask.

A reply shorter than the fixed header is `ULTIMATE_ERR_PROTOCOL`, because a size read out of half of its four bytes would look like a real answer.

RENAME AND COPY A FILE

```
err = ultimate_rename("OLD.TXT", "NEW.TXT");
err = ultimate_copy("NEW.TXT", "/USB1/BACKUP");
```

RESULT

OLD.TXT becomes NEW.TXT; BACKUP/NEW.TXT is a copy of it.

Rename uses two names in the current directory. Copy takes a source there and a destination path, preserving the source filename. The Ultimate performs the copy itself, so no bytes pass through the C64 and a large file costs no more C64 memory than a small one.

LOAD AND SAVE C64 MEMORY

```
uint16_t end;

err = ultimate_load("GAME.PRG", 0);
end = ultimate_last_end();

err = ultimate_load("FONT.PRG", 0xC000);
end = ultimate_last_end();

err = ultimate_bload("FONT.BIN", 0xC000, 2048);
err = ultimate_save("SCREEN.BIN", 0x0400, 1000);
```

RESULT

GAME.PRG END = address after loaded data
FONT.BIN: 2048 bytes loaded at $C000
SCREEN.BIN: 1000 bytes saved

The first `ultimate_load()` uses the PRG header address; the second shows the explicit-address form. Both consume the header, and `ultimate_last_end()` returns the address following the last byte. `ultimate_bload()` reads bounded raw bytes without a header. `ultimate_save()` replaces a file with the named C64 memory range.

MOUNT A DISK IMAGE

The Ultimate emulates disk drives, and mounting puts a disk image file into one of them. The drive is named by the number it answers to on the serial bus, 8 or 9, rather than by a slot. That bus is the IEC bus, after the International Electrotechnical Commission, and the number is the one a BASIC program gives in `LOAD"NAME",8`.

```
err = ultimate_mount(8, "GAMES/PITFALL.D64");
err = ultimate_swap(8);
err = ultimate_unmount(8);

err = ultimate_mount(ULTIMATE_DRIVE_LAST, "DEMO.D64");
```

RESULT

PITFALL.D64 is mounted in the drive answering as device 8.

The firmware picks the image type from the extension. `.D64`, `.D71` and `.D81` hold the sectors of a 1541, 1571 and 1581 disk; `.G64` and `.G71` hold the raw track data that a copy-protected disk needs. Any other extension is refused.

`ultimate_swap()` steps to the next image of a multi-image set, the way the Ultimate's own menu does.

`ULTIMATE_DRIVE_LAST` is zero, and asks for whichever drive was mounted into last, or drive A when nothing has been. A program that uses it works without knowing how its user configured the machine, and `ultimate_drive_info()` reports the numbers actually in use.

A drive that is switched off does not exist as far as these calls are concerned; mounting into one returns `ULTIMATE_ERR_DEVICE`. The next section switches drives on.

CONTROL THE MACHINE AND ITS DRIVES

```
ultimate_drives drives;
uint8_t running;
uint8_t i;

err = ultimate_drive_info(&drives);
if (err == ULTIMATE_OK)
  for (i = 0; i < drives.count; ++i)
    printf("type %u device %u power %u\n",
      drives.drive[i].type, drives.drive[i].device,
      drives.drive[i].power);

err = ultimate_drive_enable(ULTIMATE_DRIVE_A, 1);
err = ultimate_drive_power(ULTIMATE_DRIVE_A, &running);
```

OUTPUT

```
TYPE 0 DEVICE 8 POWER 1
TYPE 2 DEVICE 9 POWER 1
```

`ultimate_drive_info()` fills in every drive the machine has. `type` is a `CTRL_DRVTYPE_*` code: 0 for a 1541, 1 for a 1571, 2 for a 1581, and `$0F` for SoftwareIEC. `device` is the number to give `ultimate_mount()`. `power` is 1 while the drive is running.

`ultimate_drive_enable()` switches a drive on with a non-zero second argument and off with zero. `ultimate_drive_power()` reports the same state through a pointer, which may not be `NULL`.

> *Note:* Two numbering schemes meet here. `ultimate_drive_enable()` and `ultimate_drive_power()` take `ULTIMATE_DRIVE_A` or `ULTIMATE_DRIVE_B`, which name a physical drive. `ultimate_mount()` takes the device number the drive answers to on the IEC bus. `ultimate_drive_info()` connects the two: each record carries both.

Reset the machine, or open its menu

```
err = ultimate_freeze();
err = ultimate_reboot();
/* nothing after this line runs */
```

Careful: `ultimate_reboot()` resets the C64. The program that called it stops existing part way through the call, so no line after it runs. `ultimate_freeze()` stops the C64 and shows the Ultimate's menu. It returns when whoever is at the keyboard leaves that menu, which can be minutes.

Read the RAM disk layout

```
uint8_t ramdisk[CTRL_RAMDISK_BYTES];

err = ultimate_ramdisk_info(ramdisk);
```

RESULT

ramdisk holds four two-byte records.

The reply is `CTRL_RAMDISK_DRIVES` records of a drive type code and a size in 64K units, so the buffer must hold `CTRL_RAMDISK_BYTES`. A type of zero is a slot with nothing in it. These are the RAM disks GEOS MegaPatch uses.

READ AND SET THE CLOCK

The Ultimate has a battery-backed clock. It hands the time back as text, because text is what the firmware formats; nothing in the SDK parses it back into numbers.

```
char now[ULTIMATE_TIME_BUFFER];
uint16_t length;

err = ultimate_get_time(ULTIMATE_TIME_PLAIN,
    now, sizeof now, &length);
if (err == ULTIMATE_OK) puts(now);

err = ultimate_get_time(ULTIMATE_TIME_WEEKDAY,
    now, sizeof now, NULL);
if (err == ULTIMATE_OK) puts(now);
```

OUTPUT

```
2026/09/04 17:47:45
FRI 2026/09/04 17:47:45
```

`ULTIMATE_TIME_PLAIN` asks for the date and time; `ULTIMATE_TIME_WEEKDAY` puts the weekday in front of it. `ULTIMATE_TIME_BUFFER` is a buffer size that fits either form. The final parameter is optional in the usual way. A buffer too small for the answer still returns `ULTIMATE_OK`, with the part that fitted, as `ultimate_getpath()` does.

Set it

```
err = ultimate_set_time(126, 8, 22, 14, 30, 0);
```

RESULT

The clock is set to 2026/08/22 14:30:00.

Setting takes numbers rather than text, in the order year, month, day, hour, minute, second. The year is the year less 1900, so 2026 is 126. That is the firmware's own encoding. The other five fields are the numbers they appear to be.

MOVE BYTES THROUGH THE REU

```
uint16_t banks;

if (ultimate_reu_available()) {
  banks = ultimate_reu_size();
  err = ultimate_reu_stash(0x0400, 0, 1000);
  err = ultimate_reu_fetch(0x0400, 0, 1000);
}
```

RESULT

banks = installed REU size in 64K banks
1000 bytes stashed and fetched: ULTIMATE_OK

Availability is separate from size, which is returned in 64K banks. Stash copies C64 RAM to the REU; fetch copies the same range back. A DMA length of zero means 65536 bytes.

> **Program tip:** Treat the REU as a fast asset cache, not as ordinary addressable RAM. Load a level, music bank or animation frames into it once, then fetch only the piece needed for the current room or frame. It also makes a useful clean-screen or undo buffer because stash and fetch finish before they return.

Move the current file directly to and from the REU

File to REU

```
err = ultimate_open("ASSETS.BIN", DOS_FA_READ);
if (err == ULTIMATE_OK) {
  err = ultimate_seek(0);
  if (err == ULTIMATE_OK)
    err = ultimate_reu_load(0, 65536UL);
  ultimate_close();
}
```

> **RESULT**
>
> ASSETS.BIN TO REU $000000: 65536 BYTES

REU to file

```
err = ultimate_open("CAPTURE.BIN",
    DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE);
if (err == ULTIMATE_OK) {
  err = ultimate_reu_save(0, 65536UL);
  ultimate_close();
}
```

> **RESULT**
>
> REU $000000 TO CAPTURE.BIN: 65536 BYTES

These functions take no filename or file handle. The first flow opens the firmware's current DOS file, selects its first byte, then copies it to REU address zero. The second flow makes a different current file and copies REU bytes into it. Close the same firmware-side file after either transfer.

LOAD AND PLAY PCM THROUGH ULTIMATE AUDIO

Ultimate Audio plays PCM from the REU window in the background; it is not SID sample playback. Enable `MAP ULTIMATE AUDIO $DF20-DFFF` in the machine's settings, then probe it before touching a voice.

This example loads a PCM WAV directly from Ultimate storage into REU address `$004000`, configures channel zero and starts it. The PCM bytes never pass through C64 RAM.

```
ultimate_audio_voice voice = {
  0, 0, UA_VOLUME_MAX, UA_PAN_CENTER,
  0, 0, 0, 0, 1
};
uint8_t version;

err = ultimate_audio_init();
if (err == ULTIMATE_OK && ultimate_audio_available()) {
  version = ultimate_audio_version();
  err = ultimate_audio_load_wav(
    "/USB0/BOING.WAV", 0x004000UL, &voice);
  if (err == ULTIMATE_OK)
    err = ultimate_audio_configure(&voice);
  if (err == ULTIMATE_OK)
    err = ultimate_audio_start(voice.channel, voice.flags);
}
```

RESULT

The WAV data is in the REU; channel 0 starts with its file format and rate.

`ultimate_audio_init()` performs an inaudible probe using channel six and returns `ULTIMATE_ERR_NOT_SUPPORTED` when the register block is not mapped. `ultimate_audio_available()` reads the cached result, and the version call returns the raw module byte only after that result is true.

The WAV loader accepts uncompressed 8- or 16-bit PCM, mono or stereo, from 96 through 65,535Hz. It fills `reu_address`, `length`, `rate` and the format bits in `flags`; channel, volume, pan and repeat points keep the values supplied by the caller. An 8-bit WAV is unsigned, so the loader fixes its sign in place in the REU. Stereo data uses `UA_CTRL_INTERLEAVE`; the returned voice describes its left channel.

To describe raw PCM without a WAV, fill all fields yourself. Length and rate must be non-zero, all REU ranges and repeat points must fit in 24 bits, and `rate` is the rounded divider `6250000 / samples_per_second`. Use `UA_CTRL_16BIT` for signed little-endian 16-bit data, `UA_CTRL_REPEAT` with `repeat_a < repeat_b <= length`, and a pan value from `UA_PAN_LEFT` through `UA_PAN_RIGHT`.

Choose one format before configuring a raw voice:

```
voice.flags = 0; /* signed 8-bit mono */
voice.flags = UA_CTRL_16BIT; /* signed 16-bit mono */
voice.flags = UA_CTRL_INTERLEAVE; /* 8-bit stereo side */
voice.flags = UA_CTRL_16BIT |
UA_CTRL_INTERLEAVE; /* 16-bit stereo side */
```

RESULT

For an 8-bit stereo right voice add 1 to the left voice's REU address and subtract 1 from its length. For 16-bit stereo add and subtract 2 instead. Configure both voices before starting either one.

Looping, completion interrupts, volume and pan are independent choices:

```
voice.flags |= UA_CTRL_REPEAT;
voice.repeat_a = 4096;
voice.repeat_b = voice.length;

voice.flags |= UA_CTRL_IRQ;
voice.volume = UA_VOLUME_MAX; /* 0 is silent */
voice.pan = UA_PAN_LEFT;
voice.pan = UA_PAN_CENTER;
voice.pan = UA_PAN_RIGHT;
```

RESULT

REPEAT loops bytes 4096 through the end; IRQ latches and asserts completion; volume accepts 0–63 and pan accepts 0–15. The three named pan constants show the common positions.

Configure validates and writes the voice but leaves its gate closed. It also stops the channel and waits for the hardware to settle; always call start after configure. Start and stop are immediate.

Handle end-of-sample status

```
/* After installing an IRQ handler: */
voice.flags |= UA_CTRL_IRQ;
err = ultimate_audio_configure(&voice);
if (err == ULTIMATE_OK)
  err = ultimate_audio_start(voice.channel, voice.flags);

/* In that handler: */
if (ultimate_audio_irq_status() & (1u << voice.channel))
  err = ultimate_audio_irq_clear(voice.channel);

/* To cancel playback from the main program: */
err = ultimate_audio_stop(voice.channel);
```

RESULT

The bit for channel 0 is cleared after the sample ends; stop cancels playback.

`UA_CTRL_IRQ` asserts the C64 IRQ line when playback ends as well as latching the channel bit. Install an interrupt handler that clears it, or keep interrupts disabled while polling. Status returns all seven channel bits; clear takes one channel number.

INSPECT NETWORK INTERFACES

```
uint8_t count;
uint8_t mac[6];
uint8_t ipconfig[12];

err = ultimate_net_ifcount(&count);
if (err == ULTIMATE_OK && count != 0) {
  err = ultimate_net_macaddr(0, mac);
  err = ultimate_net_ipconfig(0, ipconfig);
}
```

RESULT

count = number of network interfaces
mac = six bytes; ipconfig = twelve bytes

The count call reports available interfaces. The next two calls read the six-byte MAC address and the twelve-byte IPv4 configuration for interface zero into caller-owned buffers.

MAC stands for Media Access Control, and the address is the one built into the interface.

Change an interface's address

```
err = ultimate_net_ipconfig(0, ipconfig);
if (err == ULTIMATE_OK) {
  ipconfig[3] = 60;
  err = ultimate_net_setip(0, ipconfig);
}
```

RESULT

The interface keeps its netmask and gateway and answers on .60 instead.

The block `ultimate_net_setip()` writes is the one `ultimate_net_ipconfig()` hands out: four bytes of address, then four of netmask, then four of gateway. Read the block, change the part you want and write it back.

This changes the running configuration only. Nothing is written to the Ultimate's stored settings, and the machine is already on the network by the time a C64 program runs. Neither the firmware nor the SDK checks the address against anything else on the segment.

USE TCP AND UDP SOCKETS

TCP, the Transmission Control Protocol, carries an ordered stream over a connection. UDP, the User Datagram Protocol, sends separate messages with no connection and no ordering. Both are opened the same way here and both give back a handle.

```
uint8_t handle;
uint8_t buf[128];
uint16_t got;
uint16_t sent;
static const uint8_t message[] = {0x48,0x49,13};

err = ultimate_net_connect("192.168.1.50", 6502, &handle);
if (err == ULTIMATE_OK) {
  err = ultimate_net_write(handle, message,
      sizeof message, &sent);
  do {
    err = ultimate_net_read(handle, buf, sizeof buf, &got);
  } while (err == ULTIMATE_OK && got == 0);
  if (err != ULTIMATE_END)
    ultimate_net_close(handle);
}

err = ultimate_net_udp("192.168.1.51", 6502, &handle);
if (err == ULTIMATE_OK)
  err = ultimate_net_close(handle);
```

RESULT

TCP: 3 BYTES SENT; REPLY BYTES RETURNED IN got
UDP HANDLE OPENED AND CLOSED

`ultimate_net_connect()` opens TCP; `ultimate_net_udp()` opens UDP. Both return a handle. Write reports bytes accepted through `sent`; read reports bytes stored through `got`. A successful zero-byte read means “not yet”, so poll. `ULTIMATE_END` means the peer closed and the handle is already released; call `ultimate_net_close()` only for a handle still owned by the program. On newer firmware, a truncated UDP datagram returns `ULTIMATE_ERR_TRUNCATED` with the bytes that fit in `got`; the rest is discarded. An empty UDP datagram also returns zero bytes; inspect `uci_last_device_code()` to distinguish it from no pending data.

FETCH A URL IN ONE CALL

```
static const char url[] = {
  0x68,0x74,0x74,0x70,0x3a,0x2f,0x2f,
  0x31,0x39,0x32,0x2e,0x31,0x36,0x38,0x2e,0x31,
  0x2e,0x35,0x30,0x2f,0
};
uint8_t reply[512];
uint16_t got;

err = ultimate_http_get(url, reply, sizeof reply, &got);
```

RESULT

err = ULTIMATE_OK
got = HTTP response-body bytes stored in reply

`ultimate_http_get()` creates, exchanges and closes one GET request. The explicit bytes prevent cc65 from changing lowercase URL characters to the C64 character set. `got` is the body bytes retained; a larger body returns `ULTIMATE_ERR_TRUNCATED` and has no continuation.

BUILD A MULTI-STEP HTTP REQUEST

Opening the request yourself is what adding a header needs. This example reuses `url`, `reply` and `got` from the preceding one.

```
uint8_t request;
uint8_t close_err;
static const char header[] = {
  0x61,0x63,0x63,0x65,0x70,0x74,0x3a,0x20,
  0x74,0x65,0x78,0x74,0x2f,0x70,0x6c,0x61,0x69,0x6e,0
};

err = ultimate_http_open(HTTP_VERB_GET, url, &request);
if (err == ULTIMATE_OK) {
  err = ultimate_http_header(request, header);
  if (err == ULTIMATE_OK)
    err = ultimate_http_exchange(request, HTTP_BODY_NONE,
      reply, sizeof reply, &got);
  close_err = ultimate_http_close(request);
  if (err == ULTIMATE_OK) err = close_err;
}

err = ultimate_http_free_all();
```

RESULT

```
GET WITH ONE HEADER: RESPONSE STORED
ALL HTTP OBJECTS FREED
```

`ultimate_http_open()` returns the request handle that header, exchange and close all use. `HTTP_VERB_GET` is one of the `HTTP_VERB_*` codes; `HTTP_VERB_POST`, `HTTP_VERB_PUT` and the rest are in `uci_protocol.h`. The header line is one string in `key: value` form.

Every supported HTTP method is selected at open time:

```
static const uint8_t verbs[] = {
  HTTP_VERB_GET, HTTP_VERB_PUT, HTTP_VERB_POST,
  HTTP_VERB_PATCH, HTTP_VERB_DELETE, HTTP_VERB_HEAD,
  HTTP_VERB_OPTIONS, HTTP_VERB_CONNECT, HTTP_VERB_TRACE
};
uint8_t i;

for (i = 0; i < sizeof verbs; ++i) {
  err = ultimate_http_open(verbs[i], url, &request);
  if (err == ULTIMATE_OK) ultimate_http_close(request);
}
```

RESULT

Each iteration creates and closes a request with that method. Opening alone sends nothing to the server.

This request sends no body, so the exchange is given `HTTP_BODY_NONE`. The next section builds a body and passes its handle there instead.

Close every handle you open. `ultimate_http_free_all()` recovers slots that an earlier program abandoned.

BUILD A REQUEST BODY

A POST or a PUT needs a body. The body is built inside the Ultimate rather than in C64 memory: create a slot, add one field at a time, then give its handle to `ultimate_http_exchange()`. Only one key and one value are in C64 memory at any moment, which is what lets a 38K machine post an object larger than it could hold.

`HTTP_BODY_JSON_OBJECT`	a JSON object
`HTTP_BODY_JSON_ARRAY`	a JSON array
`HTTP_BODY_URL_ENCODED`	form encoding, as an HTML form sends it
`HTTP_BODY_BINARY`	raw bytes
`HTTP_BODY_NONE`	no body; pass this to the exchange

JSON is JavaScript Object Notation, the text format most web services take.

```
/* Explicit bytes, so cc65 does not change their case. */
static const char k_name[] = {0x6e,0x61,0x6d,0x65,0};
static const char k_score[] = {0x73,0x63,0x6f,0x72,0x65,0};
static const char v_ada[] = {0x41,0x44,0x41,0};
uint8_t body;
uint8_t request;

err = ultimate_http_body(HTTP_BODY_JSON_OBJECT, &body);
if (err == ULTIMATE_OK) {
  err = ultimate_http_body_string(body, k_name, v_ada);
  err = ultimate_http_body_int(body, k_score, 4200);

  err = ultimate_http_open(HTTP_VERB_POST, url, &request);
  if (err == ULTIMATE_OK) {
    err = ultimate_http_exchange(request, body,
      reply, sizeof reply, &got);
    ultimate_http_close(request);
  }
  ultimate_http_body_free(body);
}
```

RESULT

POSTED {"name":"ADA","score":4200}; the reply is in reply.

Keys and string values go on the wire as given, and cc65 changes the case of a plain C literal exactly as it does for a URL. Build them as bytes, or fold the case back at run time.

The remaining root body format choices are short variations:

```
err = ultimate_http_body(HTTP_BODY_JSON_ARRAY, &body);
err = ultimate_http_body_int(body, "", 1);
err = ultimate_http_body_free(body);

err = ultimate_http_body(HTTP_BODY_URL_ENCODED, &body);
err = ultimate_http_body_string(body, k_name, v_ada);
err = ultimate_http_body_free(body);
```

RESULT

The first body is `[1]`; the second is the form field `name=ADA`. JSON object and binary bodies are shown above and below; `HTTP_BODY_NONE` is passed directly to exchange and creates no body.

Add a boolean, and nest a container

```
static const char k_alive[] = {0x61,0x6c,0x69,0x76,0x65,0};
static const char k_stats[] = {0x73,0x74,0x61,0x74,0x73,0};
static const char k_score[] = {0x73,0x63,0x6f,0x72,0x65,0};
static const char k_lives[] = {0x6c,0x69,0x76,0x65,0x73,0};
static const char k_runs[] = {0x72,0x75,0x6e,0x73,0};
static const char no_key[] = {0};

err = ultimate_http_body(HTTP_BODY_JSON_OBJECT, &body);
err = ultimate_http_body_bool(body, k_alive, 1);

err = ultimate_http_body_object(body, k_stats);
err = ultimate_http_body_int(body, k_score, 4200);
err = ultimate_http_body_up(body);
err = ultimate_http_body_int(body, k_lives, 3);

err = ultimate_http_body_array(body, k_runs);
err = ultimate_http_body_int(body, no_key, 1);
err = ultimate_http_body_up(body);
err = ultimate_http_body_free(body);
```

RESULT

{"alive":true,"stats":{"score":4200},"lives":3,"runs":[1]}

Adding an object or an array enters it. Every key added afterwards goes inside, until `ultimate_http_body_up()` steps back out to the parent. Inside an array the key is ignored, but the pointer must still be valid; pass an empty string as shown. Any non-zero value is true.

Send raw bytes instead

```
static const uint8_t block[] = {1,2,3,4};

err = ultimate_http_body(HTTP_BODY_BINARY, &body);
err = ultimate_http_body_binary(body, block, sizeof block);
err = ultimate_http_body_binary(body, block, sizeof block);

err = ultimate_http_body_clear(body);
err = ultimate_http_body_free(body);
```

RESULT

The body holds eight bytes, then none; the slot is returned.

Successive calls to `ultimate_http_body_binary()` append. `ultimate_http_body_clear()` empties a body and keeps the slot and its format; `ultimate_http_body_free()` returns the slot.

A `HTTP_BODY_BINARY` body takes `ultimate_http_body_binary()` and nothing else, and the JSON and form formats take everything else. The firmware answers the wrong combination with `ULTIMATE_ERR_DEVICE`.

A key is copied through a staging buffer, so it is limited to `ULTIMATE_HTTP_KEY_MAX` bytes, which is 34. A longer key returns `ULTIMATE_ERR_INVALID_ARGUMENT` and never reaches the wire. Values are not limited by that buffer, and a string value is at most 255 bytes, which is the firmware's own length byte.

Careful: The firmware has sixteen slots for the whole machine, and a program that crashes returns none of them. Free every body and every request you create. `ultimate_http_free_all()` takes back headers and bodies together, and is how a program recovers slots an earlier one abandoned.

PROBE AND CONFIGURE THE RAW TRANSPORT

```
uint8_t present = uci_signature_present();
uint8_t ident = uci_ident_register();
uint8_t old_timeout;

err = uci_init();
old_timeout = uci_get_timeout();
uci_set_timeout(UCI_TIMEOUT_FOREVER);
/* run the deliberately unbounded command */
uci_set_timeout(old_timeout);
err = uci_abort();
```

RESULT

present = 1
ident = command-interface identification byte
abort = ULTIMATE_OK

The signature and identification-register calls are immediate register reads. `uci_init()` performs the functional startup check. Timeout zero means wait forever; save and restore

the old value. `uci_abort()` releases any exchange still in progress and is safe when the interface is already idle.

EXECUTE A RAW COMMAND WITH AND WITHOUT ARGUMENTS

```
uci_request req = {0};
uint8_t reply[64];
uint8_t status[16];
uint8_t format = 0;
uint8_t sub = CTRL_HWINFO_MODEL;

req.target = UCI_TARGET_DOS1;
req.command = UCI_CMD_IDENTIFY;
req.data = reply;
req.datamax = sizeof reply;
req.status = status;
req.statusmax = sizeof status;
err = uci_exec(&req);

memset(&req, 0, sizeof req);
req.target = UCI_TARGET_DOS1;
req.command = DOS_CMD_GET_TIME;
req.args = &format;
req.arglen = 1;
req.data = reply;
req.datamax = sizeof reply;
err = uci_exec(&req);

memset(&req, 0, sizeof req);
req.target = UCI_TARGET_CONTROL;
req.command = CTRL_CMD_GET_HWINFO;
req.data = reply;
req.datamax = sizeof reply;
err = uci_exec(&req);

req.args = &sub;
req.arglen = 1;
err = uci_exec(&req);
```

RESULT

IDENTIFY: reply contains target name
GET_TIME: reply contains formatted time
GET_HWINFO: reply contains model name

The identify request has no arguments: zero initialization leaves `args`, `payload` and their lengths empty. The time request supplies its optional format byte. The hardware-info request is sent first without its optional selector and then with `CTRL_HWINFO_MODEL`; both forms request the model, which is what an omitted optional argument means on the wire. A payload is a second independent span; set it only when required.

To discard reply data or status deliberately, leave the corresponding pointer and capacity zero:

```
req.data = NULL;
req.datamax = 0;
req.status = NULL;
req.statusmax = 0;
err = uci_exec(&req);
```

RESULT

req.datalen = 0
req.statuslen = 0

READ A RAW MULTI-BLOCK REPLY

```
err = uci_exec_first(&req);
while (err == ULTIMATE_OK) {
  process(req.data, req.datalen);
  if (!uci_more_blocks()) break;
  err = uci_exec_next(&req);
}
```

RESULT

Each iteration receives one reply block; the loop ends when
uci_more_blocks() = 0

The first call submits the request and returns one reply block. The boolean call reports whether another follows; the next call releases the current block and receives another. Do not issue an unrelated command inside this loop.

DECODE AND INSPECT RAW STATUS

```
static const uint8_t raw[] = {
  0x34,0x30,0x34,0x20,0x4e,0x4f,0x54,
  0x20,0x46,0x4f,0x55,0x4e,0x44
};
uint8_t expected;
uint16_t device_code;

expected = uci_status_format(UCI_TARGET_HTTP);
err = uci_decode_status(UCI_TARGET_HTTP, raw, sizeof raw);
device_code = uci_last_device_code();
```

`uci_status_format()` describes the target's expected encoding. `uci_decode_status()` decodes actual bytes and records their numeric status; `uci_last_device_code()` retrieves it. The decoder accepts an empty status as success, so this is also the form without status bytes:

```
err = uci_decode_status(UCI_TARGET_DOS1, NULL, 0);
```

RESULT

expected = UCI_STATUS_FMT_DECIMAL3
err = ULTIMATE_ERR_DEVICE
device_code = 404
empty DOS status = ULTIMATE_OK

CHAPTER 4

PROGRAMMING IN ASSEMBLY

- The register and shared-variable conventions
- Relocating the SDK's RAM and zero page
- Every service, graphics and audio entry point by example
- Raw request blocks with optional spans

THE CALLING CONVENTION

Include `ultimate.inc` and call the unprefixed entry points. Unless an entry below says otherwise, a service returns an `ULTIMATE_*` result in `A` and may change `A`, `X`, `Y` and the flags. A byte argument uses `A`; a pointer argument uses low byte in `A` and high byte in `X`; multi-byte service arguments live in the exported `ult_*` variables.

The examples use these three ordinary ca65 macros to keep pointer and integer setup visible without repeating four instructions each time:

```
    .include "ultimate.inc"

.macro setptr location, value
    lda #<value
    sta location
    lda #>value
    sta location + 1
.endmacro

.macro setword location, value
    lda #<value
    sta location
    lda #>value
    sta location + 1
.endmacro

.macro setdword location, value
    lda #.lobyte(value):sta location
    lda #.hibyte(value):sta location + 1
    lda #.bankbyte(value):sta location + 2
    lda #.hibyte(.hiword(value)):sta location + 3
.endmacro
```

RESULT

setptr ult_buf, text stores the address of text in ult_buf.

`setptr` and `setword` emit the same bytes; their different names make the purpose clear at each call site. `setdword` stores all four bytes of a constant. Result branches always compare `A` immediately, before another instruction replaces it.

Assembly examples omit screen-printing routines so the SDK operation itself stays visible. Their RESULT boxes therefore describe returned registers, memory or hardware effects; they are not C64 screen output.

With the C64 character map, lowercase source letters assemble to the uppercase byte values shared by PETSCII and ASCII. Paths, filenames and HTTP text therefore appear in

lowercase source below but reach the firmware in uppercase.

A NATIVE PRG WITHOUT C STARTUP

Selective linking does not require the C runtime. This complete ca65 source performs the same turbo-register check as the selective-linking walkthrough (page 6). It supplies its own BASIC line, `10 SYS 2304`, instead of force-importing `__STARTUP__`. The listing comes from `guide/examples/selective.s`, which the guide build assembles:

```
.include "ultimate.inc"

.segment "LOADADDR"
    .word $0801
.segment "STARTUP"
    .word $080b, 10
    .byte $9e, "2304", 0  ; 10 SYS 2304
    .word 0

.code
.export start
start:
    jsr ultimate_turbo_available
    cmp #0
    beq absent
    lda #5                  ; green
    bne paint
absent:
    lda #2                  ; red
paint:
    sta $d020
    rts                     ; return to BASIC
```

Its complete ld65 configuration is `guide/examples/selective.cfg`:

```
MEMORY {
    HEADER: start = $07ff, size = 2, file = %O;
    RAM: start = $0801, size = $37ff, file = %O;
}
SEGMENTS {
    LOADADDR: load = HEADER, type = ro;
    STARTUP: load = RAM, type = ro, start = $0801;
    CODE: load = RAM, type = ro, start = $0900;
    RODATA: load = RAM, type = ro, optional = yes;
    BSS: load = RAM, type = bss, optional = yes;
}
```

The first two file bytes hold the PRG's load address; they are not loaded at `$07FF`. The BASIC stub loads at `$0801` and enters code at `$0900` (2304). If you move the entry address, update the stub as well. RAM ends before `$4000`; ld65 reports overflow if the program grows past it. Optional RODATA and BSS allow services that need constant data or workspace; turbo needs neither. Without C startup, initialize your own variables and obey the SDK's per-API initialization rules.

Assemble and link from the repository root, with the application object before the archive:

```
make lib
ca65 -t c64 -I bindings/asm \
  -o guide/examples/selective.o \
  guide/examples/selective.s
ld65 -C guide/examples/selective.cfg \
  -m guide/examples/selective-asm.map \
  -o guide/examples/selective-asm.prg \
  guide/examples/selective.o \
  bindings/cc65/build/ultimate.lib
```

RESULT

Running the PRG makes the border green if the turbo register is available, red otherwise, then returns to BASIC. It prints nothing and does not change CPU speed.

The map's Modules list contains only the application and `ultimate.lib(turbo.o)`. There are no C adapters or `c64.lib` members. The Segment list places CODE at `$0900`; the Imports list shows the application's reference to `ultimate_turbo_available`. More complex VIC and sample layouts appear in the ca65 configurations for Boing and Follow Me; this small configuration is not a general game memory layout.

PLACE THE SDK'S WORKING MEMORY

The SDK's zero-page block starts at `UCI_ZP`, which defaults to `$FB`. Its size is `UCI_ZP_SIZE`. The normal variable block occupies `UCI_VARS_SIZE` bytes in the BSS segment, which is where the linker puts variables that have no starting value. To choose fixed locations, rebuild a separate archive from the repository root so every SDK object receives the same definitions:

```
make -B -C bindings/cc65 BUILD=build-custom \
  ASM_DEFINES="-D UCI_VARS=52992 -D UCI_ZP=163"
```

RESULT

UCI variables begin at $CF00; zero-page workspace begins at $A3.

Link `bindings/cc65/build-custom/ultimate.lib`, not the default archive. For example, rebuild the earlier startup check against this layout:

```
cl65 -t c64 --asm-include-dir bindings/asm \
  -m guide/examples/hello-custom.map \
  -o guide/examples/hello-custom.prg \
  guide/examples/hello.s \
  bindings/cc65/build-custom/ultimate.lib
```

RESULT

The hello program is linked with SDK workspace at the selected addresses. Its behavior is the same as the Getting Started assembly check.

Decimal values avoid the shell expanding dollar-prefixed names. `-B` forces rebuilding because changing Make variables alone does not invalidate existing objects. Keep a separate build directory per layout; application-only defines do not relocate a prebuilt SDK.

With `UCI_VARS` defined, the core emits no BSS; its variables become equates from that address. Reserve both workspace ranges yourself: currently `$CF00–$CFBE` and `$A3–$A6`. ld65 cannot detect an overlap with fixed equates. This tiny hello program fits; larger programs must reserve these ranges away from code, stack, and buffers. These definitions move workspace, not code; the linker configuration places CODE and RODATA. The SDK installs no interrupt handler and is not re-entrant, so an interrupt routine must not call it while foreground code is inside a service.

INITIALIZE, DETECT AND DESCRIBE THE MACHINE

```
    jsr ultimate_init
    cmp #ULTIMATE_OK
    bne failed

    jsr ultimate_available
    beq unavailable

    lda #<capabilities
    ldx #>capabilities
    jsr ultimate_detect
    cmp #ULTIMATE_OK
    bne failed

capabilities: .res 4
```

RESULT

A = ULTIMATE_OK
capabilities records the detected targets.

`ultimate_init` performs startup. `ultimate_available` returns a boolean rather than a result code. `ultimate_detect` takes the address of the four-byte capability block directly in A/X.

Identify a target with and without the optional length output

```
    setptr ult_buf, text
    setword ult_buflen, 64
    setptr ult_outlen, text_length
    lda #UCI_TARGET_DOS1
    jsr ultimate_identify

    setword ult_outlen, 0
    lda #UCI_TARGET_DOS1
    jsr ultimate_identify

text: .res 64
text_length: .res 2
```

RESULT

text = "ULTIMATE-II DOS V1.2"
text_length = 20 with the pointer; no length is stored with zero.

Both calls terminate the text in `ult_buf`. The first stores its length through `ult_outlen`; zero disables that optional output.

Read the model with and without the optional length output

```
    setptr ult_buf, text
    setword ult_buflen, 64
    setptr ult_outlen, text_length
    jsr ultimate_get_model

    setword ult_outlen, 0
    jsr ultimate_get_model
```

RESULT

text = "Ultimate 64 Elite"
text_length is optional in the same way as IDENTIFY.

`ultimate_get_model` uses the same buffer contract but needs no target argument. Its underlying `CTRL_CMD_GET_HWINFO` command is marked deprecated by the firmware documentation; this existing wrapper remains for compatibility.

Read SID addresses through deprecated HWINFO

```
    lda #<sid_info
    ldx #>sid_info
    jsr ultimate_legacy_get_sid_info
    cmp #ULTIMATE_OK
    bne failed

    lda sid_info + ULTIMATE_SID_INFO_COUNT
    ; iterate this many ULTIMATE_SID_RECORD_SIZE-byte records
    ldx #ULTIMATE_SID_INFO_RECORDS
    lda sid_info + ULTIMATE_SID_PRIMARY_ADDRESS,x
    sta first_sid_address
    lda sid_info + ULTIMATE_SID_PRIMARY_ADDRESS + 1,x
    sta first_sid_address + 1

sid_info: .res ULTIMATE_SID_INFO_SIZE
first_sid_address: .res 2
```

RESULT

sid_info count = 4
primary addresses = $D400, $D400, $D400, $D400

Records begin at `ULTIMATE_SID_INFO_RECORDS`. Within each record, `ULTIMATE_SID_PRIMARY_ADDRESS` and `ULTIMATE_SID_SECONDARY_ADDRESS` name two little-endian words, and `ULTIMATE_SID_TYPE` names the raw type byte. The type is not a 6581 or 8580 identifier. Firmware still implements this query, but marks HWINFO deprecated and publishes no replacement C64-side UCI command. The legacy name is deliberate; handle `ULTIMATE_ERR_NOT_SUPPORTED` and read no record unless the call returned `ULTIMATE_OK`.

Turn a result into text

```
    lda error_code
    jsr ultimate_strerror
    sta message_ptr
    stx message_ptr + 1

error_code: .res 1
message_ptr: .res 2
```

RESULT

message_ptr points to the PETSCII text for error_code.

`ultimate_strerror` returns the PETSCII message pointer in `A/X`; it does not return a result code.

READ AND CHANGE THE PALETTE

```
    lda #<palette
    ldx #>palette
    jsr ultimate_palette_get

    lda #<palette
    ldx #>palette
    jsr ultimate_palette_set

    lda #6
    sta ult_color
    lda #255
    sta ult_color + 1
    lda #0
    sta ult_color + 2
    sta ult_color + 3
    jsr ultimate_palette_set_color

    jsr ultimate_palette_reset

palette: .res UCI_PALETTE_BYTES
```

RESULT

GET fills 48 bytes; SET COLOUR 6 TO RED and RESET return ULTIMATE_OK.

Get and set take a direct pointer in `A/X`. The single-colour entry reads index, red, green and blue from the four consecutive `ult_color` bytes. Reset restores the built-in live palette. Firmware without these commands returns `ULTIMATE_ERR_NOT_SUPPORTED`; branch on `A` before using the 48-byte result from `ultimate_palette_get`.

USE TURBO AND BADLINE CONTROL

```
    jsr ultimate_turbo_available
    beq no_turbo

    jsr ultimate_turbo_get
    sta old_speed

    lda #U64_SPEED_4MHZ
    jsr ultimate_turbo_set

    lda #0
    jsr ultimate_turbo_badlines
    lda #1
    jsr ultimate_turbo_badlines

    lda old_speed
    jsr ultimate_turbo_set

old_speed: .res 1
```

RESULT

old_speed = previous speed index
SET 4 MHZ, THEN RESTORE SPEED AND BADLINES: ULTIMATE_OK

Each named speed is passed in `A`:

```
    lda #U64_SPEED_1MHZ
    jsr ultimate_turbo_set
    lda #U64_SPEED_2MHZ
    jsr ultimate_turbo_set
    lda #U64_SPEED_3MHZ
    jsr ultimate_turbo_set
    lda #U64_SPEED_4MHZ
    jsr ultimate_turbo_set
    lda #U64_SPEED_MAX
    jsr ultimate_turbo_set
```

RESULT

The first four constants are portable; MAX selects this machine's fastest index. Save and restore the entry speed as in the first listing.

The available and get calls return values directly. Set returns a result. The two badline calls show both forms explicitly: zero disables them; non-zero restores them. The final set restores the speed read at entry.

COMPOSITE A SOFTWARE VSPRITE

```
    lda #<sprite
    ldx #>sprite
    jsr ultimate_vsprite_draw
    cmp #ULTIMATE_OK
    bne failed

sprite:
    .word $6000    ; destination bitmap base
    .word image      ; column-major source
    .word mask        ; column-major mask
    .word 0          ; no color-screen writes
    .byte 12, 72    ; cell column, raster line
    .byte 2, 8      ; byte columns, raster lines
    .byte 0, VSPRITE_F_MASKED

image: .byte $00,$18,$3c,$7e,$7e,$3c,$18,$00
       .byte $00,$18,$3c,$7e,$7e,$3c,$18,$00
mask: .byte $ff,$e7,$c3,$81,$81,$c3,$e7,$ff
       .byte $ff,$e7,$c3,$81,$81,$c3,$e7,$ff
```

RESULT

A = ULTIMATE_OK; the masked 2-by-8-byte image is composited into $6000.

The operation is the descriptor's final byte:

```
    lda #0
    sta sprite + VSPRITE_FLAGS
    ; OR the column-major source

    lda #VSPRITE_F_MASKED
    sta sprite + VSPRITE_FLAGS
    ; (destination AND mask) OR source

    lda #VSPRITE_F_COPY
    sta sprite + VSPRITE_FLAGS
    ; source is another bitmap base

    lda #VSPRITE_F_COLOR
    sta sprite + VSPRITE_FLAGS
    ; set VSPRITE_SCREEN and VSPRITE_COLOR too
```

RESULT

The four settings select OR, masked, bitmap-copy and color-aware OR draws. Call `ultimate_vsprite_draw` after setting one of them.

The descriptor is exactly `VSPRITE_SIZE` bytes; its named offsets are `VSPRITE_BITMAP` through `VSPRITE_FLAGS`. Pass its address directly in `A/X`. The coordinate, source-layout, flag-combination and validation rules are the ones described in *Composite A Software Vsprite* in Chapter 3.

This call needs no UCI initialization. It patches its own absolute operands, so the linked routine must be in RAM and an interrupt must not re-enter it.

CHANGE AND READ THE CURRENT DIRECTORY

```
    setptr ult_buf, path
    jsr ultimate_chdir

    setptr ult_buf, path_buffer
    setword ult_buflen, 128
    setptr ult_outlen, path_length
    jsr ultimate_getpath

    setword ult_outlen, 0
    jsr ultimate_getpath

path: .byte $2f,$55,$53,$42,$31,$2f,$47,$41,$4d,$45,$53,0
path_buffer: .res 128
path_length: .res 2
```

RESULT

path_buffer = "/USB1/GAMES"
path_length = 11 when the optional pointer is enabled.

`ultimate_chdir` consumes the terminated path through `ult_buf`. The two getpath calls show the optional length pointer enabled and disabled.

Read every directory entry

```
    jsr ultimate_opendir
    cmp #ULTIMATE_OK
    bne failed

next_entry:
    setptr ult_buf, name
    setword ult_buflen, 64
    jsr ultimate_readdir
    cmp #ULTIMATE_END
    beq directory_done
    cmp #ULTIMATE_OK
    bne failed
    lda ult_attrib
    ; use name and its DOS_ATTR_* bits here
    jmp next_entry

name: .res 64
```

RESULT

name receives one entry per call; A becomes ULTIMATE_END after the last.

Each readdir releases one reply block and writes attributes to `ult_attrib`. Issue no other command inside the walk. Call `uci_abort` if leaving before `ULTIMATE_END`.

Make a directory, and return to the configured one

```
    setptr ult_buf, dir_name
    jsr ultimate_mkdir

    jsr ultimate_home

dir_name: .byte $53,$41,$56,$45,$53,0
```

RESULT

SAVES is created; A = ULTIMATE_OK from each call.

`ultimate_mkdir` takes the terminated name through `ult_buf` and measures it itself. `ultimate_home` takes nothing at all.

OPEN, SEEK, READ, WRITE AND CLOSE FILES

```
    setptr ult_buf, read_name
    lda #DOS_FA_READ
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed

    lda #0
    sta ult_num
    sta ult_num + 1
    sta ult_num + 2
    sta ult_num + 3
    jsr ultimate_seek

    setptr ult_buf, file_buffer
    setword ult_buflen, 128
    setptr ult_outlen, bytes_read
    jsr ultimate_read
    jsr ultimate_close

read_name: .byte $52,$45,$41,$44,$4d,$45,$2e,$54,$58,$54,0
file_buffer: .res 128
bytes_read: .res 2
```

RESULT

file_buffer receives up to 128 bytes; bytes_read receives the actual count.

Open establishes the firmware's current file. Seek selects its absolute position. Read stores bytes and, when `ult_outlen` is non-zero, their count. Close releases that same firmware-side file.

Use these masks in `A` before `ultimate_open`:

Mask	Policy
`DOS_FA_READ`	read an existing file
`DOS_FA_WRITE`	write an existing file without truncating it
`DOS_FA_WRITE | DOS_FA_CREATE_NEW`	create only if absent
`DOS_FA_WRITE | DOS_FA_CREATE_ALWAYS`	create or replace
`DOS_FA_READ | DOS_FA_WRITE`	update and read back

The count output is optional:

```
    ; With a readable file already open:
    setptr ult_buf, file_buffer
    setword ult_buflen, 128
    setword ult_outlen, 0
    jsr ultimate_read
```

RESULT

file_buffer receives bytes; no byte count is stored.

Create, write and delete a file

```
    setptr ult_buf, write_name
    lda #DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed

    setptr ult_buf, message
    setword ult_buflen, message_end - message
    jsr ultimate_write
    jsr ultimate_close

    setptr ult_buf, write_name
    jsr ultimate_delete

write_name: .byte $52,$45,$53,$55,$4c,$54,$2e,$54,$58,$54,0
message: .byte $4f,$4b,13
message_end:
```

RESULT

RESULT.TXT: 3 bytes written, closed and deleted; A = ULTIMATE_OK.

The open mask creates or replaces the file. Write consumes `ult_buflen` bytes from `ult_buf`; delete consumes the terminated name after the file is closed.

ASK WHAT A FILE IS

```
    setptr ult_buf, stat_name
    setptr ult_arg2, file_info
    jsr ultimate_stat
    cmp #ULTIMATE_OK
    bne failed

    lda file_info + DOS_INFO_SIZE
    sta file_size
    lda file_info + DOS_INFO_SIZE + 1
    sta file_size + 1
    lda file_info + DOS_INFO_ATTRIB
    and #DOS_ATTR_DIR
    bne is_a_directory

stat_name: .byte $52,$45,$41,$44,$4d,$45,$2e,$54,$58,$54,0
file_info: .res ULTIMATE_FILEINFO_SIZE
file_size: .res 2
```

RESULT

file_info holds the reply; the name inside it is NUL-terminated.

`ult_buf` is the name and `ult_arg2` is the block the reply is received into. That block must be `ULTIMATE_FILEINFO_SIZE` bytes. `ult_buflen` is not an input here: `ultimate_stat` measures the name itself and overwrites `ult_buflen` doing it.

`ultimate_fstat` asks the same question about the file `ultimate_open` left open, so it takes only the block:

```
    setptr ult_arg2, file_info
    jsr ultimate_fstat
```

RESULT

file_info describes the open file; ult_buf is not read.

The `DOS_INFO_*` equates in `uci_protocol.inc` name the fields inside the block:

DOS_INFO_SIZE	+$00, size in bytes, 32-bit, low byte first
DOS_INFO_DATE	+$04, year less 1980 in bits 15–9, month 8–5, day 4–0
DOS_INFO_TIME	+$06, hour 15–11, minute 10–5, two-second units 4–0
DOS_INFO_EXTENSION	+$08, three bytes, space padded
DOS_INFO_ATTRIB	+$0B, the DOS_ATTR_* bits
DOS_INFO_NAME	+$0C, the name, terminated by the SDK

Date and time are the packed forms used by FAT, the File Allocation Table filesystem the Ultimate's storage uses, and they arrive unconverted.

RENAME AND COPY A FILE

```
    setptr ult_buf, old_name
    setptr ult_arg2, new_name
    jsr ultimate_rename

    setptr ult_buf, new_name
    setptr ult_arg2, backup_path
    jsr ultimate_copy

old_name: .byte $4f,$4c,$44,$2e,$54,$58,$54,0
new_name: .byte $4e,$45,$57,$2e,$54,$58,$54,0
backup_path: .byte $2f,$55,$53,$42,$31,$2f,$42,$41,$43,$4b,$55,$50,0
```

RESULT

OLD.TXT becomes NEW.TXT; BACKUP/NEW.TXT is a copy of it.

Rename takes current-directory names in `ult_buf` and `ult_arg2`. Copy takes a current-directory source in `ult_buf` and a destination path in `ult_arg2`; the source filename is preserved. The request block carries two spans and these commands put three runs of bytes on the wire, so the first name's own terminator separates it from the second. The Ultimate performs the copy, so no bytes pass through the C64.

LOAD AND SAVE C64 MEMORY

```
    setptr ult_buf, prg_name
    setword ult_addr, 0
    jsr ultimate_load
    jsr ultimate_last_end
    sta load_end
    stx load_end + 1

    setword ult_addr, $C000
    jsr ultimate_load

    setptr ult_buf, raw_name
    setword ult_addr, $C000
    setword ult_max, 2048
    jsr ultimate_bload

    setptr ult_buf, save_name
    setword ult_addr, $0400
    setword ult_max, 1000
    jsr ultimate_save

prg_name: .byte $47,$41,$4d,$45,$2e,$50,$52,$47,0
raw_name: .byte $46,$4f,$4e,$54,$2e,$42,$49,$4e,0
save_name: .byte $53,$43,$52,$45,$45,$4e,$2e,$42,$49,$4e,0
load_end: .res 2
```

RESULT

load_end = address after loaded PRG data
FONT.BIN loads at $C000; SCREEN.BIN receives 1000 bytes.

An `ult_addr` of zero makes load use the PRG header; `$C000` shows the explicit-address form. `ultimate_last_end` returns the previous load end in `A/X`. Bload consumes no header and obeys `ult_max`; save uses `ult_max` as its length.

MOUNT A DISK IMAGE

The drive is named by the number it answers to on the IEC serial bus, 8 or 9, rather than by a slot. `ULTIMATE_DRIVE_LAST` is zero and asks for whichever drive was mounted into last, or drive A when nothing has been.

```
    setptr ult_buf, image_name
    lda #8
    jsr ultimate_mount
    cmp #ULTIMATE_OK
    bne failed

    lda #8
    jsr ultimate_swap

    lda #8
    jsr ultimate_unmount

image_name: .byte $47,$41,$4d,$45,$53,$2f,$50,$49,$54
              .byte $46,$41,$4c,$4c,$2e,$44,$36,$34,0
```

RESULT

PITFALL.D64 is mounted, swapped and unmounted; A = ULTIMATE_OK each time.

The device number goes in `A` and is reloaded before each call, because every entry point returns its result there. `ultimate_mount` takes the image name through `ult_buf` and measures it itself.

The firmware picks the image type from the extension: `.D64`, `.D71` and `.D81` hold the sectors of a 1541, 1571 and 1581 disk, and `.G64` and `.G71` hold raw track data. Mounting into a drive that is switched off returns `ULTIMATE_ERR_DEVICE`.

```
    lda #<drives
    ldx #>drives
    jsr ultimate_drive_info
    cmp #ULTIMATE_OK
    bne failed
    lda drives + CTRL_DRVINFO_COUNT
    beq no_drives
    lda drives + CTRL_DRVINFO_FIRST + 1
    sta first_device

    lda #ULTIMATE_DRIVE_A
    ldx #1
    jsr ultimate_drive_enable

    lda #ULTIMATE_DRIVE_A
    jsr ultimate_drive_power
    lda ult_stage
    beq drive_is_off

drives: .res CTRL_DRVINFO_BYTES
first_device: .res 1
```

RESULT

drives holds a count and one record per drive; ult_stage = 1 when it runs.

`ultimate_drive_info` takes the block pointer in `A/X`. The block is a count at `CTRL_DRVINFO_COUNT`, then `CTRL_DRVINFO_RECORD` bytes per record from `CTRL_DRVINFO_FIRST`: type, then the IEC device number, then 1 while the drive is running. On any failure the count is set to zero, so a caller that skips the result check reads no records.

The block must be `CTRL_DRVINFO_BYTES` long, because the records are not only drives A and B. The firmware reports each occupied IEC bus slot in the same reply, and there are four of those, so the count reaches six on a machine running SoftwareIEC and an IEC printer. A slot has a type of `CTRL_DRVTYPE_SOFTIEC` or `CTRL_DRVTYPE_PRINTER`; read the type rather than assuming every record is a disk drive.

`ultimate_drive_enable` takes the drive in `A` and the state in `X`: zero switches it off, and any non-zero value switches it on. `ultimate_drive_power` takes the drive in `A` and reports the state in `ult_stage`, which is 1 while the drive runs.

Note: Two numbering schemes meet here. `ultimate_drive_enable` and `ultimate_drive_power` take `ULTIMATE_DRIVE_A` or `ULTIMATE_DRIVE_B`, which name a physical drive. `ultimate_mount` takes the device number the drive answers to on the IEC bus. `ultimate_drive_info` connects the two: each record carries both.
Those two entry points accept no drive beyond A and B, because the firmware has one command per drive and it has commands for those two only. That limit is `CTRL_DRIVE_SLOTS` and it is not `CTRL_DRVINFO_MAX`, which is how many records the information reply can carry.

Careful: `ultimate_drive_power` writes the flag to `ult_stage` and uses `ult_stage + 1` to `ult_stage + 3` as scratch. Read the flag before the next SDK call.

Reset the machine, or open its menu

```
    jsr ultimate_freeze

    jsr ultimate_reboot
    ; nothing here runs
```

RESULT

freeze returns when the menu is left; reboot does not return.

`ultimate_reboot` resets the C64, so the program calling it stops existing part way through the call. `ultimate_freeze` stops the C64 and shows the Ultimate's menu, and returns when whoever is at the keyboard leaves it.

Read the RAM disk layout

```
    lda #<ramdisk
    ldx #>ramdisk
    jsr ultimate_ramdisk_info

ramdisk: .res CTRL_RAMDISK_BYTES
```

RESULT

ramdisk holds CTRL_RAMDISK_DRIVES two-byte records.

Each record is a drive type code and a size in 64K units. A type of zero is a slot with nothing in it. These are the RAM disks GEOS MegaPatch uses.

READ AND SET THE CLOCK

```
    setptr ult_buf, time_text
    setword ult_buflen, ULTIMATE_TIME_BUFFER
    setword ult_outlen, 0
    lda #ULTIMATE_TIME_PLAIN
    jsr ultimate_get_time

time_text: .res ULTIMATE_TIME_BUFFER
```

RESULT

time_text = "2026/08/22 14:30:00"

`A` is the format. `ULTIMATE_TIME_PLAIN` asks for the date and time and `ULTIMATE_TIME_WEEKDAY` puts the weekday in front of it. `ULTIMATE_TIME_BUFFER` is a buffer size that fits either form. This is one of the few entry points that reads `ult_buflen` as an input rather than computing it. A buffer too small still returns `ULTIMATE_OK`, with the part that fitted.

Set it

```
    lda #126    ; 2026, as the year less 1900
    sta ult_stage + 0
    lda #8      ; month
    sta ult_stage + 1
    lda #22     ; day
    sta ult_stage + 2
    lda #14     ; hour
    sta ult_stage + 3
    lda #30     ; minute
    sta ult_stage + 4
    lda #0      ; second
    sta ult_stage + 5
    jsr ultimate_set_time
```

RESULT

The clock is set to 2026/08/22 14:30:00; A = ULTIMATE_OK.

The six bytes go in `ult_stage` in that order. `ult_stagelen` is not a caller input here; the command always sends six bytes.

> **Careful:** `ult_stage` is shared. `ultimate_mount`, `ultimate_unmount`, `ultimate_swap`, `ultimate_get_time`, `ultimate_drive_info` and `ultimate_http_body` all overwrite `ult_stage + 0`, and `ultimate_drive_power` and the keyed body calls overwrite more of it. Fill it immediately before `ultimate_set_time`.

MOVE BYTES THROUGH THE REU

```
    jsr ultimate_reu_available
    beq no_reu
    jsr ultimate_reu_size
    sta reu_banks
    stx reu_banks + 1

    setword ult_addr, $0400
    lda #0
    sta ult_reu
    sta ult_reu + 1
    sta ult_reu + 2
    sta ult_reu + 3
    setword ult_reulen, 1000
    lda #0
    sta ult_reulen + 2
    sta ult_reulen + 3
    jsr ultimate_reu_stash
    jsr ultimate_reu_fetch

reu_banks: .res 2
```

RESULT

reu_banks = installed REU size in 64K banks
1000 bytes stashed and fetched; A = ULTIMATE_OK.

Size returns 64K banks in A/X. Stash copies C64 RAM to the REU; fetch copies it back. Both read the C64 address, 32-bit REU address and 32-bit length from the shared variables.

Move the current file directly to and from the REU

Both transfers below use REU address zero and a 65536-byte length:

```
    lda #0
    sta ult_reu
    sta ult_reu + 1
    sta ult_reu + 2
    sta ult_reu + 3
    sta ult_reulen
    sta ult_reulen + 1
    lda #1
    sta ult_reulen + 2
    lda #0
    sta ult_reulen + 3
```

RESULT

ult_reu = $00000000; ult_reulen = 65536.

File to REU

```
    ; Open ASSETS.BIN first; it becomes the current DOS file.
    setptr ult_buf, asset_name
    lda #DOS_FA_READ
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed
    lda #0
    sta ult_num
    sta ult_num + 1
    sta ult_num + 2
    sta ult_num + 3
    jsr ultimate_seek
    jsr ultimate_reu_load
    jsr ultimate_close

asset_name: .byte $41,$53,$53,$45,$54,$53,$2e,$42,$49,$4e,0
```

RESULT

ASSETS.BIN to REU $000000: 65536 bytes

REU to file

```
    ; Open a writable current file before the transfer.
    setptr ult_buf, capture_name
    lda #DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed
    jsr ultimate_reu_save
    jsr ultimate_close

capture_name: .byte $43,$41,$50,$54,$55,$52,$45,$2e,$42,$49,$4e,0
```

RESULT

REU $000000 to CAPTURE.BIN: 65536 bytes

The REU address is zero and the length bytes encode 65536. Load and save take no filename: each operates on the firmware's current open file established by the preceding `ultimate_open`.

LOAD AND PLAY PCM THROUGH ULTIMATE AUDIO

```
    jsr ultimate_audio_init
    cmp #ULTIMATE_OK
    bne no_audio
    jsr ultimate_audio_available
    beq no_audio
    jsr ultimate_audio_version
    sta audio_version

    setptr ult_buf, wav_name
    lda #$00
    sta ult_reu + 0
    lda #$40
    sta ult_reu + 1
    lda #$00
    sta ult_reu + 2
    sta ult_reu + 3
    lda #<voice
    ldx #>voice
    jsr ultimate_audio_load_wav
    cmp #ULTIMATE_OK
    bne failed

    lda #<voice
    ldx #>voice
    jsr ultimate_audio_configure
    cmp #ULTIMATE_OK
    bne failed
    lda voice + UA_VOICE_CHANNEL
    ldx voice + UA_VOICE_FLAGS
    jsr ultimate_audio_start

wav_name: .byte $2f,$55,$53,$42,$30,$2f,$42,$4f,$49
          .byte $4e,$47,$2e,$57,$41,$56,0
audio_version: .byte 0
voice:
    .byte 0, 0, UA_VOLUME_MAX, UA_PAN_CENTER
    .dword 0, 0, 0, 0
    .word 1
```

RESULT

audio_version receives the module byte; channel zero starts BOING.WAV.

The descriptor at the end of the listing initializes the fields the WAV loader deliberately keeps: `channel`, `volume`, `pan`, `repeat_a` and `repeat_b`. The loader fills address, length, divider and format flags. The complete field and validation contract is in *Load And Play PCM Through Ultimate Audio* in Chapter 3.

The direct assembly ABI takes the voice pointer in `A/X`. Start is the exception: channel is in `A` and its `UA_CTRL_*` flags are in `X`. Configure leaves the gate closed; start opens it.

These descriptor values cover every format and playback option:

```
    lda #0
    sta voice + UA_VOICE_FLAGS  ; 8-bit mono
    lda #UA_CTRL_16BIT
    sta voice + UA_VOICE_FLAGS  ; 16-bit mono
    lda #UA_CTRL_INTERLEAVE
    sta voice + UA_VOICE_FLAGS  ; 8-bit stereo side
    lda #UA_CTRL_16BIT | UA_CTRL_INTERLEAVE
    sta voice + UA_VOICE_FLAGS  ; 16-bit stereo side

    lda #UA_CTRL_REPEAT | UA_CTRL_IRQ
    ora voice + UA_VOICE_FLAGS
    sta voice + UA_VOICE_FLAGS
    lda #UA_PAN_LEFT    ; or CENTER, RIGHT, or 0-15
    sta voice + UA_VOICE_PAN
```

RESULT

The format choices are 8/16-bit and mono/interleaved stereo. REPEAT and IRQ combine with any format; volume accepts 0–63 and pan accepts 0–15.

Stop and clear completion

```
    jsr ultimate_audio_irq_status
    and #1 « 0    ; channel zero ended?
    beq still_playing
    lda #0
    jsr ultimate_audio_irq_clear

    lda #0
    jsr ultimate_audio_stop
```

RESULT

The channel-zero completion bit is cleared and channel zero is stopped.

Set `UA_CTRL_IRQ` before configure and start when using the completion bit. It asserts the C64 IRQ line, so clear it from an installed handler or poll with interrupts disabled. Status returns the bits for every channel; clear and stop each take a channel in `A`.

INSPECT NETWORK INTERFACES

```
    jsr ultimate_net_ifcount
    cmp #ULTIMATE_OK
    bne failed
    lda ult_iface
    beq no_interfaces

    lda #0
    sta ult_iface
    setptr ult_buf, mac
    jsr ultimate_net_macaddr
    setptr ult_buf, ipconfig
    jsr ultimate_net_ipconfig

mac: .res 6
ipconfig: .res 12
```

RESULT

ult_iface = interface count; mac receives 6 bytes; ipconfig receives 12.

The count is returned in `ult_iface`; the address calls take an interface index there and place fixed-size replies at `ult_buf`.

Change an interface's address

```
    lda #0
    sta ult_iface
    setptr ult_buf, ipconfig
    jsr ultimate_net_ipconfig
    cmp #ULTIMATE_OK
    bne failed

    lda #60
    sta ipconfig + 3
    jsr ultimate_net_setip
```

RESULT

The interface keeps its netmask and gateway and answers on .60 instead.

`ultimate_net_setip` sends back the same twelve bytes `ultimate_net_ipconfig` returned: four of address, then four of netmask, then four of gateway. It reads `ult_iface` and `ult_buf`, and always sends twelve bytes, so `ult_buflen` is not an input.

This changes the running configuration only; nothing is written to the Ultimate's stored settings.

USE TCP AND UDP SOCKETS

```
    setptr ult_buf, host
    setword ult_port, 6502
    jsr ultimate_net_connect
    cmp #ULTIMATE_OK
    bne failed

    setptr ult_buf, message
    setword ult_buflen, message_end - message
    jsr ultimate_net_write
    ; ult_socklen is now the number accepted.

host: .byte $31,$39,$32,$2e,$31,$36,$38,$2e,$31,$2e,$35,$30,0
message: .byte $48,$49,13
message_end:
```

RESULT

TCP handle is in ult_sock; ult_socklen = 3 bytes accepted.

Read and close the TCP socket

```
poll:
    setptr ult_buf, socket_buffer
    setword ult_socklen, 128
    jsr ultimate_net_read
    cmp #ULTIMATE_END
    beq peer_closed
    cmp #ULTIMATE_OK
    bne close_socket
    lda ult_socklen
    ora ult_socklen + 1
    beq poll

close_socket:
    jsr ultimate_net_close

socket_buffer: .res 128
```

RESULT

Reply bytes are in socket_buffer; close returns ULTIMATE_OK.

UDP

```
    setptr ult_buf, host
    setword ult_port, 6502
    jsr ultimate_net_udp
    cmp #ULTIMATE_OK
    bne failed
    jsr ultimate_net_close
```

RESULT

UDP handle opens and closes; A = ULTIMATE_OK.

Connect and UDP return the handle in `ult_sock`. Write takes its length from `ult_buflen` and reports accepted bytes in `ult_socklen`. Read takes capacity and returns stored bytes

through `ult_socklen`. A zero-byte success means poll again. `ULTIMATE_END` means the handle is already gone; do not close that ended handle.

USE THE HTTP CLIENT

```
    setptr ult_url, url
    setptr ult_buf, http_reply
    setword ult_buflen, 512
    jsr ultimate_http_get
    ; ult_httplen is the number stored.

url: .byte $68,$74,$74,$70,$3a,$2f,$2f,$31,$39,$32,$2e
                  .byte $31,$36,$38,$2e,$31,$2e,$35,$30,$2f,0
accept_header: .byte $61,$63,$63,$65,$70,$74,$3a,$20,$74,$65
                  .byte $78,$74,$2f,$70,$6c,$61,$69,$6e,0
http_reply: .res 512
```

RESULT

One-call GET stores response bytes; A = ULTIMATE_OK.

Multi-step GET without a body

```
setptr ult_url, url
lda #HTTP_VERB_GET
jsr ultimate_http_open
cmp #ULTIMATE_OK
bne failed

setptr ult_url, accept_header
jsr ultimate_http_header
lda #HTTP_BODY_NONE
sta ult_body
setptr ult_buf, http_reply
setword ult_buflen, 512
jsr ultimate_http_exchange
jsr ultimate_http_close
```

RESULT

GET without a body stores response bytes; close returns ULTIMATE_OK.

The one-call GET creates and releases its own slot. The multi-step form stores the request handle in `ult_http`, and header, exchange and close all read it there. `ult_body` holds the body handle, and `HTTP_BODY_NONE` in it means no body. `ultimate_http_free_all` recovers slots abandoned by an earlier program.

The next section creates a body and posts it.

All nine method constants use the same open call. This loop creates and closes one request for each without sending it:

```
    lda #0
    sta verb_index
next_verb:
    ldx verb_index
    lda verbs,x
    jsr ultimate_http_open
    cmp #ULTIMATE_OK
    bne failed
    jsr ultimate_http_close
    inc verb_index
    lda verb_index
    cmp #verbs_end - verbs
    bne next_verb

verbs: .byte HTTP_VERB_GET, HTTP_VERB_PUT, HTTP_VERB_POST
       .byte HTTP_VERB_PATCH, HTTP_VERB_DELETE, HTTP_VERB_HEAD
       .byte HTTP_VERB_OPTIONS, HTTP_VERB_CONNECT, HTTP_VERB_TRACE
verbs_end:
verb_index: .byte 0
```

RESULT

GET, PUT, POST, PATCH, DELETE, HEAD, OPTIONS, CONNECT and TRACE request slots are each created and released.

BUILD A REQUEST BODY

The body is built inside the Ultimate: create a slot, add to it one call at a time, then run the exchange. The handle lives in `ult_body`, where `ultimate_http_exchange` already looks for it, so nothing has to carry it around.

```
    lda #HTTP_BODY_JSON_OBJECT
    jsr ultimate_http_body
    cmp #ULTIMATE_OK
    bne failed

    setptr ult_url, key_name
    setptr ult_buf, value_ada
    jsr ultimate_http_body_string

    setptr ult_url, key_score
    setword ult_val, 4200
    lda #0
    sta ult_val + 2
    sta ult_val + 3
    jsr ultimate_http_body_int

key_name: .byte $6e,$61,$6d,$65,0
key_score: .byte $73,$63,$6f,$72,$65,0
value_ada: .byte $41,$44,$41,0
```

RESULT

The body holds {"name":"ADA","score":4200}.

`ult_url` is the key for every keyed call, and it is the same variable `ultimate_http_open` uses for a URL. `ult_buf` is a string value. `ult_val` is four bytes, low byte first, and all four are sent, because the firmware sign-extends from the last byte it receives.

`ultimate_http_body_bool` reads `ult_val + 0` only, and any non-zero value is true.

The other keyed root formats use the same add calls:

```
    lda #HTTP_BODY_JSON_ARRAY
    jsr ultimate_http_body
    setptr ult_url, empty_key
    setword ult_val, 1
    jsr ultimate_http_body_int
    jsr ultimate_http_body_free

    lda #HTTP_BODY_URL_ENCODED
    jsr ultimate_http_body
    setptr ult_url, key_name
    setptr ult_buf, value_ada
    jsr ultimate_http_body_string
    jsr ultimate_http_body_free

empty_key: .byte 0
```

RESULT

The first body is `[1]`; the second is the form field `name=ADA`. JSON object and binary examples cover the other two formats, while `HTTP_BODY_NONE` is passed directly to exchange.

Send it

```
    setptr ult_url, url
    lda #HTTP_VERB_POST
    jsr ultimate_http_open
    cmp #ULTIMATE_OK
    bne failed

    setptr ult_buf, http_reply
    setword ult_buflen, 512
    jsr ultimate_http_exchange
    jsr ultimate_http_close
    jsr ultimate_http_body_free
```

RESULT

The response is in http_reply; ult_httplen is the number stored.

Set `ult_buf` back to the reply buffer before the exchange. The keyed body calls used it for the last string value, and the exchange uses it for the reply.

Nest an object or an array

```
    lda #HTTP_BODY_JSON_OBJECT
    jsr ultimate_http_body

    setptr ult_url, key_stats
    jsr ultimate_http_body_object
    jsr ultimate_http_body_up

    setptr ult_url, key_runs
    jsr ultimate_http_body_array
    setptr ult_url, no_key
    setdword ult_val, 1
    jsr ultimate_http_body_int
    jsr ultimate_http_body_up
    jsr ultimate_http_body_free

key_stats: .byte $73,$74,$61,$74,$73,0
key_runs: .byte $72,$75,$6e,$73,0
no_key: .byte 0
```

RESULT

{"stats":{},"runs":[1]}

Adding an object or an array enters it. Every key added afterwards goes inside, until `ultimate_http_body_up` steps back out to the parent. Array keys are ignored, but the pointer is required; point it at a NUL byte.

Send raw bytes instead

```
    lda #HTTP_BODY_BINARY
    jsr ultimate_http_body

    setptr ult_buf, block
    setword ult_buflen, block_end - block
    jsr ultimate_http_body_binary

    jsr ultimate_http_body_clear
    jsr ultimate_http_body_free

block: .byte 1, 2, 3, 4
block_end:
```

RESULT

The body holds four bytes, then none; the slot is returned.

`ultimate_http_body_binary` is the other entry that reads `ult_buflen` as an input. Successive calls append. `ultimate_http_body_clear` empties a body and keeps its slot and format; `ultimate_http_body_free` returns the slot.

A `HTTP_BODY_BINARY` body takes `ultimate_http_body_binary` and nothing else; the JSON and form formats take everything else. A key is staged through `ult_stage`, so it is limited to `ULTIMATE_HTTP_KEY_MAX` bytes, which is 34. A longer key returns `ULTIMATE_ERR_INVALID_ARGUMENT` and never reaches the wire.

Careful: The firmware has sixteen slots for the whole machine, and a program that crashes returns none of them. Free every body and every request. `ultimate_http_free_all` takes back headers and bodies together.

PROBE AND CONFIGURE THE RAW TRANSPORT

```
    jsr uci_present
    beq no_interface
    jsr uci_ident
    sta raw_ident

    jsr uci_get_timeout_a
    pha
    lda #UCI_TIMEOUT_FOREVER
    jsr uci_set_timeout_a
    jsr run_slow_command
    tax
    pla
    jsr uci_set_timeout_a
    txa

    jsr uci_last_code
    sta device_code
    stx device_code + 1
    jsr uci_abort
```

RESULT

uci_present returns nonzero; raw_ident receives the interface ID.
device_code receives the last target status; abort returns ULTIMATE_OK.

Present and ident are direct register reads. Timeout zero waits forever; this pattern restores the previous budget while preserving the command result. Last code returns the raw target status in A/X. Abort releases an unfinished exchange and is safe while idle.

RAW COMMANDS: OPTIONAL ARGUMENTS

```
    jsr uci_req_clear
    lda #UCI_TARGET_DOS1
    sta uci_req + UCI_REQ_TARGET
    lda #UCI_CMD_IDENTIFY
    sta uci_req + UCI_REQ_COMMAND
    setptr uci_req + UCI_REQ_DATA, raw_reply
    setword uci_req + UCI_REQ_DATAMAX, 64
    jsr uci_exec_block
    jsr uci_req_clear
    lda #UCI_TARGET_DOS1
    sta uci_req + UCI_REQ_TARGET
    lda #DOS_CMD_GET_TIME
    sta uci_req + UCI_REQ_COMMAND
    setptr uci_req + UCI_REQ_ARGS, time_format
    setword uci_req + UCI_REQ_ARGLEN, 1
    jsr uci_exec_block
    jsr uci_req_clear
    lda #UCI_TARGET_CONTROL
    sta uci_req + UCI_REQ_TARGET
    lda #CTRL_CMD_GET_HWINFO
    sta uci_req + UCI_REQ_COMMAND
    setptr uci_req + UCI_REQ_DATA, raw_reply
    setword uci_req + UCI_REQ_DATAMAX, 64
    jsr uci_exec_block
    setptr uci_req + UCI_REQ_ARGS, hwinfo_model
    setword uci_req + UCI_REQ_ARGLEN, 1
    jsr uci_exec_block
time_format: .byte 0
hwinfo_model: .byte CTRL_HWINFO_MODEL
raw_reply: .res 64
```

RESULT

IDENTIFY returns the target name.
GET_TIME returns the formatted time.
Both GET_HWINFO forms return the model name.

Clearing the request leaves every optional span empty. The time command then supplies its optional format byte. `GET_HWINFO` is sent without and then with its model selector, which is what an omitted optional argument means on the wire. Set other spans only when needed.

Use a caller-owned request block

```
    lda #<my_request
    ldx #>my_request
    jsr uci_exec

my_request: .res UCI_REQ_SIZE
```

RESULT

A = request result; lengths and retained bytes are written into my_request.

`uci_exec` takes the request pointer directly. Use it instead of the global block when the program owns more than one request.

READ A RAW MULTI-BLOCK REPLY

```
    lda #<my_request
    ldx #>my_request
    jsr uci_exec_first
    cmp #ULTIMATE_OK
    bne failed

next_block:
    ; use this block's UCI_REQ_DATALEN bytes
    lda uci_more
    beq all_blocks
    jsr uci_exec_next
    cmp #ULTIMATE_OK
    beq next_block
```

RESULT

Each iteration receives one block; uci_more = 0 after the final block.

First submits the request and returns one block. `uci_more` is the boolean state byte that says whether another follows; next releases the current block and receives it. Issue no unrelated command until the final block or call `uci_abort` when stopping early.

CHAPTER 5

OTHER TOOLCHAINS

- Building and identifying the blob
- The stable jump table
- The parameter block
- Ultimate Audio and software-vsprite extension entries
- Calling it from BASIC
- Choosing a different base address

BUILD AND IDENTIFY THE BLOB

The blob is the SDK linked as one binary with a table of jump instructions at a fixed address. Any assembler or compiler can call it, because calling it needs no linker and no object format: you load the binary and `jsr` into the table.

`make blob` produces `bindings/blob/build/ultimate-7000.bin`. Load it at `$7000`. The first three bytes are the PETSCII signature `UCI`; the fourth is the version of the ABI, the calling interface.

> *Note:* The blob has to sit below `$A000`, because `$A000–$BFFF` is BASIC ROM. A program can reach every byte of a blob at `$7000` with the machine exactly as it found it. One that ran past `$A000` would have to switch BASIC ROM out and back around every call.

> **Careful:** `$7000` is inside BASIC's memory, not above it. BASIC owns `$0801–$9FFF`: the program area grows up from `$0801` and strings and arrays grow down from `$9FFF`. A program that has replaced BASIC owns that memory instead and needs to do nothing. A program that leaves BASIC running must lower the top of BASIC memory to the base address before it loads the blob, or BASIC will write over it. *Call The Blob From BASIC*, later in this chapter, shows the two `POKE`s and the `CLR` that do it.

This example checks the default build before calling it:

```
lda $7000
cmp #$D5    ; PETSCII 'U'
bne no_sdk
lda $7001
cmp #$C3    ; PETSCII 'C'
bne no_sdk
lda $7002
cmp #$C9    ; PETSCII 'I'
bne no_sdk
lda $7003
cmp #1      ; ABI version
bne wrong_version
```

RESULT

All four comparisons match; execution continues with ABI version 1.

The version byte changes only when an existing entry changes meaning. New entries

are appended, so a program written against version 1 keeps working when the table grows.

CALL THE STABLE JUMP TABLE

Entries begin at base plus `$04`. Each entry is a three-byte `jmp`; new entries are appended so published offsets do not move.

```
    jsr $7004    ; +$04 uci_init
    jsr $701C    ; +$1C ultimate_init
```

RESULT

A = ULTIMATE_OK after each initialization call.

These are the two initialization entries. Use `ultimate_init` for application code; the lower-level `uci_init` entry is exposed for transport users. Both return a result in `A`.

The two kinds of entry

The table has two halves, and the difference decides how you read a result.

Entries from `+$04` to `+$4C` pass their arguments in registers, the way the assembly chapter's entry points do, and return the result in `A` and nowhere else.

Entries from `+$4F` upwards take their arguments from the parameter block described in the next section, because a caller cannot pass a filename in `A` and `X`. Each of these also stores its result in `bp_result` before returning it in `A`.

That second group is what a BASIC program can drive, because `SYS` can neither pass a register nor read one back. The extension tables at `+$2E8` and `+$300` contain both kinds; their individual contracts say whether they use registers or the parameter block.

The complete offset table is in `bindings/blob/README.md`. Read the offsets there rather than counting entries: the table gained the file information, disk image, clock, machine and HTTP body services after the offsets printed in this chapter were published.

USE THE PARAMETER BLOCK

Service entries that need several arguments use a 512-byte block at base plus `$100`. These are the first four fields:

Offset	Field	Purpose
`+$00`	`bp_result`	last service result
`+$05`	`bp_len`	length in or out
`+$29`	`bp_name`	40-byte area for a terminated name
`+$51`	`bp_reply`	256-byte reply area

Later fields extend beyond the first page while retaining their published offsets. Consult the blob README for the complete layout rather than deriving an address from this subset.

CALL AUDIO AND VSPRITE EXTENSIONS

Ultimate Audio occupies the extension entries from `+$2E8` through `+$300`. The normal sequence is `ultimate_audio_init`, optionally `audio_load_wav`, then `audio_configure` and `audio_start`. Configure, start, stop and clear use the `ultimate_audio_voice` structure at `BLOB + BLOB_BP_AUDIO`; the WAV entry also uses `BLOB_BP_NAME` and `BLOB_BP_REU`. Chapter 3 defines every voice field and the safe configure/start/stop sequence. The complete stable offset table is in `bindings/blob/README.md`.

The software-vsprite entry is `BLOB + BLOB_VSPRITE_DRAW`, currently `+$303`. Put the descriptor address in `BLOB_BP_ADDR` and read the result from `BLOB_BP_RESULT`:

```
BLOB = $7000

    lda #<sprite
    sta BLOB + BLOB_BP_ADDR
    lda #>sprite
    sta BLOB + BLOB_BP_ADDR + 1
    jsr BLOB + BLOB_VSPRITE_DRAW
    lda BLOB + BLOB_BP_RESULT
    bne draw_failed

sprite:
    .word $6000, image, mask, 0
    .byte 12, 72, 2, 8, 0, VSPRITE_F_MASKED
```

RESULT

The masked image is composited into the bitmap; the parameter result is zero.

The descriptor and flag contract is the one in *Composite A Software Vsprite* in Chapter 3. This is a parameter-block shim around the same assembly routine, not another renderer.

CALL THE BLOB FROM BASIC

The page-aligned block gives BASIC a calling convention it can express with `POKE`, `PEEK` and `SYS`. Before any of that, BASIC has to be told to stay out of the memory the blob occupies. Type this first, then load the blob:

```
POKE 56,112 : POKE 55,0 : CLR
```

112 is $70, the high byte of the base address, and 55 and 56 are the low and high bytes of BASIC's top-of-memory pointer at $37–$38. CLR is what makes it take effect, because it resets the string pointer from the new top. BASIC then keeps $0801–$6FFF, and the blob's code at $7000 and its variables at $9F00 are both out of its reach. Loading the blob without this leaves it where BASIC's strings and arrays will eventually be written.

This program then checks the signature, brings the SDK up, confirms that an Ultimate answered, and prints the size of the RAM expansion:

```
10 B = 28672 : P = B + 256
20 IF PEEK(B)<>213 OR PEEK(B+1)<>195 THEN PRINT "NO SDK":END
25 IF PEEK(B+2)<>201 THEN PRINT "NO SDK":END
30 SYS B + 28
40 SYS B + 82 : IF PEEK(P) THEN PRINT "NO ULTIMATE":END
50 SYS B + 175
60 PRINT "REU BANKS:"; PEEK(P+5) + PEEK(P+6)*256
```

Line 10 sets B to the blob's base, decimal 28672 for $7000, and P to the parameter block a page above it. Lines 20 and 25 compare the three signature bytes; 213, 195 and 201 are $D5, $C3 and $C9.

Line 30 calls ultimate_init at offset $1C. That is a register entry, so it returns its result in A and writes nothing to the block, and SYS cannot read A. Line 40 therefore calls getpath at offset $52, which is a parameter-block entry: it asks the Ultimate for the current directory and leaves the result in bp_result, so PEEK(P) is the first result this program can test. A non-zero value there means no Ultimate answered.

Line 50 calls reu_size at offset $AF, which puts the 16-bit bank count in bp_len. Line 60 reads it back as two bytes, low byte first, and prints it. A machine with a 2MB expansion prints 32; one with no expansion prints 0.

> **Careful:** Do not check PEEK(P) after a register entry such as $1C. Those entries never write bp_result, so the byte still holds whatever the previous call left there, and in a freshly loaded blob that is zero. The test would pass whether the call worked or not.

CHOOSE A DIFFERENT BASE ADDRESS

Prefer building at the final base: make -C bindings/blob BASE=6000 links a $6000 image directly, and VARS and ZP move the SDK's RAM the same way.

When the address is known only at run time, the build also emits a relocation table beside the binary for reloc.s to apply. The relocated emulator suite copies the default blob

to `$6000`, relocates the copy, calls it, and then compares it byte for byte against an image the linker built for `$6000` directly.

CHAPTER 6

BASIC KEYWORD REFERENCE

- Every wedge statement, function and constant
- Exact syntax, tokens, results and side effects
- One short example for every entry

COMMON RULES

All entries require the BASIC wedge from Chapter 2. Custom statements work in direct and program modes, but BASIC V2 cannot dispatch one immediately after `THEN`; branch to a line containing the statement instead. None of the custom keywords has an abbreviated entry.

Statements report their SDK result through `UERR`; target-specific status is retained in `UDEV`. Numeric syntax errors remain ordinary BASIC errors. Dedicated filename statements retain at most 63 filename bytes. The raw `UCI` statement retains 256 reply bytes, 40 status bytes and 64 argument bytes.

RAW COMMAND INTERFACE

UCI

BASIC statement

Syntax: `UCI` or `UCI` *target, command* [`,` *argument* `...`]

Modes: Direct and program

Token: `$CC` (204)

Result: `UERR`, `UDEV`, `ULEN`, `UST$`, `UDAT$` and `UBYTE(`

Purpose

Executes any published UCI command. A bare `UCI` aborts an unfinished exchange and returns the interface to idle. Target and command are bytes. String arguments contribute their bytes; numeric arguments normally contribute one byte. Known commands automatically widen arguments required by their wire format. `UW(` and `UL(` force widths explicitly.

Examples

```
10 UCI UCTRL,1
20 PRINT UERR;ULEN;UDAT$
```

OUTPUT

```
0 14 CONTROL TARGET
```

```
10 UCI UDOS1,38
20 UCI UDOS1,38,0
30 UCI UDOS1,240,UW(4660),UL(65536)
```

RESULT

The first two calls demonstrate an omitted and explicit optional argument. The third sends 4660 as two bytes and 65536 as four, both least-significant byte first.

UERR

BASIC numeric function

Syntax: `UERR`

Modes: Direct and program

Token: `$CD` (205)

Returns: The last `ULTIMATE_*` result; zero is `ULTIMATE_OK`

Purpose

Tests whether the most recent wedge statement succeeded. It is initialized to zero when the wedge starts. Appendix A defines every result code.

```
10 ULOAD "FONT.PRG",49152
20 IF UERR THEN PRINT "LOAD FAILED";UERR
```

RESULT

The program continues silently when the load succeeds and prints the SDK result when it fails.

UDEV

BASIC numeric function

Syntax: `UDEV`

Modes: Direct and program

Token: `$CE` (206)

Returns: The target's last raw status number, from 0 through 65535

Purpose

Adds firmware-specific detail to diagnostics. Do not use it for normal program control; use `UERR`, whose meanings are stable across targets.

```
10 UCI UDOS1,1
20 IF UERR THEN PRINT "DEVICE STATUS";UDEV
```

RESULT

UDEV is consulted only after UERR reports failure.

ULEN

BASIC numeric function

Syntax: `ULEN`

Modes: Direct and program

Token: `$CF` (207)

Returns: The retained reply length, from 0 through 256

Purpose

Reports how many data bytes the most recent `UCI` command retained. It is zero before the first command. A longer reply retains the first 256 bytes and sets `UERR` to `ULTIMATE_ERR_TRUNCATED`.

```
10 UCI UCTRL,1
20 FOR I=0 TO ULEN-1:PRINT UBYTE(I);:NEXT
```

RESULT

The loop visits exactly the retained reply bytes.

UST$

BASIC string function

Syntax: `UST$`

Modes: Direct and program

Token: `$D0` (208)

Returns: The status bytes from the most recent `UCI` command as a BASIC string

Purpose

Exposes the target's status text exactly as received, up to the wedge's 40-byte status buffer. Use `UERR` and `UDEV` when a stable numeric test is needed.

```
10 UCI UCTRL,1
20 PRINT UST$
```

OUTPUT

```
00,OK
```

UDAT$

BASIC string function

Syntax: `UDAT$`

Modes: Direct and program

Token: `$D1` (209)

Returns: Up to 255 retained reply bytes as a BASIC string

Purpose

Returns textual or binary reply data in one value. BASIC strings cannot contain more than 255 bytes, so use `UBYTE(255)` to inspect the final byte of a 256-byte retained reply.

```
10 UCI UCTRL,1
20 PRINT UDAT$
```

OUTPUT

```
CONTROL TARGET
```

UBYTE(

BASIC numeric function

Syntax: `UBYTE(`*index*`)`

Modes: Direct and program

Token: `$D2` (210), including the opening parenthesis

Returns: Reply byte *index*, or zero when the index is outside the retained reply

Purpose

Reads binary replies without converting them to strings. Valid retained indices are 0 through 255.

```
10 UCI UCTRL,1
20 PRINT UBYTE(0);UBYTE(1);UBYTE(999)
```

OUTPUT

```
67 79 0
```

UW(

BASIC UCI argument modifier

Syntax: `UW(`*numeric expression*`)`

Valid only: As an argument inside a `UCI` statement

Token: `$D3` (211), including the opening parenthesis

Produces: Two little-endian argument bytes

Purpose

Overrides the raw command's normal argument width. It is useful for firmware commands newer than the wedge's shape table.

```
UCI UDOS1,240,UW(4660)
```

RESULT

The argument bytes are 52,18 ($34,$12).

UL(

BASIC UCI argument modifier

Syntax: `UL(`*numeric expression*`)`

Valid only: As an argument inside a `UCI` statement

Token: `$D4` (212), including the opening parenthesis

Produces: Four little-endian argument bytes

Purpose

Forces a signed 32-bit BASIC value into the raw argument stream.

```
UCI UDOS1,240,UL(65536)
```

RESULT

The argument bytes are 0,0,1,0.

TARGET CONSTANTS

UDOS1

BASIC numeric constant

Syntax: `UDOS1`

Token: `$D5` (213)

Returns: 1, the first Ultimate DOS target

```
UCI UDOS1,1
```

RESULT

The IDENTIFY command is sent to the primary filesystem service.

UDOS2

BASIC numeric constant

Syntax: `UDOS2`

Token: `$D6` (214)

Returns: 2, the second Ultimate DOS target

```
UCI UDOS2,1
```

> **RESULT**
>
> The IDENTIFY command is sent to the optional second filesystem service.

UNET

BASIC numeric constant

Syntax: `UNET`

Token: `$D7` (215)

Returns: 3, the network target

```
UCI UNET,1
```

> **RESULT**
>
> The IDENTIFY command is sent to the network service.

UCTRL

BASIC numeric constant

Syntax: `UCTRL`

Token: `$D8` (216)

Returns: 4, the machine and cartridge-control target

```
UCI UCTRL,1
```

RESULT

The IDENTIFY command is sent to the control service.

UIEC

BASIC numeric constant

Syntax: `UIEC`

Token: `$D9` (217)

Returns: 5, the SoftwareIEC target

```
UCI UIEC,1
```

RESULT

The IDENTIFY command is sent to the high-speed IEC service.

UHTTP

BASIC numeric constant

Syntax: `UHTTP`

Token: `$DA` (218)

Returns: 6, the HTTP target

```
UCI UHTTP,1
```

RESULT

The IDENTIFY command is sent to the HTTP client service.

TURBO CONTROL

UTURBO

BASIC statement and numeric function

Syntax: `UTURBO` *index* or `UTURBO`

Modes: Direct and program

Token: `$DB` (219)

Statement result: `UERR`

Function result: Current speed index, or 255 when turbo registers are unavailable

Purpose

Sets or reads the Ultimate 64 CPU-speed index. Indices 0, 1, 2 and 3 portably mean 1, 2, 3 and 4 MHz. Indices 4 through 15 are model-dependent. The statement leaves the badline setting unchanged.

```
10 S=UTURBO:IF S=255 THEN PRINT "NO TURBO":END
20 UTURBO 0:UTURBO 1:UTURBO 2:UTURBO 3
30 UTURBO 15:IF UERR THEN END
40 PRINT UTURBO
50 UTURBO S
```

OUTPUT

```
15
```

The first four settings demonstrate the portable 1, 2, 3 and 4 MHz speeds; 15 selects the model's maximum. Indices 4–14 are accepted but deliberately not assigned a frequency here because their meanings differ by model.

FILES AND DIRECTORIES

ULOAD

BASIC statement

Syntax: `ULOAD` *filename$* [, *address*]

Modes: Direct and program

Token: `$DC` (220)

Result: `UERR`; raw target status in `UDEV`

Purpose

Loads a PRG from the current Ultimate directory. Its two-byte header is always consumed. Omitting the address, or supplying zero, uses the header's address; any other 16-bit value overrides it.

```
10 ULOAD "FONT.PRG"
20 IF UERR THEN PRINT "LOAD FAILED";UERR;UDEV
30 ULOAD "FONT.PRG",49152
```

RESULT

The first call uses the PRG's address. The second loads its body beginning at $C000. The caller must protect live memory at either destination.

UBLOAD

BASIC statement

Syntax: `UBLOAD` *filename$, address, length*

Modes: Direct and program

Token: `$DD` (221)

Result: `UERR`; raw target status in `UDEV`

Purpose

Loads at most *length* raw file bytes into a nonzero 16-bit C64 address. It does not consume or interpret a PRG header. Address and length must both be nonzero.

```
10 UBLOAD "FONT.BIN",49152,2048
20 IF UERR THEN PRINT "LOAD FAILED";UERR;UDEV
```

RESULT

At most 2048 bytes are written beginning at $C000.

USAVE

BASIC statement

Syntax: `USAVE` *filename$, start, length*

Modes: Direct and program

Token: `$DE` (222)

Result: `UERR`; raw target status in `UDEV`

Purpose

Creates or replaces a file with *length* bytes beginning at the 16-bit C64 address *start*. Length must be nonzero.

```
10 USAVE "SCREEN.BIN",1024,1000
20 IF UERR THEN PRINT "SAVE FAILED";UERR;UDEV
```

RESULT

SCREEN.BIN receives the 1000-byte text screen.

UDIR

BASIC statement

Syntax: `UDIR`

Modes: Direct and program

Token: `$DF` (223)

Result: `UERR`; directory names are printed as they arrive

Purpose

Lists the current Ultimate directory, one entry per line. A slash follows each directory. Normal end-of-directory leaves `UERR` at zero.

```
UDIR
```

OUTPUT

```
TEMP/
```

The output above is the controlled root directory used by the hardware test; real directory contents vary.

RAM EXPANSION

USTASH

BASIC statement

Syntax: `USTASH` *C64 address, REU address, length*

Modes: Direct and program

Token: `$E0` (224)

Result: `UERR`

Purpose

Copies C64 memory into the RAM expansion with DMA. The C64 address is 16-bit, the REU address is 24-bit, and the length is 16-bit; a zero length means 65536 bytes. The transfer is complete when the statement returns.

```
10 USTASH 1024,0,1000
20 IF UERR THEN PRINT "NO REU"
```

RESULT

The 1000-byte text screen is copied to REU address zero.

UFETCH

BASIC statement

Syntax: `UFETCH` *C64 address, REU address, length*

Modes: Direct and program

Token: `$E1` (225)

Result: `UERR`

Purpose

Copies data from the RAM expansion into C64 memory. Address widths and the zero-means-65536 length convention are the same as `USTASH`.

```
10 UFETCH 1024,0,1000
20 IF UERR THEN PRINT "NO REU"
```

> **RESULT**
>
> The first 1000 REU bytes replace the C64 text screen.

UREU

BASIC numeric function

Syntax: `UREU`

Modes: Direct and program

Token: `$E2` (226)

Returns: Installed REU size in 64K banks, or zero when unavailable

Purpose

Tests for a RAM expansion and measures it in one operation. Multiply by 64 for kilobytes.

```
10 PRINT UREU
20 PRINT UREU*64
```

OUTPUT

```
32
2048
```

This is the actual result with the hardware-test configuration set to a 2 MB REU; the value

follows the machine's setting.

CHAPTER 7

C API REFERENCE

- Every public structure, function and capability macro
- Exact prototypes, parameters, return values and side effects
- Short, compilable call examples

COMMON RULES

Include `ultimate.h` for application services and `uci.h` only for raw transport calls. Unless an entry says otherwise, a `uint8_t` service returns `ULTIMATE_OK` on success or an `ULTIMATE_ERR_*` value from Appendix A. Caller-supplied buffers remain owned by the caller and must stay valid until the call returns. The SDK allocates no memory. For UCI-backed calls, an entry's Returns field names call-specific outcomes; transport and target failures may also return the applicable Appendix A error.

Call `ultimate_init()` once before any UCI-backed service. Turbo, REU DMA and software vsprites state their separate requirements. cc65 character maps string literals; uppercase filename/path literals are ASCII-compatible, while case-sensitive ASCII URLs, headers, keys and values should be constructed as bytes or converted at run time.

SDK DATA TYPES

The following entries define every public structure used by the C API. Function parameter descriptions link back here; the structure definitions in `ultimate.h` and `uci.h` remain the compiler's authority.

ultimate_capabilities

C structure

Header: `ultimate.h`

Filled by `ultimate_detect()`; callers normally allocate it once and keep it for feature checks.

Field	Meaning
`uint8_t present`	1 when a working command interface answered.
`uint8_t ident`	Raw UCI identification-register value.
`uint16_t targets`	Bit N is set when UCI target N identified itself.

Example

```
ultimate_capabilities caps;
uint8_t err = ultimate_detect(&caps);
if (err == ULTIMATE_OK && ultimate_has_http(&caps)) {
    /* HTTP target is available */
}
```

ultimate_sid

C structure

Header: `ultimate.h`

One raw SID mapping returned inside `ultimate_sid_info`. It describes addresses and firmware mapping type; it does not identify a 6581 versus 8580.

Field	Meaning
`uint16_t primary_address`	Primary memory-mapped SID address.
`uint16_t secondary_address`	Second mapping, or zero when absent.
`uint8_t type`	Raw implementation/address-mask code; preserve unknown values.

Example

```
ultimate_sid_info info;
ultimate_sid *sid;
uint8_t err = ultimate_legacy_get_sid_info(&info);
if (err == ULTIMATE_OK && info.count != 0)
    sid = &info.sid[0];
```

ultimate_sid_info

C structure

Header: `ultimate.h`

Count-prefixed result from deprecated control HWINFO. At most `ULTIMATE_SID_MAX` (4) records are retained.

Field	Meaning
`uint8_t count`	Valid entries in `sid`; zero on any call failure.
`ultimate_sid sid[ULTIMATE_SID_MAX]`	SID mapping records; only indices below `count` are valid.

Example

```
ultimate_sid_info info;
uint8_t err = ultimate_legacy_get_sid_info(&info);
if (err == ULTIMATE_OK && info.count != 0) {
    /* info.sid[0] is valid */
}
```

ultimate_vsprite

C structure

Header: `ultimate.h`

A complete software-vsprite draw descriptor. It is read only during `ultimate_vsprite_draw()`; the bitmap, and optionally screen memory, are changed in place. The descriptor is 14 bytes in the cc65 ABI.

Field	Meaning
`uint8_t *bitmap`	Base of a standard 8000-byte C64 bitmap.
`const uint8_t *source`	Image bytes in column-major order; for `VSPRITE_F_COPY`, the base of another C64 bitmap.
`const uint8_t *mask`	Column-major mask required by `VSPRITE_F_MASKED` and ignored otherwise; the destination is ANDed with the mask, then the source is ORed into it.
`uint8_t *screen`	Screen-memory base required by `VSPRITE_F_COLOR` and ignored otherwise.
`uint8_t x`	Destination bitmap byte/cell column, 0–39; this is not a pixel coordinate.
`uint8_t y`	Destination raster line, 0–199.
`uint8_t width`	Width in bitmap byte columns; must be nonzero and fit from `x`.
`uint8_t height`	Height in raster lines; must be nonzero and fit from `y`.
`uint8_t color`	Screen byte written to cells touched by visible source bytes in colour mode.
`uint8_t flags`	0 for OR, or exactly one of `VSPRITE_F_MASKED`, `VSPRITE_F_COPY`, or `VSPRITE_F_COLOR`.

For ordinary and masked draws, source (and mask) layout is every raster line of byte column zero, followed by every line of byte column one, and so on. The caller supplies pre-shifted images when sub-cell horizontal movement is needed.

Complete masked-draw example

```
static const uint8_t image[8] =
    { 0x18, 0x3c, 0x7e, 0xff, 0xff, 0x7e, 0x3c, 0x18 };
static const uint8_t mask[8] =
    { 0xe7, 0xc3, 0x81, 0x00, 0x00, 0x81, 0xc3, 0xe7 };
ultimate_vsprite sprite;
uint8_t err;

sprite.bitmap = (uint8_t *)0xe000;
sprite.source = image;
sprite.mask = mask;
sprite.screen = (uint8_t *)0xcc00;
sprite.x = 18; /* byte column */
sprite.y = 80; /* raster line */
sprite.width = 1; /* 8 pixels */
sprite.height = 8;
sprite.color = 2; /* ignored in masked mode */
sprite.flags = VSPRITE_F_MASKED;
err = ultimate_vsprite_draw(&sprite);
```

RESULT

The 8-by-8 image is composited at bitmap byte column 18, raster line 80. Set `flags` to 0, `VSPRITE_F_COPY`, or `VSPRITE_F_COLOR` for the other three operations and populate the fields described above.

ultimate_fileinfo

C structure

Header: `ultimate.h`

Metadata returned by `ultimate_stat()` and `ultimate_fstat()`.

Field	Meaning
`uint32_t size`	File size in bytes.
`uint16_t date`	Packed FAT date: year minus 1980, month and day.
`uint16_t time`	Packed FAT time: hour, minute and two-second units.
`char extension[3]`	Three space-padded bytes; not NUL-terminated.
`uint8_t attrib`	`DOS_ATTR_*` bits.
`char name[ULTIMATE_NAME_MAX + 1]`	NUL-terminated name, bounded by `ULTIMATE_NAME_MAX`.

Example

```
ultimate_fileinfo info;
uint8_t err = ultimate_stat("DEMO.PRG", &info);
if (err == ULTIMATE_OK) {
    /* info.name, info.size and info.attrib are valid */
}
```

ultimate_audio_voice

C structure

Header: `ultimate.h`

One Ultimate Audio PCM channel descriptor. Configuration reads all fields and leaves the channel stopped. The descriptor is `UA_VOICE_SIZE` (22) bytes in the cc65 ABI.

Field	Meaning
`uint8_t channel`	Channel 0 through `UA_CHANNELS-1`.
`uint8_t flags`	`UA_CTRL_*` format/repeat/IRQ bits, excluding `UA_CTRL_GATE`.
`uint8_t volume`	0 through `UA_VOLUME_MAX`.
`uint8_t pan`	`UA_PAN_LEFT` through `UA_PAN_RIGHT`.
`uint32_t reu_address`	First PCM byte in the 24-bit REU window.
`uint32_t length`	PCM byte count, 1 through $FFFFFF.
`uint32_t repeat_a`	Loop-start byte offset relative to `reu_address`.
`uint32_t repeat_b`	Loop-end byte offset relative to `reu_address`.
`uint16_t rate`	Nonzero divider: round(6250000 / samples per second).

Values 7 and 8 are both centred pan positions. The address and byte count may end at 16 MB but may not wrap beyond it. A 16-bit or interleaved voice needs an even length. With `UA_CTRL_REPEAT`, the required relation is `repeat_a < repeat_b <= length`.

Complete 8-bit mono example

```
ultimate_audio_voice voice;
uint8_t err;
voice.channel = 0;
voice.flags = 0;
voice.volume = UA_VOLUME_MAX;
voice.pan = UA_PAN_CENTER;
voice.reu_address = 0x4000UL;
voice.length = 8000UL;
voice.repeat_a = 0;
voice.repeat_b = 0;
voice.rate = 781; /* about 8000 samples/second */
err = ultimate_audio_configure(&voice);
if (err == ULTIMATE_OK)
    err = ultimate_audio_start(voice.channel, voice.flags);
```

ultimate_drive

C structure

Header: `ultimate.h`

One drive or occupied IEC-bus record inside `ultimate_drives`.

Field	Meaning
`uint8_t type`	`CTRL_DRVTYPE_*`; SoftwareIEC and printer values describe bus slots, not physical drives.
`uint8_t device`	IEC device number accepted by mount operations.
`uint8_t power`	1 while the drive is running, otherwise 0.

Example

```
ultimate_drives drives;
ultimate_drive *drive;
uint8_t err = ultimate_drive_info(&drives);
if (err == ULTIMATE_OK && drives.count != 0)
    drive = &drives.drive[0];
```

ultimate_drives

C structure

Header: `ultimate.h`

Count-prefixed result from `ultimate_drive_info()`.

Field	Meaning
`uint8_t count`	Valid records; never greater than records actually received or `ULTIMATE_DRIVES_MAX`.
`ultimate_drive drive[ULTIMATE_DRIVES_MAX]`	Drive/bus records; only indices below `count` are valid.

Example

```
ultimate_drives drives;
uint8_t err = ultimate_drive_info(&drives);
if (err == ULTIMATE_OK && drives.count != 0) {
    /* drives.drive[0] is valid */
}
```

uci_request

C structure

Header: `uci.h`

One low-level command/reply exchange. All pointed-to spans remain owned by the caller. The cc65 layout is `UCI_REQ_SIZE` (22) bytes and matches the generated assembly offsets.

Field	Meaning
`uint8_t target`	`UCI_TARGET_*`, optionally ORed with `UCI_TARGET_NO_REPLY`.
`uint8_t command`	Target-specific command byte.
`const uint8_t *args`	Optional fixed argument bytes.
`uint16_t arglen`	Byte count at `args`; use zero with NULL.
`const uint8_t *payload`	Optional payload span sent after arguments.
`uint16_t payloadlen`	Byte count at `payload`; use zero with NULL.
`uint8_t *data`	Reply-data buffer, or NULL to discard without a truncation result.
`uint16_t datamax`	Capacity of `data`.
`uint16_t datalen`	Output: reply-data bytes retained.
`uint8_t *status`	Optional raw status buffer.
`uint16_t statusmax`	Capacity of `status`.
`uint16_t statuslen`	Output: status bytes retained.

Complete IDENTIFY example

```
uint8_t reply[64];
uint8_t status[16];
uci_request req = { 0 };
uint8_t err;
req.target = UCI_TARGET_DOS1;
req.command = UCI_CMD_IDENTIFY;
req.data = reply;
req.datamax = sizeof reply;
req.status = status;
req.statusmax = sizeof status;
err = uci_exec(&req);
```

RESULT

On success, `reply[0..req.datalen-1]` contains the identification data and `status[0..req.statuslen-1]` contains the raw target status.

INITIALIZATION AND DISCOVERY

ultimate_init()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_init(void)`

Parameters

None.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_NO_DEVICE`, or `ULTIMATE_ERR_TIMEOUT`.

Purpose

Initializes the SDK and functionally checks the command interface. Call it once before other UCI-backed services.

Example

```
uint8_t err = ultimate_init();
```

RESULT

The SDK is ready only when `err == ULTIMATE_OK`.

ultimate_available()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_available(void)`

Parameters

None.

Returns

1 after successful initialization; otherwise 0.

Purpose

Reads cached SDK state without touching hardware.

Example

```
if (!ultimate_available()) return 1;
```

RESULT

The program exits when initialization has not succeeded.

ultimate_detect()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_detect(ultimate_capabilities *caps)`

Parameters

`caps` points to writable `ultimate_capabilities` storage.

Returns

`ULTIMATE_OK` after probing all known targets, including when some are absent; `ULTIMATE_ERR_INVALID_ARGUMENT` for a null `caps`; or the `ULTIMATE_ERR_NO_DEVICE`/`ULTIMATE_ERR_TIMEOUT` result from initializing the transport.

Purpose

Fills `present`, `ident`, and the target bit mask. It costs one UCI round trip per known target, so normally call it once.

Example

```
ultimate_capabilities caps; err = ultimate_detect(&caps);
```

RESULT

`caps.targets` identifies the services actually implemented.

ultimate_identify()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_identify(uint8_t target, char *buf, uint16_t buflen, uint16_t *outlen)`

Parameters

`target` is `UCI_TARGET_*`; `buf` has `buflen` bytes; `outlen` may be `NULL`.

Returns

`ULTIMATE_OK`, or `ULTIMATE_ERR_NOT_SUPPORTED` when the target is absent.

Purpose

Reads a target's identification string, stores at most `buflen-1` bytes, and terminates it. A supplied `outlen` excludes the terminator.

Example

```
err = ultimate_identify(UCI_TARGET_DOS1, text, sizeof text, &length);
```

RESULT

`text` contains the DOS identity and `length` its stored length.

ultimate_get_model()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_get_model(char *buf, uint16_t buflen, uint16_t *outlen)`

Parameters

`buf` has `buflen` bytes; `outlen` may be `NULL`.

Returns

`ULTIMATE_OK` or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Reads the hardware model through the deprecated control HWINFO command. Retained for compatibility; new code must handle its disappearance.

Example

```
err = ultimate_get_model(text, sizeof text, NULL);
```

RESULT

`text` is terminated when the call succeeds.

ultimate_legacy_get_sid_info()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_legacy_get_sid_info(ultimate_sid_info *info)`

Parameters

`info` points to writable `ultimate_sid_info` storage.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_NOT_SUPPORTED`, `ULTIMATE_ERR_PROTOCOL`, or `ULTIMATE_ERR_TRUNCATED`; `info->count` is zero on failure.

Purpose

Reads up to four SID mapping records through deprecated HWINFO. Type bytes are raw implementation/mapping codes, not 6581/8580 model identifiers.

Example

```
ultimate_sid_info info; err = ultimate_legacy_get_sid_info(&info);
```

RESULT

On success, `info.sid[0].primary_address` is the first SID address.

CAPABILITY MACROS

ultimate_has_target()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_has_target(caps, target)`

Parameters

`caps` points to `ultimate_capabilities` filled by `ultimate_detect()`; `target` is a target number.

Returns

1 when that target bit is set, otherwise 0.

Purpose

Tests an arbitrary target without another hardware transaction.

Example

```
if (ultimate_has_target(&caps, UCI_TARGET_HTTP)) use_http();
```

RESULT

The branch runs only when detection found the HTTP target.

ultimate_has_dos()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_has_dos(caps)`

Parameters

`caps` points to detected `ultimate_capabilities`.

Returns

1 when DOS target 1 exists.

Purpose

Tests the primary filesystem service.

Example

```
if (ultimate_has_dos(&caps)) list_files();
```

RESULT

Filesystem code runs only when DOS1 was detected.

ultimate_has_dos2()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_has_dos2(caps)`

Parameters

`caps` points to detected `ultimate_capabilities`.

Returns

1 when DOS target 2 exists.

Purpose

Tests the optional second filesystem service.

Example

```
if (ultimate_has_dos2(&caps)) inspect_second_dos();
```

RESULT

The branch is skipped on machines exposing only DOS1.

ultimate_has_network()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_has_network(caps)`

Parameters

`caps` points to detected `ultimate_capabilities`.

Returns

1 when the network target exists.

Purpose

Tests socket-service availability.

Example

```
if (ultimate_has_network(&caps)) connect_peer();
```

RESULT

Network code runs only when its target was detected.

ultimate_has_control()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_has_control(caps)`

Parameters

`caps` points to detected `ultimate_capabilities`.

Returns

1 when the control target exists.

Purpose

Tests machine-control service availability.

Example

```
if (ultimate_has_control(&caps)) read_drives();
```

RESULT

Control operations are attempted only when supported.

ultimate_has_softiec()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_has_softiec(caps)`

Parameters

`caps` points to detected `ultimate_capabilities`.

Returns

1 when the SoftwareIEC target identifies itself.

Purpose

Reports target presence, not whether a particular fast load will work; `ultimate_load()` performs the honest command-level test.

Example

```
uint8_t present = ultimate_has_softiec(&caps);
```

> **RESULT**
>
> `present` describes detection only.

ultimate_has_http()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_has_http(caps)`

Parameters

`caps` points to detected `ultimate_capabilities`.

Returns

1 when the HTTP target exists.

Purpose

Tests firmware HTTP-client availability.

Example

```
if (ultimate_has_http(&caps)) fetch_page();
```

> **RESULT**
>
> HTTP code runs only when its target was detected.

PALETTE, TURBO AND VSPRITE GRAPHICS

ultimate_palette_get()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_palette_get(uint8_t *palette)`

Parameters

`palette` points to `ULTIMATE_PALETTE_BYTES` writable bytes.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_NOT_SUPPORTED`, or `ULTIMATE_ERR_PROTOCOL` unless a full 48-byte reply arrives.

Purpose

Reads sixteen live RGB triples.

Example

```
uint8_t          palette[ULTIMATE_PALETTE_BYTES];          err          =
ultimate_palette_get(palette);
```

RESULT

`palette[0..2]` is colour zero's red, green and blue.

ultimate_palette_set()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_palette_set(const uint8_t *palette)`

Parameters

`palette` points to 48 bytes containing sixteen RGB triples.

Returns

`ULTIMATE_OK` or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Replaces the live palette in one command without writing flash or a VPL file.

Example

```
err = ultimate_palette_set(palette);
```

RESULT

All sixteen live colours change when the call succeeds.

ultimate_palette_set_color()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_palette_set_color(uint8_t index, uint8_t r, uint8_t g, uint8_t b)`

Parameters

`index` is 0–15; RGB components are 0–255.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_INVALID_ARGUMENT` for another index, or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Changes one live palette colour.

Example

```
err = ultimate_palette_set_color(6, 255, 0, 0);
```

RESULT

Colour index 6 becomes red.

ultimate_palette_reset()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_palette_reset(void)`

Parameters

None.

Returns

`ULTIMATE_OK` or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Restores the machine's built-in live palette.

Example

```
err = ultimate_palette_reset();
```

RESULT

Temporary palette changes are removed.

ultimate_turbo_available()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_turbo_available(void)`

Parameters

None; `ultimate_init()` is not required.

Returns

1 when the Ultimate 64 turbo registers answer, otherwise 0.

Purpose

Tests the owner's Turbo Control setting through memory-mapped hardware.

Example

```
if (!ultimate_turbo_available()) run_at_one_mhz();
```

> **RESULT**
>
> The fallback runs on a plain C64 or when turbo registers are disabled.

ultimate_turbo_get()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_turbo_get(void)`

Parameters

None.

Returns

The current speed index, or `U64_TURBO_UNAVAILABLE` ($FF).

Purpose

Reads the CPU-speed index without changing the badline setting.

Example

```
uint8_t old = ultimate_turbo_get();
```

> **RESULT**
>
> `old` can later restore the entry speed when it is not $FF.

ultimate_turbo_set()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_turbo_set(uint8_t index)`

Parameters

`index` is 0–15. Constants through `U64_SPEED_4MHZ` are portable; `U64_SPEED_MAX` means this machine's maximum.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_INVALID_ARGUMENT`, or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Sets CPU speed and leaves badlines unchanged.

Example

```
err = ultimate_turbo_set(U64_SPEED_1MHZ);
err = ultimate_turbo_set(U64_SPEED_2MHZ);
err = ultimate_turbo_set(U64_SPEED_3MHZ);
err = ultimate_turbo_set(U64_SPEED_4MHZ);
err = ultimate_turbo_set(U64_SPEED_MAX);
```

RESULT

The calls demonstrate every named portable speed and the model's maximum; unnamed indices 4–14 remain model-dependent.

ultimate_turbo_badlines()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_turbo_badlines(uint8_t on)`

Parameters

Zero disables badlines; any nonzero value restores them.

Returns

`ULTIMATE_OK` or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Changes VIC badline cycle stealing without changing CPU speed. Check the real display before shipping with badlines disabled.

Example

```
err = ultimate_turbo_badlines(0); err = ultimate_turbo_badlines(1);
```

RESULT

The two calls demonstrate both settings.

ultimate_vsprite_draw()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_vsprite_draw(const ultimate_vsprite *sprite)`

Parameters

`sprite` points to a complete `ultimate_vsprite` descriptor; its linked type entry defines every field, layout and valid operation.

Returns

`ULTIMATE_OK` or `ULTIMATE_ERR_INVALID_ARGUMENT`.

Purpose

Composites an OR, masked, bitmap-copy, or colour-aware software vsprite. It needs no Ultimate or UCI, executes from RAM, self-modifies, and is not re-entrant.

Example

```
sprite.flags = 0; err = ultimate_vsprite_draw(&sprite);
sprite.flags = VSPRITE_F_MASKED; err = ultimate_vsprite_draw(&sprite);
sprite.flags = VSPRITE_F_COPY; err = ultimate_vsprite_draw(&sprite);
sprite.flags = VSPRITE_F_COLOR; err = ultimate_vsprite_draw(&sprite);
```

RESULT

The four calls demonstrate OR, masked, bitmap-copy and colour-aware draws.

DIRECTORIES AND FILES

ultimate_chdir()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_chdir(const char *path)`

Parameters

`path` is a NUL-terminated absolute or relative path.

Returns

An `ULTIMATE_*` result.

Purpose

Changes the current Ultimate DOS directory.

Example

```
err = ultimate_chdir("/USB1/GAMES");
```

RESULT

Later filesystem calls use that directory.

ultimate_getpath()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_getpath(char *buf, uint16_t buflen, uint16_t *outlen)`

Parameters

`buf` has `buflen` bytes; `outlen` may be `NULL`.

Returns

An `ULTIMATE_*` result; clipping to `buflen-1` is still success.

Purpose

Returns the current path terminated, and optionally its stored length.

Example

```
err = ultimate_getpath(path, sizeof path, &length);
```

RESULT

`path` contains the current directory.

ultimate_opendir()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_opendir(void)`

Parameters

None.

Returns

An `ULTIMATE_*` result.

Purpose

Begins a block-by-block walk of the current directory. Issue no unrelated command until it reaches `ULTIMATE_END` or is aborted.

Example

```
err = ultimate_opendir();
```

RESULT

A successful call prepares the first `ultimate_readdir()`.

ultimate_readdir()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_readdir(char *name, uint16_t namelen, uint8_t *attrib)`

Parameters

`name` has `namelen` bytes; `attrib` may be `NULL`.

Returns

`ULTIMATE_OK` for one entry, `ULTIMATE_END` after the last, or an error.

Purpose

Consumes one directory reply block, terminates the name, and optionally stores `DOS_ATTR_*`. Call `uci_abort()` when stopping early.

Example

```
err = ultimate_readdir(name, sizeof name, &attrib);
```

RESULT

On `ULTIMATE_OK`, `name` and `attrib` describe exactly one entry; call again until `ULTIMATE_END`.

ultimate_open()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_open(const char *name, uint8_t attrib)`

Parameters

`name` is terminated; `attrib` combines `DOS_FA_READ`, `DOS_FA_WRITE`, `DOS_FA_CREATE_NEW`, or `DOS_FA_CREATE_ALWAYS`.

Returns

An `ULTIMATE_*` result.

Purpose

Opens a firmware-side current file. Create-always replaces; create-new fails when the name already exists.

Example

```
err = ultimate_open("INPUT", DOS_FA_READ);
err = ultimate_open("OUTPUT", DOS_FA_WRITE);
err = ultimate_open("BOTH", DOS_FA_READ | DOS_FA_WRITE);
err = ultimate_open("NEW", DOS_FA_WRITE | DOS_FA_CREATE_NEW);
err = ultimate_open("REPLACE", DOS_FA_WRITE | DOS_FA_CREATE_ALWAYS);
```

RESULT

The calls demonstrate read, write, read/write, create-new and create-or-replace modes.

ultimate_close()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_close(void)`

Parameters

None.

Returns

An `ULTIMATE_*` result; closing with nothing open is harmless.

Purpose

Closes the current firmware-side file.

Example

```
err = ultimate_close();
```

RESULT

The current file handle is released.

ultimate_read()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_read(uint8_t *buf, uint16_t len, uint16_t *outlen)`

Parameters

`buf` has `len` bytes; `outlen` may be `NULL`; a file is open.

Returns

An `ULTIMATE_*` result. A short successful read means end-of-file.

Purpose

Reads up to `len` bytes, walking the firmware's reply blocks internally.

Example

```
err = ultimate_read(buf, sizeof buf, &got);
```

RESULT

`got` is the actual byte count when supplied.

ultimate_write()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_write(const uint8_t *buf, uint16_t len)`

Parameters

`buf` points to `len` bytes; a writable file is open.

Returns

An `ULTIMATE_*` result.

Purpose

Writes the complete caller span to the current file.

Example

```
err = ultimate_write(message, sizeof message);
```

RESULT

The bytes are appended or written at the current file position.

ultimate_seek()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_seek(uint32_t pos)`

Parameters

`pos` is an absolute byte position in the current open file.

Returns

An `ULTIMATE_*` result.

Purpose

Moves the firmware-side file position.

Example

```
err = ultimate_seek(1024UL);
```

> **RESULT**
>
> The next read or write begins at byte 1024.

ultimate_delete()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_delete(const char *name)`

Parameters

`name` is a terminated current-directory name or path.

Returns

An `ULTIMATE_*` result.

Purpose

Deletes one file.

Example

```
err = ultimate_delete("RESULT.TXT");
```

> **RESULT**
>
> RESULT.TXT is removed on success.

ultimate_stat()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_stat(const char *name, ultimate_fileinfo *info)`

Parameters

`name` is terminated; `info` points to writable `ultimate_fileinfo` storage.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_PROTOCOL` for a short reply, or another error.

Purpose

Reads size, packed FAT date/time, extension, attributes and terminated name.

Example

```
ultimate_fileinfo info; err = ultimate_stat("DEMO.PRG", &info);
```

> **RESULT**
>
> `info.size` is valid only after success.

ultimate_fstat()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_fstat(ultimate_fileinfo *info)`

Parameters

`info` points to writable `ultimate_fileinfo` storage; a file is open.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_PROTOCOL`, or another error.

Purpose

Reads metadata for the current open file.

Example

```
ultimate_fileinfo info; err = ultimate_fstat(&info);
```

RESULT

The open file's metadata fills `info`.

ultimate_rename()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_rename(const char *from, const char *to)`

Parameters

Both pointers name terminated entries in the current directory.

Returns

An `ULTIMATE_*` result.

Purpose

Renames one entry without copying its contents through the C64.

Example

```
err = ultimate_rename("OLD.TXT", "NEW.TXT");
```

RESULT

OLD.TXT becomes NEW.TXT.

ultimate_copy()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_copy(const char *name, const char *destination_path)`

Parameters

`name` is a current-directory source; `destination_path` names the target directory.

Returns

An `ULTIMATE_*` result.

Purpose

Copies inside the Ultimate and preserves the source filename.

Example

```
err = ultimate_copy("NEW.TXT", "/USB1/BACKUP");
```

RESULT

The destination is /USB1/BACKUP/NEW.TXT.

ultimate_mkdir()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_mkdir(const char *name)`

Parameters

`name` is a terminated new directory name.

Returns

An `ULTIMATE_*` result.

Purpose

Creates one directory in the current directory.

Example

```
err = ultimate_mkdir("SAVES");
```

RESULT

SAVES is created.

ultimate_home()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_home(void)`

Parameters

None.

Returns

`ULTIMATE_OK` or `ULTIMATE_ERR_NOT_SUPPORTED` when no home is configured.

Purpose

Changes to the Ultimate's configured home directory.

Example

```
err = ultimate_home();
```

RESULT

Later filesystem calls use the configured home.

DISK IMAGES AND CLOCK

ultimate_mount()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_mount(uint8_t device, const char *image)`

Parameters

`device` is an IEC number or `ULTIMATE_DRIVE_LAST`; `image` is a terminated D64, D71, D81, G64 or G71 path.

Returns

An `ULTIMATE_*` result; a disabled drive normally reports `ULTIMATE_ERR_DEVICE`.

Purpose

Mounts an image in the selected emulated drive.

Example

```
err = ultimate_mount(8, "GAMES/PITFALL.D64");
```

RESULT

PITFALL.D64 is mounted in the drive answering as IEC device 8.

ultimate_unmount()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_unmount(uint8_t device)`

Parameters

`device` is an IEC number or `ULTIMATE_DRIVE_LAST`.

Returns

An `ULTIMATE_*` result.

Purpose

Removes the current image from a drive.

Example

```
err = ultimate_unmount(8);
```

RESULT

The drive answering as device 8 becomes empty.

ultimate_swap()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_swap(uint8_t device)`

Parameters

`device` is an IEC number or `ULTIMATE_DRIVE_LAST`.

Returns

An `ULTIMATE_*` result.

Purpose

Advances a multi-image set the same way as the Ultimate menu.

Example

```
err = ultimate_swap(8);
```

RESULT

Device 8 selects its next image.

ultimate_get_time()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_get_time(uint8_t format, char *buf, uint16_t buflen, uint16_t *outlen)`

Parameters

`format` is `ULTIMATE_TIME_PLAIN` or `ULTIMATE_TIME_WEEKDAY`; `buf` has `buflen` bytes; `outlen` may be `NULL`.

Returns

An `ULTIMATE_*` result.

Purpose

Reads the battery-backed clock as terminated firmware-formatted text.

Example

```
err = ultimate_get_time(ULTIMATE_TIME_PLAIN, now, sizeof now, &length);
err = ultimate_get_time(ULTIMATE_TIME_WEEKDAY, now, sizeof now, &length);
```

RESULT

Plain output resembles `2026/08/22 14:30:00`; weekday output prefixes a day name.

ultimate_set_time()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_set_time(uint8_t year_1900, uint8_t month, uint8_t day, uint8_t hour, uint8_t minute, uint8_t second)`

Parameters

The year is calendar year minus 1900; remaining arguments are month, day, hour, minute and second.

Returns

An `ULTIMATE_*` result.

Purpose

Sets the battery-backed clock.

Example

```
err = ultimate_set_time(126, 8, 22, 14, 30, 0);
```

RESULT

The clock becomes 2026-08-22 14:30:00.

LOADING AND SAVING MEMORY

ultimate_load()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_load(const char *name, uint16_t addr)`

Parameters

`name` is a non-null terminated PRG path; zero `addr` uses its header, and a nonzero value overrides it.

Returns

An `ULTIMATE_*` result; after success `ultimate_last_end()` supplies the end address.

Purpose

Consumes the two-byte PRG header and loads its body. The SDK first tries SoftwareIEC and transparently falls back to DOS reads.

Example

```
err = ultimate_load("GAME.PRG", 0); err = ultimate_load("FONT.PRG",
0xc000);
```

RESULT

The calls demonstrate header-selected and caller-selected addresses.

ultimate_bload()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_bload(const char *name, uint16_t addr, uint16_t max)`

Parameters

`name` is terminated; `addr` and `max` must be nonzero.

Returns

An `ULTIMATE_*` result; `ultimate_last_end()` supplies the end address after success.

Purpose

Loads at most `max` raw bytes without consuming a header.

Example

```
err = ultimate_bload("FONT.BIN", 0xc000, 2048);
```

RESULT

At most 2048 bytes are written from $C000.

ultimate_save()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_save(const char *name, uint16_t start, uint16_t len)`

Parameters

`name` is terminated; `start` is a C64 address; `len` is nonzero.

Returns

An `ULTIMATE_*` result.

Purpose

Creates or replaces a file with a C64 memory range.

Example

```
err = ultimate_save("SCREEN.BIN", 0x0400, 1000);
```

RESULT

SCREEN.BIN receives the 1000-byte text screen.

ultimate_last_end()

C function

Header: `ultimate.h`

Prototype: `uint16_t ultimate_last_end(void)`

Parameters

None.

Returns

The address immediately following the last successful `ultimate_load()` or `ultimate_bload()`.

Purpose

Retrieves cached load state without touching hardware.

Example

```
uint16_t end = ultimate_last_end();
```

RESULT

`end-1` is the final address written by the preceding successful load.

RAM EXPANSION

ultimate_reu_available()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_reu_available(void)`

Parameters

None; UCI initialization is not required.

Returns

1 when the REU registers answer, otherwise 0.

Purpose

Tests whether RAM expansion is enabled.

Example

```
if (!ultimate_reu_available()) return ULTIMATE_ERR_NOT_SUPPORTED;
```

RESULT

The program refuses an REU-dependent path when no expansion is mapped.

ultimate_reu_size()

C function

Header: `ultimate.h`

Prototype: `uint16_t ultimate_reu_size(void)`

Parameters

None.

Returns

Size in 64K banks, or zero when unavailable.

Purpose

Measures the expansion non-destructively by testing alias boundaries.

Example

```
uint16_t banks = ultimate_reu_size();
```

RESULT

2 means 128 KB, 16 means 1 MB, 128 means 8 MB and 256 means 16 MB.

ultimate_reu_stash()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_reu_stash(uint16_t addr, uint32_t reuaddr, uint16_t len)`

Parameters

`addr` is a C64 address; `reuaddr` is within 24 bits; zero `len` means 65536 bytes.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_NOT_SUPPORTED`, or `ULTIMATE_ERR_INVALID_ARGUMENT`.

Purpose

DMA-copies C64 memory into the expansion and returns after completion.

Example

```
err = ultimate_reu_stash(0x0400, 0, 1000);
```

RESULT

The text screen is copied to REU address zero.

ultimate_reu_fetch()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_reu_fetch(uint16_t addr, uint32_t reuaddr, uint16_t len)`

Parameters

`addr` is a C64 address; `reuaddr` is within 24 bits; zero `len` means 65536 bytes.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_NOT_SUPPORTED`, or `ULTIMATE_ERR_INVALID_ARGUMENT`.

Purpose

DMA-copies expansion data into C64 memory.

Example

```
err = ultimate_reu_fetch(0x0400, 0, 1000);
```

RESULT

The first 1000 REU bytes replace the text screen.

ultimate_reu_load()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_reu_load(uint32_t reuaddr, uint32_t len)`

Parameters

A readable file is open; `reuaddr` and `len` describe a range within the 16 MB REU window.

Returns

An `ULTIMATE_*` result.

Purpose

Transfers the current open file directly into the expansion. It may take a long time and restores the caller's UCI timeout.

Example

```
err = ultimate_open("ASSETS.BIN", DOS_FA_READ); err = ultimate_reu_load(0, 65536UL);
```

RESULT

The first 65536 file bytes move to REU address zero without passing through C64 RAM.

ultimate_reu_save()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_reu_save(uint32_t reuaddr, uint32_t len)`

Parameters

A writable file is open; `reuaddr` and `len` describe a range within the 16 MB REU window.

Returns

An `ULTIMATE_*` result.

Purpose

Transfers expansion data directly into the current open file and restores the caller's timeout.

Example

```
err = ultimate_open("CAPTURE.BIN", DOS_FA_WRITE | DOS_FA_CREATE_ALWAYS);
err = ultimate_reu_save(0, 65536UL);
```

RESULT

65536 REU bytes are written without passing through C64 RAM.

Program tip: Keep large level data, decompression sources, sampled audio and screen backups in the REU. Use stash/fetch for small live-memory swaps and load/save when the data should move directly between storage and expansion memory.

ULTIMATE AUDIO

ultimate_audio_init()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_init(void)`

Parameters

Ultimate Audio must be mapped at $DF20–$DFFF.

Returns

`ULTIMATE_OK` or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Probes the PCM engine with a silent byte. Call once before other audio calls.

Example

```
err = ultimate_audio_init();
```

RESULT

Audio services are usable only after success.

ultimate_audio_available()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_available(void)`

Parameters

Call `ultimate_audio_init()` first.

Returns

1 after successful audio initialization, otherwise 0.

Purpose

Reads cached audio state without touching hardware.

Example

```
if (!ultimate_audio_available()) return 1;
```

RESULT

The program exits when the PCM engine is unavailable.

ultimate_audio_version()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_version(void)`

Parameters

Audio must be available.

Returns

The raw module-version byte.

Purpose

Reads the mapped engine's version register.

Example

```
uint8_t version = ultimate_audio_version();
```

RESULT

`version` is meaningful only after audio initialization succeeds.

ultimate_audio_configure()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_configure(const ultimate_audio_voice *voice)`

Parameters

`voice` points to a fully initialized `ultimate_audio_voice`; its linked type entry defines all fields and constraints.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_INVALID_ARGUMENT`, or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Stops, validates and programs one voice, leaving its gate closed. Flags select 8/16-bit, mono/interleaved, repeat and IRQ operation.

Example

```
voice.flags = 0; err = ultimate_audio_configure(&voice);
voice.flags = UA_CTRL_16BIT; err = ultimate_audio_configure(&voice);
voice.flags         =         UA_CTRL_INTERLEAVE;         err         =
ultimate_audio_configure(&voice);
voice.flags = UA_CTRL_REPEAT; err = ultimate_audio_configure(&voice);
voice.flags = UA_CTRL_IRQ; err = ultimate_audio_configure(&voice);
```

> **RESULT**
>
> The calls isolate 8-bit mono, 16-bit, stereo interleave, repeat and completion IRQ settings; valid flags may be combined.

ultimate_audio_start()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_start(uint8_t channel, uint8_t flags)`

Parameters

`channel` is 0 through `UA_CHANNELS-1`; `flags` repeats only the configured `UA_CTRL_FLAGS` format/playback bits. Do not include `UA_CTRL_GATE`; the call adds it.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_INVALID_ARGUMENT`, or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Opens a configured channel's gate immediately.

Example

```
err = ultimate_audio_start(0, UA_CTRL_16BIT | UA_CTRL_REPEAT);
```

RESULT

Channel zero begins playing.

ultimate_audio_stop()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_stop(uint8_t channel)`

Parameters

`channel` is 0 through `UA_CHANNELS-1`.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_INVALID_ARGUMENT`, or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Closes one channel's gate immediately.

Example

```
err = ultimate_audio_stop(0);
```

RESULT

Channel zero stops.

ultimate_audio_irq_status()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_irq_status(void)`

Parameters

Audio must be available.

Returns

A bit mask with one end-of-sample bit per channel.

Purpose

Reads latched PCM completion state.

Example

```
uint8_t ended = ultimate_audio_irq_status();
```

RESULT

`ended & (1u << channel)` is nonzero when that channel completed.

ultimate_audio_irq_clear()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_irq_clear(uint8_t channel)`

Parameters

`channel` is 0 through `UA_CHANNELS-1`.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_INVALID_ARGUMENT`, or `ULTIMATE_ERR_NOT_SUPPORTED`.

Purpose

Clears one channel's latched completion bit and its asserted IRQ.

Example

```
err = ultimate_audio_irq_clear(channel);
```

RESULT

That channel's completion bit is clear.

ultimate_audio_load_wav()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_audio_load_wav(const char *name, uint32_t reuaddr, ultimate_audio_voice *voice)`

Parameters

`name` is a terminated PCM WAV path; `reuaddr` begins its REU data; `voice` points to `ultimate_audio_voice` and preserves caller channel, volume, pan and repeat points.

Returns

DOS errors, `ULTIMATE_ERR_PROTOCOL` for broken RIFF, `ULTIMATE_ERR_NOT_SUPPORTED` for unsupported format/rate/range, or an REU error. The voice is invalid on failure.

Purpose

Loads 8/16-bit mono/stereo PCM, fixes unsigned 8-bit samples in place, closes the file on every path, and fills address, length, rate and format flags.

Example

```
err = ultimate_audio_load_wav("/USB1/BOING.WAV", 0x4000UL, &voice);
```

RESULT

On success, configure and start `voice`; for stereo derive the right channel as described in Chapter 3.

MACHINE AND DRIVE CONTROL

ultimate_reboot()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_reboot(void)`

Parameters

None.

Returns

Does not return.

Purpose

Resets the C64; the calling program stops existing during the command.

Example

```
ultimate_reboot(); /* no following statement runs */
```

RESULT

The machine restarts.

ultimate_freeze()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_freeze(void)`

Parameters

None.

Returns

An `ULTIMATE_*` result after the user leaves the menu.

Purpose

Invokes the Ultimate freeze menu and waits for human release.

Example

```
err = ultimate_freeze();
```

RESULT

Execution resumes after the menu closes.

ultimate_drive_enable()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_drive_enable(uint8_t drive, uint8_t on)`

Parameters

`drive` is `ULTIMATE_DRIVE_A` or `ULTIMATE_DRIVE_B`; zero `on` disables, nonzero enables.

Returns

An `ULTIMATE_*` result.

Purpose

Switches one physical emulated drive off or on.

Example

```
err = ultimate_drive_enable(ULTIMATE_DRIVE_A, 0); err = ultimate_drive_enable(ULTIMATE_DRIVE_A, 1);
```

RESULT

The calls demonstrate both states.

ultimate_drive_power()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_drive_power(uint8_t drive, uint8_t *on)`

Parameters

`drive` is A or B; `on` is a required writable pointer.

Returns

An `ULTIMATE_*` result; `*on` is 1 while running.

Purpose

Reads one physical drive's power state.

Example

```
err = ultimate_drive_power(ULTIMATE_DRIVE_A, &running);
```

> **RESULT**
>
> `running` contains zero or one after success.

ultimate_drive_info()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_drive_info(ultimate_drives *drives)`

Parameters

`drives` points to writable `ultimate_drives` storage.

Returns

An `ULTIMATE_*` result; count never exceeds records actually received.

Purpose

Reports type, IEC device number and power for disk drives and occupied IEC slots. SoftwareIEC and printer records are bus slots, not physical drives.

Example

```
ultimate_drives drives; err = ultimate_drive_info(&drives);
```

> **RESULT**
>
> `drives.drive[i].device` is the number accepted by mount operations.

ultimate_ramdisk_info()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_ramdisk_info(uint8_t *info)`

Parameters

`info` has `CTRL_RAMDISK_BYTES` writable bytes.

Returns

An `ULTIMATE_*` result.

Purpose

Reads GEOS MegaPatch RAM-disk records: type followed by size in 64K units.

Example

```
uint8_t info[CTRL_RAMDISK_BYTES]; err = ultimate_ramdisk_info(info);
```

RESULT

A record type of zero marks an empty slot.

NETWORK SOCKETS

ultimate_net_ifcount()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_ifcount(uint8_t *count)`

Parameters

`count` is a required writable pointer.

Returns

An `ULTIMATE_*` result; `*count` is valid after success.

Purpose

Reports the number of network interfaces.

Example

```
err = ultimate_net_ifcount(&count);
```

RESULT

Indices zero through `count-1` may be queried.

ultimate_net_macaddr()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_macaddr(uint8_t iface, uint8_t *mac)`

Parameters

`iface` is an interface index; `mac` has `UCI_NET_MACADDR_BYTES` writable bytes.

Returns

An `ULTIMATE_*` result.

Purpose

Reads the six-byte hardware address.

Example

```
uint8_t mac[UCI_NET_MACADDR_BYTES]; err = ultimate_net_macaddr(0, mac);
```

RESULT

`mac[0..5]` contains interface zero's address.

ultimate_net_ipconfig()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_ipconfig(uint8_t iface, uint8_t *ipconfig)`

Parameters

`iface` is an interface index; `ipconfig` has 12 writable bytes.

Returns

An `ULTIMATE_*` result.

Purpose

Reads IPv4 address, netmask and gateway as three four-byte network-order addresses.

Example

```
uint8_t ip[UCI_NET_IPCONFIG_BYTES]; err = ultimate_net_ipconfig(0, ip);
```

RESULT

`ip[0..3]`, `ip[4..7]` and `ip[8..11]` hold the three addresses.

ultimate_net_setip()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_setip(uint8_t iface, const uint8_t *ipconfig)`

Parameters

`iface` is an interface index; `ipconfig` supplies the same 12-byte address/netmask/gateway layout returned above.

Returns

An `ULTIMATE_*` result.

Purpose

Changes only the running network configuration; it does not write stored settings or detect address conflicts.

Example

```
err = ultimate_net_setip(0, ip);
```

RESULT

Interface zero immediately uses the supplied configuration.

ultimate_net_connect()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_connect(const char *host, uint16_t port, uint8_t *handle)`

Parameters

`host` is terminated; `port` is a TCP port; `handle` is required.

Returns

An `ULTIMATE_*` result; `*handle` receives the socket on success.

Purpose

Opens a TCP socket. DNS/connect may take about 30 seconds, so this call waits without the normal timeout limit and restores the caller's budget.

Example

```
err = ultimate_net_connect("192.0.2.1", 80, &handle);
```

RESULT

`handle` names the connected TCP socket.

ultimate_net_udp()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_udp(const char *host, uint16_t port, uint8_t *handle)`

Parameters

`host` is terminated; `port` is a UDP port; `handle` is required.

Returns

An `ULTIMATE_*` result; `*handle` receives the socket on success.

Purpose

Opens a connected UDP socket with the same unbounded-connect timeout handling as the TCP call.

Example

```
err = ultimate_net_udp("192.0.2.1", 6502, &handle);
```

RESULT

`handle` names the UDP socket.

ultimate_net_close()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_close(uint8_t handle)`

Parameters

`handle` names an open socket.

Returns

An `ULTIMATE_*` result.

Purpose

Closes a socket explicitly. Do not close after `ultimate_net_read()` returns `ULTIMATE_END`; the firmware already removed it.

Example

```
err = ultimate_net_close(handle);
```

RESULT

The socket handle is released.

ultimate_net_read()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_read(uint8_t handle, uint8_t *buf, uint16_t bufsize, uint16_t *got)`

Parameters

`handle` is open; `buf` has `bufsize` bytes; `got` is required. The SDK refuses unsafe requests above `UCI_NET_READ_MAX`.

Returns

`ULTIMATE_OK` with zero or more bytes, `ULTIMATE_END` once when the peer closes, `ULTIMATE_ERR_TRUNCATED` when a UDP datagram was longer than the request on newer firmware, or another error.

Purpose

Polls without waiting. At most `bufsize-UCI_NET_READ_PREFIX` data bytes fit because the

firmware prefixes its count.

Example

```
err = ultimate_net_read(handle, buf, sizeof buf, &got);
```

RESULT

Use retained data when `err == ULTIMATE_OK` or `ULTIMATE_ERR_TRUNCATED` and `got != 0`; stop on `ULTIMATE_END`. A truncated UDP datagram cannot be resumed.

ultimate_net_write()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_net_write(uint8_t handle, const uint8_t *buf, uint16_t len, uint16_t *sent)`

Parameters

`handle` is open; `buf` supplies `len` bytes; `sent` is required.

Returns

An `ULTIMATE_*` result; `*sent` is the accepted count.

Purpose

Writes one span and reports rather than assumes how much the firmware took.

Example

```
err = ultimate_net_write(handle, buf, len, &sent);
```

RESULT

After success, compare `sent` with `len`.

Program tip: Use a small state machine around nonblocking reads: connect once, poll during the program's normal update loop, consume available bytes, and treat `ULTIMATE_END` as the terminal state.

HTTP REQUESTS

ultimate_http_get()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_get(const char *url, uint8_t *buf, uint16_t bufsize, uint16_t *got)`

Parameters

`url` is terminated ASCII; `buf` has `bufsize` bytes; `got` is required.

Returns

`ULTIMATE_OK` for HTTP below 400, `ULTIMATE_ERR_DEVICE` for HTTP errors, `ULTIMATE_ERR_TRUNCATED` when the body is larger, or another transport error.

Purpose

Creates, executes and frees a GET request. It waits through DNS/connect and restores the timeout. The HTTP status remains in `ultimate_device_code()`.

Example

```
err = ultimate_http_get(url, buf, sizeof buf, &got);
```

RESULT

`buf[0..got-1]` contains the retained body, including an HTTP error body.

ultimate_http_open()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_open(uint8_t verb, const char *url, uint8_t *handle)`

Parameters

`verb` is any `HTTP_VERB_*` from GET through TRACE; `url` is terminated ASCII; `handle` is required.

Returns

An `ULTIMATE_*` result; `*handle` receives a request slot.

Purpose

Creates a request for later headers and exchange.

Example

```
err = ultimate_http_open(HTTP_VERB_GET, url, &request);
err = ultimate_http_open(HTTP_VERB_PUT, url, &request);
err = ultimate_http_open(HTTP_VERB_POST, url, &request);
err = ultimate_http_open(HTTP_VERB_PATCH, url, &request);
err = ultimate_http_open(HTTP_VERB_DELETE, url, &request);
err = ultimate_http_open(HTTP_VERB_HEAD, url, &request);
err = ultimate_http_open(HTTP_VERB_OPTIONS, url, &request);
err = ultimate_http_open(HTTP_VERB_CONNECT, url, &request);
err = ultimate_http_open(HTTP_VERB_TRACE, url, &request);
```

RESULT

Each call allocates a separate request using the named HTTP method; close each successful handle.

ultimate_http_header()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_header(uint8_t handle, const char *line)`

Parameters

`handle` is an open request; `line` is terminated `key: value` ASCII.

Returns

An `ULTIMATE_*` result.

Purpose

Adds one HTTP header. Call repeatedly for multiple fields.

Example

```
static const char header[] = {
0x61,0x63,0x63,0x65,0x70,0x74,0x3a,0x20,
0x61,0x70,0x70,0x6c,0x69,0x63,0x61,0x74,0x69,0x6f,0x6e,0x2f,
0x6a,0x73,0x6f,0x6e,0};
err = ultimate_http_header(request, header);
```

RESULT

The header is attached to the request slot.

ultimate_http_exchange()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_exchange(uint8_t handle, uint8_t body, uint8_t *buf, uint16_t bufsize, uint16_t *got)`

Parameters

`handle` is an open request; `body` is a body handle or `HTTP_BODY_NONE`; `buf` has `bufsize` bytes; `got` is required.

Returns

The same HTTP/result/truncation rules as `ultimate_http_get()`.

Purpose

Sends the prepared request. It waits through DNS/connect and restores the timeout, but does not free the header or body slots.

Example

```
err = ultimate_http_exchange(request, HTTP_BODY_NONE, buf, sizeof buf,
&got);
```

RESULT

A request without a body completes and retains up to `sizeof buf` bytes.

ultimate_http_close()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_close(uint8_t handle)`

Parameters

`handle` names an open request/header slot.

Returns

An `ULTIMATE_*` result.

Purpose

Returns one request slot to the firmware.

Example

```
err = ultimate_http_close(request);
```

RESULT

The request slot is reusable.

ultimate_http_free_all()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_free_all(void)`

Parameters

None.

Returns

An `ULTIMATE_*` result.

Purpose

Frees every HTTP header and body slot held by the machine. Use for startup recovery; it may also invalidate slots owned by another C64 program.

Example

```
err = ultimate_http_free_all();
```

RESULT

All sixteen firmware slots become available.

HTTP REQUEST BODIES

ultimate_http_body()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body(uint8_t format, uint8_t *handle)`

Parameters

`format` is `HTTP_BODY_BINARY`, `HTTP_BODY_JSON_OBJECT`, `HTTP_BODY_JSON_ARRAY`, or `HTTP_BODY_URL_ENCODED`; `handle` is required.

Returns

An `ULTIMATE_*` result; `*handle` receives the new slot.

Purpose

Creates an empty firmware-side body in any supported format. Binary bodies accept only the binary append call; the other formats accept typed items.

Example

```
err = ultimate_http_body(HTTP_BODY_BINARY, &body);
err = ultimate_http_body(HTTP_BODY_JSON_OBJECT, &body);
err = ultimate_http_body(HTTP_BODY_JSON_ARRAY, &body);
err = ultimate_http_body(HTTP_BODY_URL_ENCODED, &body);
```

RESULT

The calls create binary, JSON-object, JSON-array and URL-encoded bodies; free every successful handle.

ultimate_http_body_free()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_free(uint8_t handle)`

Parameters

`handle` names a body slot.

Returns

An `ULTIMATE_*` result.

Purpose

Returns one body slot to the firmware.

Example

```
err = ultimate_http_body_free(body);
```

RESULT

The body slot is reusable.

ultimate_http_body_clear()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_clear(uint8_t handle)`

Parameters

`handle` names a body slot.

Returns

An `ULTIMATE_*` result.

Purpose

Empties a body while retaining its slot and format.

Example

```
err = ultimate_http_body_clear(body);
```

RESULT

The body can be rebuilt without allocating another slot.

ultimate_http_body_up()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_up(uint8_t handle)`

Parameters

`handle` names a JSON/form body currently inside a child container.

Returns

An `ULTIMATE_*` result.

Purpose

Leaves the current object or array and returns to its parent.

Example

```
err = ultimate_http_body_up(body);
```

RESULT

Subsequent items are added beside the completed child container.

ultimate_http_body_int()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_int(uint8_t handle, const char *key, int32_t value)`

Parameters

`handle` names a nonbinary body; `key` is non-NULL and at most `ULTIMATE_HTTP_KEY_MAX` bytes. Inside arrays pass an empty string; the pointed-to key is ignored.

Returns

An `ULTIMATE_*` result, including `ULTIMATE_ERR_INVALID_ARGUMENT` for an overlong key.

Purpose

Adds a signed 32-bit integer.

Example

```
static const char key[] = {0x73,0x63,0x6f,0x72,0x65,0};
err = ultimate_http_body_int(body, key, -6502L);
```

RESULT

An object gets `"score":-6502`; for an array, pass an empty key string and only the value is added.

ultimate_http_body_bool()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_bool(uint8_t handle, const char *key, uint8_t value)`

Parameters

`handle` names a nonbinary body; `key` follows the same rules; zero is false and nonzero is true.

Returns

An `ULTIMATE_*` result.

Purpose

Adds a Boolean value.

Example

```
static const char key[] = {0x72,0x65,0x61,0x64,0x79,0};
err = ultimate_http_body_bool(body, key, 1);
err = ultimate_http_body_bool(body, key, 0);
```

RESULT

The calls add true and false values respectively.

ultimate_http_body_string()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_string(uint8_t handle, const char *key, const char *value)`

Parameters

`handle` names a nonbinary body; `key` follows the same rules; `value` is terminated and at most 255 bytes.

Returns

An `ULTIMATE_*` result.

Purpose

Adds one string value.

Example

```
static const char key[] = {0x6e,0x61,0x6d,0x65,0};
static const char value[] = {0x62,0x6f,0x69,0x6e,0x67,0};
err = ultimate_http_body_string(body, key, value);
```

RESULT

The object gets `"name":"boing"`.

ultimate_http_body_object()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_object(uint8_t handle, const char *key)`

Parameters

`handle` names a nonbinary body; `key` follows the same rules.

Returns

An `ULTIMATE_*` result.

Purpose

Adds an object and enters it. Call `ultimate_http_body_up()` to leave.

Example

```
static const char key[] = {0x70,0x6c,0x61,0x79,0x65,0x72,0};
err = ultimate_http_body_object(body, key);
```

RESULT

Following typed values are children of `player`.

ultimate_http_body_array()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_array(uint8_t handle, const char *key)`

Parameters

`handle` names a nonbinary body; `key` follows the same rules.

Returns

An `ULTIMATE_*` result.

Purpose

Adds an array and enters it. Keys on items inside the array are ignored.

Example

```
static const char key[] = {0x73,0x63,0x6f,0x72,0x65,0x73,0};
err = ultimate_http_body_array(body, key);
```

RESULT

Following typed values become array elements until a body-up call.

ultimate_http_body_binary()

C function

Header: `ultimate.h`

Prototype: `uint8_t ultimate_http_body_binary(uint8_t handle, const uint8_t *data, uint16_t len)`

Parameters

`handle` names an `HTTP_BODY_BINARY` slot; `data` supplies `len` bytes.

Returns

An `ULTIMATE_*` result.

Purpose

Appends raw bytes; successive calls extend the same body.

Example

```
err = ultimate_http_body_binary(body, chunk, sizeof chunk);
```

RESULT

The chunk is appended byte-for-byte.

> **Program tip:** Build a large JSON document a field at a time inside the Ultimate, exchange it, then free both body and request handles on every path. This avoids keeping the whole document in the C64's limited RAM.

ERRORS AND DIAGNOSTICS

ultimate_strerror()

C function

Header: `ultimate.h`

Prototype: `const char *ultimate_strerror(uint8_t err)`

Parameters

`err` is an `ULTIMATE_*` result.

Returns

A non-null pointer to stable PETSCII English text.

Purpose

Formats SDK-level diagnostics without exposing target-specific meanings.

Example

```
printf("load: %s\n", ultimate_strerror(ULTIMATE_ERR_IO));
```

OUTPUT

```
LOAD: I/O ERROR
```

ultimate_device_code()

C function-like macro

Header: `ultimate.h`

Syntax: `ultimate_device_code()`

Parameters

None.

Returns

The most recent raw target status, or `UCI_DEVICE_CODE_NONE`.

Purpose

Aliases `uci_last_device_code()` for diagnostics.

Example

```
uint16_t device = ultimate_device_code();
```

RESULT

`device` holds firmware-specific detail; application control should use the SDK result instead.

LOW-LEVEL UCI TRANSPORT

uci_signature_present()

C function

Header: `uci.h`

Prototype: `uint8_t uci_signature_present(void)`

Parameters

None.

Returns

1 when the documented identification signature is present, otherwise 0.

Purpose

Performs a quick register-only probe. It cannot distinguish a healthy from a wedged interface.

Example

```
if (!uci_signature_present()) return 1;
```

RESULT

The program exits when no UCI signature is mapped.

uci_ident_register()

C function

Header: `uci.h`

Prototype: `uint8_t uci_ident_register(void)`

Parameters

None.

Returns

The raw identification-register byte.

Purpose

Exposes register detail for diagnostics and new bindings.

Example

```
uint8_t ident = uci_ident_register();
```

RESULT

`ident` contains the unprocessed hardware value.

uci_init()

C function

Header: `uci.h`

Prototype: `uint8_t uci_init(void)`

Parameters

None.

Returns

`ULTIMATE_OK`, `ULTIMATE_ERR_NO_DEVICE`, or `ULTIMATE_ERR_TIMEOUT`.

Purpose

Checks the signature, clears latent state, aborts an abandoned exchange if needed, and installs the default timeout.

Example

```
uint8_t err = uci_init();
```

RESULT

The raw transport is idle and ready only after success.

uci_exec()

C function

Header: `uci.h`

Prototype: `uint8_t uci_exec(uci_request *req)`

Parameters

`req` points to a fully initialized `uci_request` containing target, command, input spans and reply buffers.

Returns

An `ULTIMATE_*` result. `ULTIMATE_ERR_TRUNCATED` means the command completed but a non-null data buffer filled; output lengths are always updated.

Purpose

Runs one command through every reply block. With `UCI_TARGET_NO_REPLY`, waits only for idle and collects nothing.

Example

```
err = uci_exec(&req);
```

RESULT

`req.datalen` and `req.statuslen` describe the retained reply.

uci_exec_first()

C function

Header: `uci.h`

Prototype: `uint8_t uci_exec_first(uci_request *req)`

Parameters

`req` is the same `uci_request` layout used by `uci_exec()`.

Returns

An `ULTIMATE_*` result for the first reply block; `req->datalen` is reset to this block's length.

Purpose

Starts a boundary-preserving exchange such as directory reading.

Example

```
err = uci_exec_first(&req);
```

RESULT

The first logical block is available without being joined to the next.

uci_exec_next()

C function

Header: `uci.h`

Prototype: `uint8_t uci_exec_next(uci_request *req)`

Parameters

`req` is the same `uci_request` passed to `uci_exec_first()`; no other command may intervene.

Returns

An `ULTIMATE_*` result for the next block; final status is decoded on the last.

Purpose

Advances one live boundary-preserving exchange.

Example

```
if (uci_more_blocks()) err = uci_exec_next(&req);
```

RESULT

The next block replaces the previous contents and length.

uci_more_blocks()

C function

Header: `uci.h`

Prototype: `uint8_t uci_more_blocks(void)`

Parameters

Call after `uci_exec_first()` or `uci_exec_next()`.

Returns

1 when another block follows, otherwise 0.

Purpose

Reads current transport state without issuing a command.

Example

```
while (uci_more_blocks()) err = uci_exec_next(&req);
```

RESULT

The loop consumes all remaining blocks.

uci_abort()

C function

Header: `uci.h`

Prototype: `uint8_t uci_abort(void)`

Parameters

None.

Returns

An `ULTIMATE_*` result.

Purpose

Abandons an exchange and forces the interface back to idle. Safe at any time.

Example

```
err = uci_abort();
```

RESULT

A half-read directory or stopped program no longer holds the interface.

uci_set_timeout()

C function

Header: `uci.h`

Prototype: `void uci_set_timeout(uint8_t units)`

Parameters

`units` counts groups of 256 status polls; zero means forever.

Returns

Nothing.

Purpose

Sets the polling budget. Its wall time changes with CPU speed.

Example

```
uci_set_timeout(UCI_TIMEOUT_DEFAULT);
uci_set_timeout(UCI_TIMEOUT_FOREVER);
```

RESULT

The calls demonstrate bounded default and unbounded operation.

uci_get_timeout()

C function

Header: `uci.h`

Prototype: `uint8_t uci_get_timeout(void)`

Parameters

None.

Returns

The current timeout units; zero means forever.

Purpose

Reads the transport's current polling budget.

Example

```
uint8_t saved = uci_get_timeout();
```

RESULT

`saved` can restore the caller's policy after a temporary change.

uci_last_device_code()

C function

Header: `uci.h`

Prototype: `uint16_t uci_last_device_code(void)`

Parameters

None.

Returns

The last decoded raw status: DOS-family 0–99, HTTP 0–999, SoftwareIEC 0–255, or `UCI_DEVICE_CODE_NONE`.

Purpose

Exposes target-specific diagnostic detail.

Example

```
uint16_t device = uci_last_device_code();
```

RESULT

Interpret `device` only in the context of the target just called.

uci_status_format()

C function

Header: `uci.h`

Prototype: `uint8_t uci_status_format(uint8_t target)`

Parameters

`target` is a `UCI_TARGET_*` value.

Returns

The expected `UCI_STATUS_FMT_*` encoding.

Purpose

Reports protocol documentation; it does not inspect bytes and must not be used as the decoder because some firmware replies violate the target convention.

Example

```
uint8_t dos = uci_status_format(UCI_TARGET_DOS1);
uint8_t http = uci_status_format(UCI_TARGET_HTTP);
uint8_t iec = uci_status_format(UCI_TARGET_SOFTIEC);
```

RESULT

The values are `UCI_STATUS_FMT_DECIMAL2`, `UCI_STATUS_FMT_DECIMAL3`, and `UCI_STATUS_FMT_BINARY`, respectively. Other target numbers use the two-digit default.

uci_decode_status()

C function

Header: `uci.h`

Prototype: `uint8_t uci_decode_status(uint8_t target, const uint8_t *status, uint16_t statuslen)`

Parameters

`target` supplies context; `status` points to `statuslen` raw bytes. Pass the complete status when possible; a nonzero length requires a non-null pointer.

Returns

The decoded `ULTIMATE_*` result and, as a side effect, a new raw device code. Empty status is success.

Purpose

Detects two- and three-digit numeric forms, HTTP response lines, SoftwareIEC binary status, and unnumbered target-error text from the bytes rather than trusting the expected target format.

Example

```
static const uint8_t dos[] = {0x30,0x30,0x2c,0x4f,0x4b};
static const uint8_t http[] = {0x34,0x30,0x34,0x20};
static const uint8_t line[] =
    {0x48,0x54,0x54,0x50,0x2f,0x31,0x2e,0x31,0x20,0x32,0x30,0x30};
static const uint8_t iec[] = {0x00};
static const uint8_t text[] = {0x46,0x55,0x4c,0x4c};
err = uci_decode_status(UCI_TARGET_DOS1, dos, sizeof dos);
err = uci_decode_status(UCI_TARGET_HTTP, http, sizeof http);
err = uci_decode_status(UCI_TARGET_HTTP, line, sizeof line);
err = uci_decode_status(UCI_TARGET_SOFTIEC, iec, sizeof iec);
err = uci_decode_status(UCI_TARGET_DOS1, text, sizeof text);
err = uci_decode_status(UCI_TARGET_DOS1, NULL, 0);
```

RESULT

The calls demonstrate two-digit status, three-digit firmware HTTP status, an HTTP response line, binary status, unnumbered target-error text, and empty success. `uci_last_device_code()` retains a decoded number or `UCI_DEVICE_CODE_NONE` when none exists.

CHAPTER 8

ASSEMBLY API REFERENCE

- Every callable symbol exported by `ultimate.inc`
- Register and shared-cell inputs, outputs and clobbers
- Short ca65 call examples

COMMON RULES

Include `ultimate.inc` and link `ultimate.lib`. These are native 6502 entries: arguments use registers or the exported `ult_*` cells, never the C stack. Calls are not re-entrant. They install no interrupt handler and are safe with interrupts disabled. The SDK owns four zero-page bytes at $FB–$FE unless `UCI_ZP` is changed before assembly.

Except where an entry returns a value, `A` is an `ULTIMATE_*` result; `beq` therefore branches on success. A 16-bit value returned in `A/X` has no flag guarantee. Initialize UCI-backed services with `ultimate_init`; turbo, REU, audio and vsprites state their own probes. Pointers are little-endian, with the low byte in `A` and high byte in `X`. Strings sent to the Ultimate must be ASCII, not ca65-mapped PETSCII. Examples reuse the ordinary `setptr`, `setword` and `setdword` macros defined in Chapter 4.

INITIALIZATION AND DISCOVERY

`ultimate_init`

Assembly routine

Call: `jsr ultimate_init`

Inputs

None.

Outputs

`A` is `ULTIMATE_OK`, `ULTIMATE_ERR_NO_DEVICE`, or `ULTIMATE_ERR_TIMEOUT`; N and Z reflect `A`.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Initializes and functionally checks the command interface.

Example

```
jsr ultimate_init
bne no_ultimate
```

RESULT

The branch is taken unless the SDK is ready.

ultimate_available

Assembly routine

Call: `jsr ultimate_available`

Inputs

None.

Outputs

`A` is 1 after successful initialization, otherwise 0; N and Z reflect `A`.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads cached availability without touching hardware.

Example

```
jsr ultimate_available
beq not_ready
```

RESULT

The branch is taken until `ultimate_init` succeeds.

ultimate_detect

Assembly routine

Call: `jsr ultimate_detect`

Inputs

`A/X` points to four writable bytes: present, ident and a 16-bit target mask.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change. `ult_buf`, `ult_buflen` and `ult_outlen`.

Purpose

Probes every known target and fills a reusable capability block.

Example

```
lda #<caps
ldx #>caps
jsr ultimate_detect
```

RESULT

On success, bit N of `caps+2` identifies target N.

ultimate_identify

Assembly routine

Call: `jsr ultimate_identify`

Inputs

`A` is a `UCI_TARGET_*`; `ult_buf` points to output, `ult_buflen` is its capacity and `ult_outlen` points to an optional 16-bit stored length.

Outputs

`A` is a result code; the text is NUL-terminated when capacity permits.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads one target's identification string.

Example

```
lda #UCI_TARGET_DOS1
jsr ultimate_identify
```

RESULT

The caller's buffer contains the DOS identity on success.

ultimate_get_model

Assembly routine

Call: `jsr ultimate_get_model`

Inputs

`ult_buf`, `ult_buflen` and optional `ult_outlen` describe the output.

Outputs

`A` is a result code; output is terminated when capacity permits.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads the model using deprecated control HWINFO compatibility.

Example

```
jsr ultimate_get_model
```

RESULT

Handle `ULTIMATE_ERR_NOT_SUPPORTED`; HWINFO may disappear.

ultimate_legacy_get_sid_info

Assembly routine

Call: `jsr ultimate_legacy_get_sid_info`

Inputs

`A/X` points to `ULTIMATE_SID_INFO_SIZE` writable bytes.

Outputs

`A` is a result code; byte zero is the record count on success.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads up to four raw SID mapping records through deprecated HWINFO.

Example

```
lda #<sidinfo
ldx #>sidinfo
jsr ultimate_legacy_get_sid_info
```

RESULT

Records begin at `ULTIMATE_SID_INFO_RECORDS`; each five-byte record is a little-endian primary address, little-endian secondary address, and raw type byte at the named `ULTIMATE_SID_*` offsets.

ultimate_strerror

Assembly routine

Call: `jsr ultimate_strerror`

Inputs

`A` is an `ULTIMATE_*` result code.

Outputs

`A/X` points to a NUL-terminated PETSCII explanation.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Maps an SDK result to printable text.

Example

```
lda #ULTIMATE_ERR_IO
jsr ultimate_strerror
```

RESULT

`A/X` points to `I/O ERROR`.

PALETTE, TURBO AND VSPRITE GRAPHICS

ultimate_palette_get

Assembly routine

Call: `jsr ultimate_palette_get`

Inputs

`A/X` points to 48 writable bytes.

Outputs

`A` is a result code; success fills sixteen RGB triples.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads the live VIC-II palette.

Example

```
lda #<palette
ldx #>palette
jsr ultimate_palette_get
```

RESULT

`palette` contains exactly 48 bytes on success.

ultimate_palette_set

Assembly routine

Call: `jsr ultimate_palette_set`

Inputs

`A/X` points to 48 RGB bytes.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Replaces all sixteen live colours without writing flash.

Example

```
lda #<palette
ldx #>palette
jsr ultimate_palette_set
```

RESULT

The live palette changes on success.

ultimate_palette_set_color

Assembly routine

Call: `jsr ultimate_palette_set_color`

Inputs

`ult_color+0..3` contain index, red, green and blue.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Changes one live palette colour; the index must be 0–15.

Example

```
lda #6:sta ult_color
lda #255:sta ult_color+1
lda #0:sta ult_color+2:sta ult_color+3
jsr ultimate_palette_set_color
```

RESULT

Colour 6 becomes red.

ultimate_palette_reset

Assembly routine

Call: `jsr ultimate_palette_reset`

Inputs

None.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Restores the built-in live palette.

Example

```
jsr ultimate_palette_reset
```

RESULT

Temporary palette changes are removed.

ultimate_turbo_available

Assembly routine

Call: `jsr ultimate_turbo_available`

Inputs

None; UCI initialization is not required.

Outputs

`A` is 1 when the Ultimate 64 turbo registers answer, otherwise 0.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Probes the memory-mapped turbo registers.

Example

```
jsr ultimate_turbo_available
beq one_mhz_fallback
```

RESULT

The fallback runs on unsupported or owner-disabled hardware.

ultimate_turbo_get

Assembly routine

Call: `jsr ultimate_turbo_get`

Inputs

None.

Outputs

`A` is the current speed index or `U64_TURBO_UNAVAILABLE` ($FF).

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads CPU speed without changing badlines.

Example

```
jsr ultimate_turbo_get
sta old_speed
```

RESULT

`old_speed` can restore a supported entry speed.

ultimate_turbo_set

Assembly routine

Call: `jsr ultimate_turbo_set`

Inputs

`A` is a speed index 0–15; 1–4 MHz constants are portable and `U64_SPEED_MAX` means this machine's maximum.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Sets CPU speed and leaves badlines unchanged.

Example

```
lda #U64_SPEED_1MHZ:jsr ultimate_turbo_set
lda #U64_SPEED_2MHZ:jsr ultimate_turbo_set
lda #U64_SPEED_3MHZ:jsr ultimate_turbo_set
lda #U64_SPEED_4MHZ:jsr ultimate_turbo_set
lda #U64_SPEED_MAX:jsr ultimate_turbo_set
```

RESULT

The calls demonstrate every named portable speed and model maximum.

ultimate_turbo_badlines

Assembly routine

Call: `jsr ultimate_turbo_badlines`

Inputs

`A`=0 disables badlines; any nonzero value enables them.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Changes VIC cycle stealing without changing CPU speed.

Example

```
lda #0:jsr ultimate_turbo_badlines
lda #1:jsr ultimate_turbo_badlines
```

RESULT

The two calls demonstrate both settings.

ultimate_vsprite_draw

Assembly routine

Call: `jsr ultimate_vsprite_draw`

Inputs

`A/X` points to a `VSPRITE_SIZE` descriptor. Its named offsets hold bitmap, source, optional mask and screen pointers; byte-column `X` (0–39), raster-line `Y` (0–199), nonzero width and height that fit the bitmap, colour, and exactly one operation: 0, `VSPRITE_F_MASKED`, `VSPRITE_F_COPY`, or `VSPRITE_F_COLOR`. Image and mask bytes are column-major.

Outputs

`A` is a result code; the destination bitmap changes in place.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change. The routine self-modifies and therefore must execute from RAM.

Purpose

Draws OR, masked, bitmap-copy or colour-aware software vsprites without UCI.

Example

```
lda #0:sta sprite+VSPRITE_FLAGS
lda #<sprite:ldx #>sprite:jsr ultimate_vsprite_draw
lda #VSPRITE_F_MASKED:sta sprite+VSPRITE_FLAGS
lda #<sprite:ldx #>sprite:jsr ultimate_vsprite_draw
lda #VSPRITE_F_COPY:sta sprite+VSPRITE_FLAGS
lda #<sprite:ldx #>sprite:jsr ultimate_vsprite_draw
lda #VSPRITE_F_COLOR:sta sprite+VSPRITE_FLAGS
lda #<sprite:ldx #>sprite:jsr ultimate_vsprite_draw
```

RESULT

The four calls select OR, masked, bitmap-copy and colour-aware draws.

DIRECTORIES AND FILES

ultimate_chdir

Assembly routine

Call: `jsr ultimate_chdir`

Inputs

`ult_buf` points to a NUL-terminated ASCII path.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Changes the current Ultimate DOS directory.

Example

```
setptr ult_buf,path
jsr ultimate_chdir
```

RESULT

Later filesystem calls use `path`.

ultimate_getpath

Assembly routine

Call: `jsr ultimate_getpath`

Inputs

`ult_buf`, `ult_buflen` and optional `ult_outlen` describe the output.

Outputs

`A` is a result code; the current path is terminated when possible.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads the current DOS path.

Example

```
setptr ult_buf,buffer
jsr ultimate_getpath
```

RESULT

The caller's buffer contains the current path.

ultimate_opendir

Assembly routine

Call: `jsr ultimate_opendir`

Inputs

None.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Begins a block-preserving directory exchange.

Example

```
jsr ultimate_opendir
```

RESULT

Success prepares the first `ultimate_readdir`.

ultimate_readdir

Assembly routine

Call: `jsr ultimate_readdir`

Inputs

`ult_buf` points to a name buffer and `ult_buflen` is its size.

Outputs

`A` is `ULTIMATE_OK`, `ULTIMATE_END`, or an error; `ult_attrib` receives `DOS_ATTR_*`.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Consumes one directory entry. Call `uci_abort` if stopping early.

Example

```
jsr ultimate_readdir
cmp #ULTIMATE_END
```

RESULT

One terminated name is returned until the final call reports end.

ultimate_open

Assembly routine

Call: `jsr ultimate_open`

Inputs

`A` is a `DOS_FA_*` mask; `ult_buf` points to a terminated filename.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Opens the firmware-side current file.

Example

```
lda #DOS_FA_READ:jsr ultimate_open
lda #DOS_FA_WRITE:jsr ultimate_open
lda #(DOS_FA_READ | DOS_FA_WRITE):jsr ultimate_open
lda #(DOS_FA_WRITE | DOS_FA_CREATE_NEW):jsr ultimate_open
lda #(DOS_FA_WRITE | DOS_FA_CREATE_ALWAYS):jsr ultimate_open
```

RESULT

With `ult_buf` changed to a suitable name before each call, these are read, write, read/write, create-new and create-or-replace modes.

ultimate_close

Assembly routine

Call: `jsr ultimate_close`

Inputs

None.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Closes the current firmware-side file.

Example

```
jsr ultimate_close
```

RESULT

The open file handle is released.

ultimate_read

Assembly routine

Call: `jsr ultimate_read`

Inputs

`ult_buf` points to output, `ult_buflen` is its capacity and `ult_outlen` optionally points to the stored byte count.

Outputs

`A` is a result code; a short successful read means end-of-file.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads from the current file and drains all reply blocks internally.

Example

```
jsr ultimate_read
```

RESULT

The buffer and optional count describe the bytes read.

ultimate_write

Assembly routine

Call: `jsr ultimate_write`

Inputs

`ult_buf` points to bytes and `ult_buflen` is their count.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Writes one complete span to the current file.

Example

```
jsr ultimate_write
```

> **RESULT**
>
> The file position advances by `ult_buflen` on success.

ultimate_seek

Assembly routine

Call: `jsr ultimate_seek`

Inputs

`ult_num` contains a 32-bit absolute file position.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Moves the current file position.

Example

```
setdword ult_num,1024
jsr ultimate_seek
```

> **RESULT**
>
> The next transfer begins at byte 1024.

ultimate_delete

Assembly routine

Call: `jsr ultimate_delete`

Inputs

`ult_buf` points to a terminated name or path.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Deletes one file.

Example

```
setptr ult_buf,name
jsr ultimate_delete
```

RESULT

The named file is removed on success.

ultimate_stat

Assembly routine

Call: `jsr ultimate_stat`

Inputs

`ult_buf` points to a name; `ult_arg2` points to `ULTIMATE_FILEINFO_SIZE` writable bytes.

Outputs

`A` is a result code; success fills the metadata block.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads metadata for a named entry.

Example

```
setptr ult_buf,name
setptr ult_arg2,fileinfo
jsr ultimate_stat
```

RESULT

Size, FAT time, extension, attributes and name are returned.

ultimate_fstat

Assembly routine

Call: `jsr ultimate_fstat`

Inputs

`ult_arg2` points to a writable file-info block; a file is open.

Outputs

`A` is a result code; success fills the metadata block.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads metadata for the current file.

Example

```
setptr ult_arg2,fileinfo
jsr ultimate_fstat
```

RESULT

The open file's metadata is returned.

ultimate_rename

Assembly routine

Call: `jsr ultimate_rename`

Inputs

`ult_buf` points to the old name; `ult_arg2` points to the new.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Renames a current-directory entry.

Example

```
setptr ult_buf,oldname
setptr ult_arg2,newname
jsr ultimate_rename
```

RESULT

The entry has the new name on success.

ultimate_copy

Assembly routine

Call: `jsr ultimate_copy`

Inputs

`ult_buf` points to a source name; `ult_arg2` points to a destination directory.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Copies inside the Ultimate while preserving the source filename.

Example

```
setptr ult_buf,name
setptr ult_arg2,destination
jsr ultimate_copy
```

RESULT

No file contents pass through C64 RAM.

ultimate_mkdir

Assembly routine

Call: `jsr ultimate_mkdir`

Inputs

`ult_buf` points to a terminated directory name.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Creates one directory.

Example

```
setptr ult_buf,dirname
jsr ultimate_mkdir
```

> **RESULT**
>
> The directory exists on success.

ultimate_home

Assembly routine

Call: `jsr ultimate_home`

Inputs
None.

Outputs
`A` is a result code.

Clobbers
Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose
Changes to the configured Ultimate home directory.

Example
```
jsr ultimate_home
```

> **RESULT**
>
> Later filesystem calls use that directory.

DISK IMAGES AND CLOCK

ultimate_mount

Assembly routine

Call: `jsr ultimate_mount`

Inputs
`A` is an IEC device number or `ULTIMATE_DRIVE_LAST`; `ult_buf` points to an image path.

Outputs
`A` is a result code.

Clobbers
Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Mounts a D64, D71, D81, G64 or G71 image.

Example

```
lda #8
jsr ultimate_mount
```

RESULT

The image named by `ult_buf` is mounted in device 8.

ultimate_unmount

Assembly routine

Call: `jsr ultimate_unmount`

Inputs

`A` is an IEC device number or `ULTIMATE_DRIVE_LAST`.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Removes the current image from a drive.

Example

```
lda #8
jsr ultimate_unmount
```

RESULT

Device 8 becomes empty.

ultimate_swap

Assembly routine

Call: `jsr ultimate_swap`

Inputs

`A` is an IEC device number or `ULTIMATE_DRIVE_LAST`.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Advances a multi-image set.

Example

```
lda #8
jsr ultimate_swap
```

RESULT

Device 8 selects its next image.

ultimate_get_time

Assembly routine

Call: `jsr ultimate_get_time`

Inputs

`A` is `ULTIMATE_TIME_PLAIN` or `ULTIMATE_TIME_WEEKDAY`; output uses `ult_buf`, `ult_buflen` and optional `ult_outlen`.

Outputs

`A` is a result code; success returns terminated time text.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads the battery-backed clock.

Example

```
lda #ULTIMATE_TIME_PLAIN:jsr ultimate_get_time
lda #ULTIMATE_TIME_WEEKDAY:jsr ultimate_get_time
```

RESULT

Plain output resembles `2026/08/22 14:30:00`; weekday output prefixes a day name.

ultimate_set_time

Assembly routine

Call: `jsr ultimate_set_time`

Inputs

`ult_stage+0..5` contain year minus 1900, month, day, hour, minute and second.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Sets the battery-backed clock.

Example

```
lda #126:sta ult_stage
lda #8:sta ult_stage+1
jsr ultimate_set_time
```

RESULT

The remaining four staging bytes must also be set before this call.

LOADING AND SAVING MEMORY

ultimate_load

Assembly routine

Call: `jsr ultimate_load`

Inputs

`ult_buf` points to a PRG name; `ult_addr=0` uses its header and a nonzero address overrides it.

Outputs

`A` is a result code; `ult_end` is the address after the load.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change. The DOS fallback changes `ult_buf`, `ult_buflen`, `ult_outlen`, `ult_addr` and `ult_max`.

Purpose

Consumes the PRG header and tries SoftwareIEC before DOS fallback.

Example

```
setptr ult_buf,name
setword ult_addr,0
jsr ultimate_load
```

RESULT

The PRG loads at its stored address on success.

ultimate_bload

Assembly routine

Call: `jsr ultimate_bload`

Inputs

`ult_buf` points to a name; nonzero `ult_addr` is destination and nonzero `ult_max` is the maximum raw length.

Outputs

`A` is a result code; `ult_end` is the address after the load.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Loads uninterpreted bytes without consuming a header.

Example

```
setword ult_addr,$c000
setword ult_max,2048
jsr ultimate_bload
```

RESULT

At most 2048 bytes are written from $C000.

ultimate_save

Assembly routine

Call: `jsr ultimate_save`

Inputs

`ult_buf` points to a name, `ult_addr` is the C64 start and nonzero `ult_max` is the length.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Creates or replaces a file from C64 memory.

Example

```
setword ult_addr,$0400
setword ult_max,1000
jsr ultimate_save
```

RESULT

The text screen is saved on success.

ultimate_last_end

Assembly routine

Call: `jsr ultimate_last_end`

Inputs

None.

Outputs

`A/X` is the address after the last successful load.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads cached load state without touching hardware.

Example

```
jsr ultimate_last_end
sta end
stx end+1
```

RESULT

`end-1` is the last address written.

RAM EXPANSION

ultimate_reu_available

Assembly routine

Call: `jsr ultimate_reu_available`

Inputs

None; UCI initialization is not required.

Outputs

`A` is 1 when REU registers answer, otherwise 0.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Probes for enabled RAM expansion.

Example

```
jsr ultimate_reu_available
beq no_reu
```

RESULT

The branch is taken when no REU is mapped.

ultimate_reu_size

Assembly routine

Call: `jsr ultimate_reu_size`

Inputs

None.

Outputs

`A/X` is the expansion size in 64K banks, or zero when absent.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Measures the REU non-destructively.

Example

```
jsr ultimate_reu_size
sta banks
stx banks+1
```

RESULT

16 banks means 1 MB; 256 banks means 16 MB.

ultimate_reu_stash

Assembly routine

Call: `jsr ultimate_reu_stash`

Inputs

`ult_addr` is a C64 address; the low 24 bits of `ult_reu` are the REU address and its high byte is ignored; the low word of `ult_reulen` is the 16-bit length and zero means 65536 bytes; its high word is ignored.

Outputs

`A` is a result code after DMA completes.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Copies C64 memory into the REU.

Example

```
setword ult_addr,$0400
setdword ult_reu,0
setdword ult_reulen,1000
jsr ultimate_reu_stash
```

RESULT

The text screen is copied to REU address zero.

ultimate_reu_fetch

Assembly routine

Call: `jsr ultimate_reu_fetch`

Inputs

`ult_addr`, `ult_reu` and `ult_reulen` describe the same ranges as `ultimate_reu_stash`.

Outputs

`A` is a result code after DMA completes.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Copies REU data into C64 memory.

Example

```
setword ult_addr,$0400
jsr ultimate_reu_fetch
```

RESULT

The selected REU bytes replace C64 memory.

ultimate_reu_load

Assembly routine

Call: `jsr ultimate_reu_load`

Inputs

A readable file is open; `ult_reu` is destination and `ult_reulen` is a 32-bit length.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Transfers the open file directly into expansion memory.

Example

```
setdword ult_reu,0
setdword ult_reulen,65536
jsr ultimate_reu_load
```

RESULT

The bytes do not pass through C64 RAM.

ultimate_reu_save

Assembly routine

Call: `jsr ultimate_reu_save`

Inputs

A writable file is open; `ult_reu` is source and `ult_reulen` is a 32-bit length.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Transfers expansion memory directly into the open file.

Example

```
setdword ult_reu,0
setdword ult_reulen,65536
jsr ultimate_reu_save
```

RESULT

The bytes do not pass through C64 RAM.

Program tip: Use REU memory for large level data, decompression sources, sampled audio and screen backups. Stash/fetch swap live C64 memory; load/save avoid routing bulk file data through the C64 address space.

ULTIMATE AUDIO

ultimate_audio_init

Assembly routine

Call: `jsr ultimate_audio_init`

Inputs

Ultimate Audio is mapped at $DF20–$DFFF.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Probes the PCM engine with a silent byte.

Example

```
jsr ultimate_audio_init
bne no_audio
```

RESULT

Other audio routines are usable only after success.

ultimate_audio_available

Assembly routine

Call: `jsr ultimate_audio_available`

Inputs

Call `ultimate_audio_init` first.

Outputs

`A` is 1 after successful audio initialization, otherwise 0.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads cached audio state.

Example

```
jsr ultimate_audio_available
```

RESULT

Zero means the PCM engine is unavailable.

ultimate_audio_version

Assembly routine

Call: `jsr ultimate_audio_version`

Inputs

Audio is available.

Outputs

`A` is the raw module-version byte.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads the mapped engine's version register.

Example

```
jsr ultimate_audio_version
sta version
```

RESULT

`version` is valid only after audio initialization.

ultimate_audio_configure

Assembly routine

Call: `jsr ultimate_audio_configure`

Inputs

`A/X` points to a `UA_VOICE_SIZE` descriptor. Named offsets hold channel 0–6; only `UA_CTRL_FLAGS`; volume 0–63; pan 0–15; a 24-bit REU start and nonzero length that do not pass 16 MB; repeat offsets; and a nonzero rate. A 16-bit or interleaved length is even. Repeat requires `repeat_a < repeat_b <= length`.

Outputs

`A` is a result code; the channel is configured but stopped.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Validates and programs one PCM voice.

Example

```
lda #0:sta voice+UA_VOICE_FLAGS
lda #<voice:ldx #>voice:jsr ultimate_audio_configure
lda #UA_CTRL_16BIT:sta voice+UA_VOICE_FLAGS
lda #<voice:ldx #>voice:jsr ultimate_audio_configure
lda #UA_CTRL_INTERLEAVE:sta voice+UA_VOICE_FLAGS
lda #<voice:ldx #>voice:jsr ultimate_audio_configure
lda #UA_CTRL_REPEAT:sta voice+UA_VOICE_FLAGS
lda #<voice:ldx #>voice:jsr ultimate_audio_configure
lda #UA_CTRL_IRQ:sta voice+UA_VOICE_FLAGS
lda #<voice:ldx #>voice:jsr ultimate_audio_configure
```

RESULT

The examples isolate 8-bit mono, 16-bit, interleave, repeat and IRQ; valid flags may be combined.

ultimate_audio_start

Assembly routine

Call: `jsr ultimate_audio_start`

Inputs

`A` is channel 0–6; `X` contains only the descriptor's `UA_CTRL_FLAGS` bits. Do not include `UA_CTRL_GATE`; the call adds it.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Opens a configured channel's gate.

Example

```
lda #0
ldx #UA_CTRL_REPEAT
jsr ultimate_audio_start
```

RESULT

Channel zero begins playing.

ultimate_audio_stop

Assembly routine

Call: `jsr ultimate_audio_stop`

Inputs

`A` is channel 0–6.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Closes one channel's gate.

Example

```
lda #0
jsr ultimate_audio_stop
```

RESULT

Channel zero stops.

ultimate_audio_irq_status

Assembly routine

Call: `jsr ultimate_audio_irq_status`

Inputs

None.

Outputs

`A` is the latched end-of-sample channel mask.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads PCM completion state.

Example

```
jsr ultimate_audio_irq_status
and #1
```

RESULT

A nonzero result means channel zero completed.

ultimate_audio_irq_clear

Assembly routine

Call: `jsr ultimate_audio_irq_clear`

Inputs

`A` is channel 0–6.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Clears one completion latch and its asserted IRQ.

Example

```
lda #0
jsr ultimate_audio_irq_clear
```

RESULT

Channel zero's completion state is clear.

ultimate_audio_load_wav

Assembly routine

Call: `jsr ultimate_audio_load_wav`

Inputs

`ult_buf` points to a PCM WAV filename; `ult_reu` is its REU destination; `A/X` points to a `UA_VOICE_SIZE` descriptor whose channel, volume, pan and repeat offsets are initialized by the caller.

Outputs

`A` is a result code; success fills address, length, rate and flags.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change. `ult_buf`, `ult_buflen`, `ult_outlen`, `ult_num`, `ult_addr`, `ult_reulen`, `ult_arg2` and `ult_attrib`.

Purpose

Loads supported 8/16-bit mono/stereo PCM and prepares a voice.

Example

```
lda #<voice
ldx #>voice
jsr ultimate_audio_load_wav
```

RESULT

Configure and start the filled descriptor after success.

MACHINE AND DRIVE CONTROL

ultimate_reboot

Assembly routine

Call: `jsr ultimate_reboot`

Inputs

None.

Outputs

No return.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change. All machine state is reset.

Purpose

Reboots the C64 through the control target.

Example

```
jsr ultimate_reboot
```

RESULT

No following instruction executes.

ultimate_freeze

Assembly routine

Call: `jsr ultimate_freeze`

Inputs

None.

Outputs

`A` is a result code after the user leaves the Ultimate menu.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Enters the freezer menu and waits for the machine to resume.

Example

```
jsr ultimate_freeze
```

RESULT

Execution resumes after the menu releases the C64.

ultimate_drive_enable

Assembly routine

Call: `jsr ultimate_drive_enable`

Inputs

`A` is `ULTIMATE_DRIVE_A` or `ULTIMATE_DRIVE_B`; `X`=0 disables and nonzero enables.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Changes one emulated drive's enabled state.

Example

```
lda #ULTIMATE_DRIVE_B
ldx #0
jsr ultimate_drive_enable
```

RESULT

Drive B is disabled on success.

ultimate_drive_power

Assembly routine

Call: `jsr ultimate_drive_power`

Inputs

`A` is `ULTIMATE_DRIVE_A` or `ULTIMATE_DRIVE_B`.

Outputs

`A` is a result code; `ult_stage`=1 when powered, 0 when off.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads one drive's power state.

Example

```
lda #ULTIMATE_DRIVE_A
jsr ultimate_drive_power
lda ult_stage
```

RESULT

The final `A` is the Boolean power state after a successful call.

ultimate_drive_info

Assembly routine

Call: `jsr ultimate_drive_info`

Inputs

`A/X` points to `CTRL_DRVINFO_BYTES` writable bytes.

Outputs

`A` is a result code; byte `CTRL_DRVINFO_COUNT` is the number of records. Three-byte records begin at `CTRL_DRVINFO_FIRST` and contain type, IEC device number and power.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads emulated drives and occupied IEC bus slots.

Example

```
lda #<drives
ldx #>drives
jsr ultimate_drive_info
```

RESULT

The returned count is bounded by the records that actually arrived.

ultimate_ramdisk_info

Assembly routine

Call: `jsr ultimate_ramdisk_info`

Inputs

`A/X` points to `CTRL_RAMDISK_BYTES` writable bytes.

Outputs

`A` is a result code; success fills `CTRL_RAMDISK_DRIVES` two-byte records containing type and size in 64K units.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads control-target RAM-disk state.

Example

```
lda #<ramdisk
ldx #>ramdisk
jsr ultimate_ramdisk_info
```

RESULT

The caller's block contains the decoded reply on success.

NETWORKING

ultimate_net_ifcount

Assembly routine

Call: `jsr ultimate_net_ifcount`

Inputs

None.

Outputs

`A` is a result code; `ult_iface` is the interface count.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Counts configured network interfaces.

Example

```
jsr ultimate_net_ifcount
lda ult_iface
```

RESULT

The final `A` is the count after a successful call.

ultimate_net_macaddr

Assembly routine

Call: `jsr ultimate_net_macaddr`

Inputs

`ult_iface` is an interface index; `ult_buf` points to six bytes.

Outputs

`A` is a result code; success writes the MAC address.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads one interface's hardware address.

Example

```
lda #0:sta ult_iface
setptr ult_buf,mac
jsr ultimate_net_macaddr
```

RESULT

Six binary address bytes are returned.

ultimate_net_ipconfig

Assembly routine

Call: `jsr ultimate_net_ipconfig`

Inputs

`ult_iface` is an interface index; `ult_buf` points to 12 bytes.

Outputs

`A` is a result code; success writes IP, netmask and gateway.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads one interface's IPv4 configuration.

Example

```
lda #0:sta ult_iface
setptr ult_buf,ipconfig
jsr ultimate_net_ipconfig
```

RESULT

Three four-byte network-order addresses are returned.

ultimate_net_setip

Assembly routine

Call: `jsr ultimate_net_setip`

Inputs

`ult_iface` is an interface index; `ult_buf` points to 12 bytes of IP, netmask and gateway.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Writes a static IPv4 configuration.

Example

```
lda #0:sta ult_iface
setptr ult_buf,ipconfig
jsr ultimate_net_setip
```

RESULT

The interface uses the supplied addresses on success.

ultimate_net_connect

Assembly routine

Call: `jsr ultimate_net_connect`

Inputs

`ult_port` is a little-endian TCP port; `ult_buf` points to an ASCII host name or address.

Outputs

`A` is a result code; `ult_sock` is the socket handle.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Opens TCP. Name lookup and connect wait without a timeout, then restore the caller's budget.

Example

```
setword ult_port,80
setptr ult_buf,host
jsr ultimate_net_connect
```

RESULT

Use `ult_sock` only after success.

ultimate_net_udp

Assembly routine

Call: `jsr ultimate_net_udp`

Inputs

`ult_port` and `ult_buf` supply a port and ASCII peer.

Outputs

`A` is a result code; `ult_sock` is the socket handle.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Opens a connected UDP socket with the same wait policy as TCP.

Example

```
setword ult_port,123
setptr ult_buf,host
jsr ultimate_net_udp
```

RESULT

Use `ult_sock` only after success.

ultimate_net_close

Assembly routine

Call: `jsr ultimate_net_close`

Inputs

`ult_sock` is an open socket handle.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Releases one firmware-side socket.

Example

```
jsr ultimate_net_close
```

> **RESULT**
>
> The handle is no longer usable on success.

ultimate_net_read

Assembly routine

Call: `jsr ultimate_net_read`

Inputs

`ult_sock` is a socket; `ult_buf` is output; `ult_socklen` is its capacity.

Outputs

`A` is a result code; `ult_socklen` becomes bytes stored.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Polls once without waiting. `ULTIMATE_END` reports peer close once.

Example

```
setword ult_socklen,256
jsr ultimate_net_read
```

> **RESULT**
>
> Zero bytes with success means try again later.

ultimate_net_write

Assembly routine

Call: `jsr ultimate_net_write`

Inputs

`ult_sock` is a socket; `ult_buf` points to bytes and `ult_buflen` is their count.

Outputs

`A` is a result code; `ult_socklen` becomes bytes accepted.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Writes one caller span to a socket.

Example

```
jsr ultimate_net_write
```

RESULT

Advance by `ult_socklen` when the socket accepts a partial span.

HTTP CLIENT

ultimate_http_get

Assembly routine

Call: `jsr ultimate_http_get`

Inputs

`ult_url` points to an ASCII URL; `ult_buf` and `ult_buflen` describe response storage.

Outputs

`A` is a result code; `ult_httplen` is bytes retained and `uci_last_code` supplies HTTP status.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Performs a complete GET using one firmware HTTP slot.

Example

```
setptr ult_url,url
setptr ult_buf,buffer
jsr ultimate_http_get
```

RESULT

The response body is retained up to the supplied capacity.

ultimate_http_open

Assembly routine

Call: `jsr ultimate_http_open`

Inputs

`A` is any `HTTP_VERB_*`; `ult_url` points to an ASCII URL.

Outputs

`A` is a result code; `ult_http` is a handle or $FF on failure.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Allocates a request slot. All nine published HTTP verb constants are valid.

Example

```
lda #HTTP_VERB_GET:jsr ultimate_http_open
lda #HTTP_VERB_PUT:jsr ultimate_http_open
lda #HTTP_VERB_POST:jsr ultimate_http_open
lda #HTTP_VERB_PATCH:jsr ultimate_http_open
lda #HTTP_VERB_DELETE:jsr ultimate_http_open
lda #HTTP_VERB_HEAD:jsr ultimate_http_open
lda #HTTP_VERB_OPTIONS:jsr ultimate_http_open
lda #HTTP_VERB_CONNECT:jsr ultimate_http_open
lda #HTTP_VERB_TRACE:jsr ultimate_http_open
```

RESULT

Each call creates a separate request of the named method; close each successful `ult_http` handle.

ultimate_http_header

Assembly routine

Call: `jsr ultimate_http_header`

Inputs

`ult_http` is a request handle; `ult_url` points to one ASCII `key: value` line.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Adds one request header before exchange.

Example

```
setptr ult_url,header
jsr ultimate_http_header
```

RESULT

The header is attached to the open request.

ultimate_http_exchange

Assembly routine

Call: `jsr ultimate_http_exchange`

Inputs

`ult_http` is a request; `ult_body` is a body handle or `HTTP_BODY_NONE`; `ult_buf`/`ult_buflen` receive response.

Outputs

`A` is a result code; `ult_httplen` is bytes retained and `uci_last_code` supplies HTTP status.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Sends an open request. DNS/connect wait without limit and restore the caller's timeout afterwards.

Example

```
jsr ultimate_http_exchange
```

RESULT

The body buffer and HTTP status describe the response.

ultimate_http_close

Assembly routine

Call: `jsr ultimate_http_close`

Inputs

`ult_http` is a request handle.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Releases one firmware HTTP slot.

Example

```
jsr ultimate_http_close
```

RESULT

The request handle is no longer usable.

ultimate_http_free_all

Assembly routine

Call: `jsr ultimate_http_free_all`

Inputs

None.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Releases every HTTP request and body slot owned by the firmware.

Example

```
jsr ultimate_http_free_all
```

RESULT

Useful cleanup after abandoning a partially built request.

HTTP REQUEST BODIES

Body values are built inside the Ultimate. For keyed JSON/object and URL-form values, `ult_url` points to an ASCII key of at most `ULTIMATE_HTTP_KEY_MAX` bytes. For an array element, point it to a one-byte NUL string; a null pointer is invalid.

ultimate_http_body

Assembly routine

Call: `jsr ultimate_http_body`

Inputs

`A` is `HTTP_BODY_JSON_OBJECT`, `HTTP_BODY_JSON_ARRAY`, `HTTP_BODY_URL_ENCODED`, or `HTTP_BODY_BINARY`.

Outputs

`A` is a result code; `ult_body` is a handle or $FF on failure.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Allocates an empty firmware-side request body.

Example

```
lda #HTTP_BODY_BINARY:jsr ultimate_http_body
lda #HTTP_BODY_JSON_OBJECT:jsr ultimate_http_body
lda #HTTP_BODY_JSON_ARRAY:jsr ultimate_http_body
lda #HTTP_BODY_URL_ENCODED:jsr ultimate_http_body
```

RESULT

The calls create binary, JSON-object, JSON-array and URL-encoded bodies; free each successful `ult_body` handle.

ultimate_http_body_free

Assembly routine

Call: `jsr ultimate_http_body_free`

Inputs

`ult_body` is a body handle.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Releases one firmware-side body slot.

Example

```
jsr ultimate_http_body_free
```

RESULT

The handle is no longer usable.

ultimate_http_body_clear

Assembly routine

Call: `jsr ultimate_http_body_clear`

Inputs

`ult_body` is a body handle.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Removes all values but preserves the body format and slot.

Example

```
jsr ultimate_http_body_clear
```

> **RESULT**
>
> The body is empty and reusable.

ultimate_http_body_up

Assembly routine

Call: `jsr ultimate_http_body_up`

Inputs

`ult_body` is positioned inside a nested object or array.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Moves the body cursor to its parent container.

Example

```
jsr ultimate_http_body_up
```

> **RESULT**
>
> The next value is added beside the container just left.

ultimate_http_body_int

Assembly routine

Call: `jsr ultimate_http_body_int`

Inputs

`ult_body` is a handle; `ult_url` points to a key, or to an empty string for an array element; `ult_val` is a signed 32-bit integer.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Adds one integer value.

Example

```
setptr ult_url,key
setdword ult_val,-6502
jsr ultimate_http_body_int
```

RESULT

The keyed value is added to the current container.

ultimate_http_body_bool

Assembly routine

Call: `jsr ultimate_http_body_bool`

Inputs

`ult_body` is a handle; `ult_url` points to a key, or to an empty string for an array element; `ult_val`=0 means false and any nonzero value means true.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Adds one Boolean value.

Example

```
setptr ult_url,key
setdword ult_val,1
jsr ultimate_http_body_bool
setdword ult_val,0
jsr ultimate_http_body_bool
```

RESULT

The calls add true and false respectively.

ultimate_http_body_string

Assembly routine

Call: `jsr ultimate_http_body_string`

Inputs

`ult_body` is a handle; `ult_url` points to a key, or to an empty string for an array element; `ult_buf` points to a NUL-terminated ASCII value.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Adds one escaped string value.

Example

```
setptr ult_url,key
setptr ult_buf,value
jsr ultimate_http_body_string
```

RESULT

The string is added to the current container.

ultimate_http_body_object

Assembly routine

Call: `jsr ultimate_http_body_object`

Inputs

`ult_body` is a handle; `ult_url` points to a key, or to an empty string for an array element.

Outputs

`A` is a result code; success enters the new object.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Adds and enters a nested JSON object.

Example

```
setptr ult_url,key
jsr ultimate_http_body_object
```

RESULT

Call `ultimate_http_body_up` to leave it.

ultimate_http_body_array

Assembly routine

Call: `jsr ultimate_http_body_array`

Inputs

`ult_body` is a handle; `ult_url` points to a key, or to an empty string for an array element.

Outputs

`A` is a result code; success enters the new array.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Adds and enters a nested JSON array.

Example

```
setptr ult_url,key
jsr ultimate_http_body_array
```

RESULT

Point `ult_url` at a NUL byte for each unkeyed array element.

ultimate_http_body_binary

Assembly routine

Call: `jsr ultimate_http_body_binary`

Inputs

`ult_body` is a binary body; `ult_buf` points to bytes and `ult_buflen` is their count.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Appends raw bytes to a binary request body.

Example

```
setptr ult_buf,payload
setword ult_buflen,payload_end-payload
jsr ultimate_http_body_binary
```

RESULT

The payload bytes are appended without text conversion.

LOW-LEVEL UCI TRANSPORT

Use these entries for commands that the service layer does not wrap. The `uci_request` layout and `UCI_REQ_*` offsets are generated in `uci_protocol.inc`. A request's argument, payload, data and status spans remain caller-owned. `uci_more` is the exported byte read after a boundary-preserving call; nonzero means another reply block follows.

uci_present

Assembly routine

Call: `jsr uci_present`

Inputs

None.

Outputs

`A` is 1 when the documented signature is present, otherwise 0.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Performs a fast register-only presence probe.

Example

```
jsr uci_present
beq absent
```

RESULT

This does not prove that the interface can finish a command.

uci_ident

Assembly routine

Call: `jsr uci_ident`

Inputs

None.

Outputs

`A` is the raw identification register.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads diagnostic identity without blocking.

Example

```
jsr uci_ident
sta ident
```

RESULT

`ident` contains the unvalidated register byte.

uci_req_clear

Assembly routine

Call: `jsr uci_req_clear`

Inputs

None.

Outputs

No defined register result; all `UCI_REQ_SIZE` bytes of `uci_req` are zero.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Clears the SDK-owned request block before selected fields are filled.

Example

```
jsr uci_req_clear
```

> **RESULT**
>
> No stale pointer or length remains in `uci_req`.

uci_exec_block

Assembly routine

Call: `jsr uci_exec_block`

Inputs

Fill the exported `uci_req` block.

Outputs

`A` is a result code; output lengths are written into the block.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Runs the SDK-owned request block to completion.

Example

```
jsr uci_req_clear
lda #UCI_TARGET_DOS1
sta uci_req+UCI_REQ_TARGET
jsr uci_exec_block
```

> **RESULT**
>
> The request completes and all reply blocks are drained.

uci_exec

Assembly routine

Call: `jsr uci_exec`

Inputs

`A/X` points to any writable `UCI_REQ_SIZE` request block.

Outputs

`A` is a result code; output lengths are written into that block.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Runs a caller-owned request block to completion.

Example

```
lda #<request
ldx #>request
jsr uci_exec
```

RESULT

The caller may keep several independent request blocks.

uci_exec_first

Assembly routine

Call: `jsr uci_exec_first`

Inputs

`A/X` points to a writable request block.

Outputs

`A` is a result for the first reply block; `uci_more` describes whether another follows.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Starts a boundary-preserving exchange such as a directory walk.

Example

```
lda #<request
ldx #>request
jsr uci_exec_first
```

RESULT

Only the first logical reply block is retained.

uci_exec_next

Assembly routine

Call: `jsr uci_exec_next`

Inputs

A boundary-preserving exchange is live and `uci_more` is nonzero.

Outputs

`A` is the next block's result; `uci_more` is updated.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Advances the live exchange by one reply block.

Example

```
lda uci_more
beq finished
jsr uci_exec_next
```

RESULT

The request block's data length is reset for this block.

uci_abort

Assembly routine

Call: `jsr uci_abort`

Inputs

None.

Outputs

`A` is a result code.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Abandons any live exchange and returns the interface to idle.

Example

```
jsr uci_abort
```

> **RESULT**
>
> A stopped directory walk no longer holds the interface.

uci_set_timeout_a

Assembly routine

Call: `jsr uci_set_timeout_a`

Inputs

`A` is a budget in units of 256 polls; zero means forever.

Outputs

No defined register result.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Sets the transport polling budget. Wall time varies with CPU speed.

Example

```
lda #UCI_TIMEOUT_DEFAULT
jsr uci_set_timeout_a
```

> **RESULT**
>
> Later bounded calls use the default budget.

uci_get_timeout_a

Assembly routine

Call: `jsr uci_get_timeout_a`

Inputs

None.

Outputs

`A` is the current timeout budget; zero means forever.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Reads the current polling budget.

Example

```
jsr uci_get_timeout_a
sta saved_timeout
```

RESULT

The byte can restore caller policy after a temporary change.

uci_last_code

Assembly routine

Call: `jsr uci_last_code`

Inputs

None.

Outputs

`A/X` is the most recently decoded raw device status, or $FFFF when none was supplied.

Clobbers

Registers not listed as outputs are undefined; processor flags and the four bytes at `UCI_ZP` may change.

Purpose

Returns target-specific diagnostic detail.

Example

```
jsr uci_last_code
sta device_code
stx device_code+1
```

RESULT

Interpret the number only in the context of the target just called.

CHAPTER 9

APPENDIX

- Result codes

RESULT CODES

0	ULTIMATE_OK	success
1	ULTIMATE_ERR_NO_DEVICE	command-interface signature absent
2	ULTIMATE_ERR_TIMEOUT	polling budget exhausted
3	ULTIMATE_ERR_PROTOCOL	malformed or inconsistent transport reply
4	ULTIMATE_ERR_NOT_SUPPORTED	target, command or direct hardware unavailable
5	ULTIMATE_ERR_INVALID_ARGUMENT	caller supplied an invalid argument
6	ULTIMATE_ERR_IO	filesystem, network or transfer failure
7	ULTIMATE_ERR_DEVICE	target reported a failure
8	ULTIMATE_ERR_TRUNCATED	retained buffer did not hold the whole reply
9	ULTIMATE_ERR_ABORTED	exchange was abandoned
10	ULTIMATE_END	directory or socket stream ended normally

`ultimate_strerror()` returns a printable description. `ultimate_device_code()` returns the last target's raw numeric status for diagnostics; its meaning depends on the target.

Index

UBLOAD (BASIC statement), 109
UBYTE((BASIC numeric function), 104
UCI (BASIC statement), 100
uci_abort() (C function), 176
uci_abort (Assembly routine), 239
uci_decode_status() (C function), 178
uci_exec() (C function), 174
uci_exec (Assembly routine), 237
uci_exec_block (Assembly routine), 237
uci_exec_first() (C function), 174
uci_exec_first (Assembly routine), 238
uci_exec_next() (C function), 175
uci_exec_next (Assembly routine), 239
uci_get_timeout() (C function), 177
uci_get_timeout_a (Assembly routine), 240
uci_ident (Assembly routine), 236
uci_ident_register() (C function), 173
uci_init() (C function), 173
uci_last_code (Assembly routine), 241
uci_last_device_code() (C function), 177
uci_more_blocks() (C function), 175
uci_present (Assembly routine), 235
uci_req_clear (Assembly routine), 236
uci_request (C structure), 121
uci_set_timeout() (C function), 176
uci_set_timeout_a (Assembly routine), 240
uci_signature_present() (C function), 172
uci_status_format() (C function), 178
UCTRL (BASIC numeric constant), 106
UDAT$ (BASIC string function), 103
UDEV (BASIC numeric function), 102
UDIR (BASIC statement), 110

UDOS1 (BASIC numeric constant), 105
UDOS2 (BASIC numeric constant), 106
UERR (BASIC numeric function), 101
UFETCH (BASIC statement), 112
UHTTP (BASIC numeric constant), 107
UIEC (BASIC numeric constant), 107
UL((BASIC UCI argument modifier), 105
ULEN (BASIC numeric function), 102
ULOAD (BASIC statement), 109
ultimate_audio_available() (C function), 150
ultimate_audio_available (Assembly routine), 211
ultimate_audio_configure() (C function), 151
ultimate_audio_configure (Assembly routine), 212
ultimate_audio_init() (C function), 149
ultimate_audio_init (Assembly routine), 210
ultimate_audio_irq_clear() (C function), 153
ultimate_audio_irq_clear (Assembly routine), 214
ultimate_audio_irq_status() (C function), 152
ultimate_audio_irq_status (Assembly routine), 214
ultimate_audio_load_wav() (C function), 153
ultimate_audio_load_wav (Assembly routine), 215
ultimate_audio_start() (C function), 151
ultimate_audio_start (Assembly routine), 213
ultimate_audio_stop() (C function), 152
ultimate_audio_stop (Assembly routine), 213
ultimate_audio_version() (C function), 150
ultimate_audio_version (Assembly routine), 211
ultimate_audio_voice (C structure), 119
ultimate_available() (C function), 122
ultimate_available (Assembly routine), 182
ultimate_bload() (C function), 144
ultimate_bload (Assembly routine), 204
ultimate_capabilities (C structure), 115
ultimate_chdir() (C function), 133
ultimate_chdir (Assembly routine), 191
ultimate_close() (C function), 136
ultimate_close (Assembly routine), 194
ultimate_copy() (C function), 140
ultimate_copy (Assembly routine), 199
ultimate_delete() (C function), 138
ultimate_delete (Assembly routine), 196
ultimate_detect() (C function), 123
ultimate_detect (Assembly routine), 182

ultimate_device_code() (C function-like macro), 171
ultimate_drive (C structure), 120
ultimate_drive_enable() (C function), 155
ultimate_drive_enable (Assembly routine), 217
ultimate_drive_info() (C function), 156
ultimate_drive_info (Assembly routine), 218
ultimate_drive_power() (C function), 155
ultimate_drive_power (Assembly routine), 217
ultimate_drives (C structure), 120
ultimate_fileinfo (C structure), 118
ultimate_freeze() (C function), 154
ultimate_freeze (Assembly routine), 216
ultimate_fstat() (C function), 139
ultimate_fstat (Assembly routine), 197
ultimate_get_model() (C function), 124
ultimate_get_model (Assembly routine), 183
ultimate_get_time() (C function), 143
ultimate_get_time (Assembly routine), 202
ultimate_getpath() (C function), 134
ultimate_getpath (Assembly routine), 192
ultimate_has_control() (C function-like macro), 127
ultimate_has_dos() (C function-like macro), 125
ultimate_has_dos2() (C function-like macro), 126
ultimate_has_http() (C function-like macro), 128
ultimate_has_network() (C function-like macro), 126
ultimate_has_softiec() (C function-like macro), 127
ultimate_has_target() (C function-like macro), 125
ultimate_home() (C function), 141
ultimate_home (Assembly routine), 200
ultimate_http_body() (C function), 165
ultimate_http_body (Assembly routine), 229
ultimate_http_body_array() (C function), 170
ultimate_http_body_array (Assembly routine), 234
ultimate_http_body_binary() (C function), 170
ultimate_http_body_binary (Assembly routine), 234
ultimate_http_body_bool() (C function), 168
ultimate_http_body_bool (Assembly routine), 232
ultimate_http_body_clear() (C function), 166
ultimate_http_body_clear (Assembly routine), 230
ultimate_http_body_free() (C function), 166
ultimate_http_body_free (Assembly routine), 230
ultimate_http_body_int() (C function), 167
ultimate_http_body_int (Assembly routine), 231

ultimate_http_body_object() (C function), 169
ultimate_http_body_object (Assembly routine), 233
ultimate_http_body_string() (C function), 168
ultimate_http_body_string (Assembly routine), 233
ultimate_http_body_up() (C function), 167
ultimate_http_body_up (Assembly routine), 231
ultimate_http_close() (C function), 164
ultimate_http_close (Assembly routine), 228
ultimate_http_exchange() (C function), 164
ultimate_http_exchange (Assembly routine), 227
ultimate_http_free_all() (C function), 165
ultimate_http_free_all (Assembly routine), 228
ultimate_http_get() (C function), 162
ultimate_http_get (Assembly routine), 225
ultimate_http_header() (C function), 163
ultimate_http_header (Assembly routine), 226
ultimate_http_open() (C function), 162
ultimate_http_open (Assembly routine), 226
ultimate_identify() (C function), 123
ultimate_identify (Assembly routine), 183
ultimate_init() (C function), 122
ultimate_init (Assembly routine), 181
ultimate_last_end() (C function), 145
ultimate_last_end (Assembly routine), 205
ultimate_legacy_get_sid_info() (C function), 124
ultimate_legacy_get_sid_info (Assembly routine), 184
ultimate_load() (C function), 144
ultimate_load (Assembly routine), 203
ultimate_mkdir() (C function), 140
ultimate_mkdir (Assembly routine), 199
ultimate_mount() (C function), 141
ultimate_mount (Assembly routine), 200
ultimate_net_close() (C function), 160
ultimate_net_close (Assembly routine), 223
ultimate_net_connect() (C function), 159
ultimate_net_connect (Assembly routine), 222
ultimate_net_ifcount() (C function), 157
ultimate_net_ifcount (Assembly routine), 219
ultimate_net_ipconfig() (C function), 158
ultimate_net_ipconfig (Assembly routine), 221
ultimate_net_macaddr() (C function), 157
ultimate_net_macaddr (Assembly routine), 220
ultimate_net_read() (C function), 160

ultimate_net_read (Assembly routine), 224
ultimate_net_setip() (C function), 158
ultimate_net_setip (Assembly routine), 221
ultimate_net_udp() (C function), 159
ultimate_net_udp (Assembly routine), 223
ultimate_net_write() (C function), 161
ultimate_net_write (Assembly routine), 224
ultimate_open() (C function), 135
ultimate_open (Assembly routine), 193
ultimate_opendir() (C function), 134
ultimate_opendir (Assembly routine), 192
ultimate_palette_get() (C function), 128
ultimate_palette_get (Assembly routine), 185
ultimate_palette_reset() (C function), 130
ultimate_palette_reset (Assembly routine), 187
ultimate_palette_set() (C function), 129
ultimate_palette_set (Assembly routine), 186
ultimate_palette_set_color() (C function), 129
ultimate_palette_set_color (Assembly routine), 187
ultimate_ramdisk_info() (C function), 156
ultimate_ramdisk_info (Assembly routine), 219
ultimate_read() (C function), 136
ultimate_read (Assembly routine), 195
ultimate_readdir() (C function), 135
ultimate_readdir (Assembly routine), 193
ultimate_reboot() (C function), 154
ultimate_reboot (Assembly routine), 216
ultimate_rename() (C function), 139
ultimate_rename (Assembly routine), 198
ultimate_reu_available() (C function), 146
ultimate_reu_available (Assembly routine), 206
ultimate_reu_fetch() (C function), 147
ultimate_reu_fetch (Assembly routine), 208
ultimate_reu_load() (C function), 148
ultimate_reu_load (Assembly routine), 209
ultimate_reu_save() (C function), 148
ultimate_reu_save (Assembly routine), 209
ultimate_reu_size() (C function), 146
ultimate_reu_size (Assembly routine), 207
ultimate_reu_stash() (C function), 147
ultimate_reu_stash (Assembly routine), 207
ultimate_save() (C function), 145
ultimate_save (Assembly routine), 205

ultimate_seek() (C function), 137
ultimate_seek (Assembly routine), 196
ultimate_set_time() (C function), 143
ultimate_set_time (Assembly routine), 203
ultimate_sid (C structure), 116
ultimate_sid_info (C structure), 116
ultimate_stat() (C function), 138
ultimate_stat (Assembly routine), 197
ultimate_strerror() (C function), 171
ultimate_strerror (Assembly routine), 185
ultimate_swap() (C function), 142
ultimate_swap (Assembly routine), 202
ultimate_turbo_available() (C function), 130
ultimate_turbo_available (Assembly routine), 188
ultimate_turbo_badlines() (C function), 132
ultimate_turbo_badlines (Assembly routine), 190
ultimate_turbo_get() (C function), 131
ultimate_turbo_get (Assembly routine), 188
ultimate_turbo_set() (C function), 131
ultimate_turbo_set (Assembly routine), 189
ultimate_unmount() (C function), 142
ultimate_unmount (Assembly routine), 201
ultimate_vsprite (C structure), 117
ultimate_vsprite_draw() (C function), 132
ultimate_vsprite_draw (Assembly routine), 190
ultimate_write() (C function), 137
ultimate_write (Assembly routine), 195
UNET (BASIC numeric constant), 106
UREU (BASIC numeric function), 112
USAVE (BASIC statement), 110
UST$ (BASIC string function), 103
USTASH (BASIC statement), 111
UTURBO (BASIC statement and numeric function), 108
UW((BASIC UCI argument modifier), 104

ONE LIBRARY, FOUR WAYS IN

Use files, REU memory, PCM audio, software vsprites, the live palette, sockets and HTTP from a C64 program without implementing the command-interface handshake.

BASIC, cc65 C, ca65 assembly and the standalone jump table all reach the same 6502 implementation.

- BASIC V2 keywords at the `READY.` prompt
- A cc65 library and public headers
- A documented ca65 calling convention
- A standalone jump table for other toolchains

```
READY.
PRINT UREU
 32
READY.
```

This expression reads the enabled REU size in 64K banks; 32 means 2MB.

FOR THE 1541 ULTIMATE-II, THE ULTIMATE 64 AND THE COMMODORE 64 ULTIMATE

FIRST EDITION · MIT LICENCE